

Collision Geometry Is Linear

Diophantine Certificates for Rational Signal Machines and Conservative Particle Dynamics

Event-sparse chambers, canonical quadratic and quartic certificates,
two routes to complete chronology, a conservative universal frontend,
and sparse lattice certificates

AI-assisted research report prepared for Vladimir Reshetnikov from four manuscripts
(ProveIt’s incoming reports: batch 78, manuscripts 07 and 11; batch 79, manuscripts 16 and 19), 2
October 2026

Abstract

A rational signal machine computes by the collisions of finitely many kinds of particles moving at fixed rational speeds on a line. Its first two Parts print two independent manuscripts that prove one theorem by two routes: once a complete finite collision history is fixed, its realizations are exactly the natural zeros of a sum of squares of integer affine residuals, of degree two, with exactly one witness tuple for each realization. In both, typed endpoint conditions on maximal adjacency intervals exclude unrecorded earlier collisions, omitted simultaneous inputs and omitted simultaneous sites, and causal depth controls the denominators of the event coordinates.

Part I (source 07) eliminates every event coordinate. The realizing initial-gap vectors form a relatively open rational polyhedral cone cut out by at most $3(n-1) + (3m+1)E$ linear conditions, and the only witnesses are the slacks of the strict rows. The chamber geometry yields small integral realizations, an NP upper bound for bounded synthesis, a robustness–codimension criterion, a canonical first-hit quartic for a fixed event budget, exact quasi-polynomial seed counts, an exactly solved first-collision tournament and a four-speed accumulating clock with exponentially growing denominators. Part II (source 11) keeps batch times and event positions as natural witnesses on a fixed rational grid, adds halting semantics, proves that the witness matrix has full column rank (so the certificate is strictly convex in its witnesses), gives linear witness and residual ledgers and a full-layer reference construction, and bounds the canonical witnesses by $60^K \max(1, M_0)$ for the four-speed interface of Durand-Lose’s universal machine.

Parts III and IV (sources 12 and 13, from batch 79) certify conservative dynamics at a fixed horizon. Part III writes out a reversible number-conserving universal signal machine (Morita’s universal Turing machine through Durand-Lose’s stacks) with 18 live signals, whose event dynamics is an 18-dimensional integer-linear map with a canonical quadratic step certificate; Part IV compiles one-dimensional number-conserving lattice dynamics into a quartic sum of squares with a unique natural witness and proves decidability at total mass at most two.

None of the four Parts compiles unbounded halting into one fixed-arity polynomial or improves the repository’s 75- and 87-operation bounds; Parts I and II do not transcribe the published universal machine, and Part III transcribes a different one. Re-derived devices are printed as pointers or second routes. Part I’s convex-quadratic barrier is a special case of Part XI of the collection’s report *canonical-diophantine-certificates*. The report is AI-assisted and unrefereed; nothing in it is formalized in Lean or Rocq.

Status. Conventional proofs and finite exact-arithmetic checks; not kernel-checked. Section 1 compares the two routes, introduces Parts III and IV, and states what is not claimed; Appendix E gives the provenance.

Contents

1	Scope, the two routes, and conventions	7
1.1	The two sources	7
1.2	One theorem by two routes	7
1.3	Notions and letters	9
1.4	Relation to the repository and to the literature	11
1.5	Status and what is not claimed	12
1.6	Research questions	13
1.7	Parts III and IV: conservative frontends (batch 79)	14
1.8	Notions and letters of Parts III and IV	15
1.9	Relation of Parts III and IV to the repository	17
1.10	Status and what Parts III and IV do not claim	18
1.11	Research questions answered or touched by Parts III and IV	19
I	Collision geometry is linear (source 07)	19
2	The question and the contribution	20
2.1	A concrete gap between universality and arithmetic	20
2.2	Results and their logical scope	21
3	Rational signal machines and chronological skeletons	21
3.1	Particles, velocities, and collision rules	21
3.2	Initial coordinates and normalization	22
3.3	Batches and stable signal identities	22
4	Eliminating every event coordinate	23
4.1	Intercept propagation	23
4.2	Local incidence does not imply a real computation	24
5	A sparse certificate for complete collision chronology	24
5.1	Maximal adjacency intervals	24
5.2	Soundness and completeness	25
5.3	The event-sparse bound	26
6	Quadratic Diophantine certificates with unique slacks	27
6.1	Positive denominator clearing	27
6.2	What uniqueness does not assert	28
7	Causal depth controls denominators and coefficient size	28
7.1	A denominator invariant	28

7.2	An explicit coefficient bound	29
8	Small integral realizations and finite-horizon synthesis	29
9	Robustness and the rank of synchronization constraints	30
10	Finite-horizon acceptance, semilinearity, and counting	31
10.1	Disjoint first-acceptance chambers	31
10.2	A single quartic for an unknown bounded first-hit skeleton	32
10.3	Exact quasi-polynomials rather than merely leading asymptotics	33
11	An exact first-collision tournament	34
11.1	Three chambers and a complete polynomial	34
11.2	Counting, generating function, and inversion	35
11.3	Independent simultaneous sites	36
12	A four-speed clock with exponentially growing denominators	36
12.1	Rules and a closed-form execution	36
12.2	A denominator lower bound without fragile synchronization	37
13	Why a fixed universal convex quadratic cannot follow	38
13.1	The classical unbounded existence result remains available	39
14	Implementation, reproducible checks, and trust boundaries	40
14.1	What the attached code computes	40
14.2	Run and reproduce	40
14.3	An auditable resource ledger	41
14.4	What has and has not been verified	42
15	Further research questions and a concrete development path	42
15.1	Formalizing the chronology theorem	42
15.2	An audited universal signal-machine frontend	43
15.3	Sharper denominator invariants	43
15.4	From few rows to small arithmetic circuits	44
15.5	Avoiding exhaustive first-hit enumeration	44
15.6	Robust universality and minimal synchronization rank	44
15.7	Counting complexity and asymptotic structure	44
15.8	Algebraic velocities and nonlinear motion	45
15.9	Accumulation limits without fictitious infinite witnesses	45
16	Conclusion	45

II	Direct Diophantine certificates (source 11)	45
17	Purpose and relation to prior work	46
18	Model and exact theorem	47
19	Rational normalization and denominator control	48
19.1	Normalization is an external interface	48
19.2	Causal depth and a fixed grid	49
20	The sparse construction	49
20.1	Geometry variables and flight equations	49
20.2	Maximal adjacency lifetimes	50
20.3	Typed endpoint constraints	51
21	Proof of geometric correctness	51
21.1	Why endpoint types suffice	51
21.2	Soundness and completeness	52
22	Canonical witnesses and strict convexity	53
23	A full-layer reference construction	54
24	Worked examples	54
24.1	One annihilation with its complete quadratic	54
24.2	A hidden collision exactly at the final cut	55
24.3	An outgoing fan at a finite cut	55
25	Witness height for the four-speed interface	55
25.1	Exponential denominator growth can be necessary	56
26	Fixed-schema arithmetic and the unbounded boundary	57
26.1	Strictly positive witnesses	58
27	Verification evidence and research questions	58
27.1	Questions with concrete next steps	59
28	Conclusion	60
III	Conservative signal machines as Diophantine frontends (source 12)	60
29	Results and scope	61

30 Primary sources and the literal interpreter	62
30.1 The source machine and its terminal convention	62
30.2 The finite input language	63
30.3 What is imported and what is checked	63
31 Finite-support stacks with a fixed number of particles	63
31.1 Exact rational encoding	63
31.2 The complete parametric collision gadget	64
31.3 Geometric proof of push and pop	65
32 Reversible control and the fixed population	65
32.1 Alternating write and move form	65
32.2 Layout and tape invariant	66
32.3 Terminal exits and conservation	66
32.4 No accumulation on encoded runs	67
32.5 The ten-signal existence specialization	67
33 The exact input loader and its cost	67
33.1 Both finite tape halves	67
33.2 Arithmetic and representation cost	68
34 The complete simultaneous-event normal form	68
34.1 Post-event germs and first-collision guards	68
34.2 Why no coordinate permutation is needed	69
35 Integer lifting and internalized finite control	70
35.1 The pivot-scaled integer lift	70
35.2 A uniform alternative for comparison	71
35.3 Control uses one coordinate	71
35.4 Limits of the normal form	72
36 A direct canonical quadratic Diophantine packet	73
36.1 Finite linear branch systems	73
36.2 The exact witness and residual ledger	75
36.3 Fixed horizons and terminal reachability	76
37 Exact replay and the release contract	76
37.1 Literal machine counts	76
37.2 Replays and their meaning	76
37.3 Reproduction and boundaries	77

IV Sparse Diophantine certificates for finite mass lattice dynamics (source 13)	79
38 Results and model	79
38.1 Canonical records and fixed parameters	80
39 The canonical arithmetic layer	81
39.1 Comparison and equality without ambiguous ties	81
39.2 Streaming and all simultaneous collision groups	81
39.3 A single table-row selector simplex	82
39.4 Rank redistribution	82
39.5 A fixed sorting network	83
40 Correctness and exact paid costs	83
40.1 Scalar ledger	84
40.2 Restricted row lists	84
40.3 The domain of uniqueness and an orthant-exact option	85
41 The reversible conservative source interface	85
41.1 Source syntax and encoding	86
41.2 The printed sign defect and the repaired local injection	86
41.3 Exact source microtimes and the pulse observer	87
41.4 An explicit false observer on an unsafe source	88
42 Safe universal sources and the paid loader	89
42.1 The existence argument and its exact scope	89
42.2 A uniform initial loader with no hidden input oracle	90
42.3 Paying for the exact first pulse	90
42.4 The fully specified doubling example	91
43 Geometric barriers and translating periodicity	91
43.1 Finite rings and global halting	93
44 Decidability at total mass at most two	94
44.1 Single particles are finite-state walkers	94
44.2 Exact acceleration to the next encounter	94
44.3 Only finitely many encounter shapes	95
44.4 Testing every intermediate time	95
44.5 Why the fixed-rule algorithm is polynomial	96
44.6 What particle bounds from the literature do and do not say	97
45 Horizon-free semilinearity at mass at most two	97

45.1	A linear guard for the entire independent prefix	97
45.2	First entry with the absolute position preserved	98
45.3	Finite rule-fixed orbit templates after entry	98
45.4	Effective elimination and canonical signed input	99
46	Canonical fixed-arity quartics for the low-mass relation	99
46.1	Inequalities and congruences have unique arithmetic witnesses	100
46.2	Boolean gates and the exact ledger	100
47	Reproduction and evidence	101
47.1	What each layer of verification establishes	101
47.2	Selected literal fixtures	102
47.3	Audit coverage	102
48	Further questions and boundaries	104
A	Part I: a minimal worked use of the compiler	105
B	Part I: dependency map and claim status	106
C	Part II: replay and build guide	106
D	Part II: claim and source guide	107
E	Provenance of this report	108
F	Part III: the complete Morita transition data	112
G	Part III: a small complete quadratic packet	112
H	Part III: the complete finite mode closure and numeric packet	113
H.1	A finite family covering every encoded run	113
H.2	Exact guard and witness counts	114
H.3	What the numeric export contains	114
	References	116

1 Scope, the two routes, and conventions

[write] This section was written when the two manuscripts were merged. Part I and Part II print the two sources; text there without a [write] marker is the source’s own. Sections 1.7–1.11 were added on 2 October 2026, when two manuscripts of batch 79 became Parts III and IV; where they make a sentence of the earlier subsections stale, a dated note says so.

1.1 The two sources

This report is built from two AI-assisted research manuscripts that arrived in one delivery as batch 78 of ProveIt’s incoming reports (manuscripts 07 and 11; Appendix E gives the full provenance). They are called *source 07* and *source 11* after their batch-78 manuscript numbers, which are also the file prefixes of their shipped programs and data. Neither cites the other, and a text-overlap check at placement found almost no shared wording (at most 0.5 per cent of either text’s eight-word sequences occur in the other). They nevertheless prove the same theorem, by the same mechanism, in two different witness formats (Section 1.2).

- **Part I (source 07, the base), *Collision geometry is linear*.** Arbitrary rational speeds, of either sign. Every event time and position is eliminated as a rational linear form in the initial gaps, so a complete chronological skeleton becomes a homogeneous rational chamber (Theorem 5.3) and a convex quadratic with unique slack witnesses (Theorem 6.1). The chamber geometry then gives denominator and height bounds, small integral realizations, an NP upper bound, robustness and codimension, finite-horizon semilinearity, a canonical first-hit quartic, exact Ehrhart counts, two exact examples and a convex-quadratic barrier.
- **Part II (source 11), *Direct Diophantine certificates*.** Speeds and positions normalized to natural numbers by an external Galilean change of coordinates. Batch times and event positions stay in the certificate as natural witnesses on the grid $B = D^K$, and the certificate has prefix and halting versions (Theorem 18.3). Its additions are the halting semantics with terminal rays, full column rank of the witness matrix and hence strict convexity in the witnesses (Lemma 22.1), an explicit witness and residual ledger, a full-layer reference construction with exact counts (Section 23), the witness-height bound for Durand-Lose’s four-speed interface (Theorem 25.2) and a positive-witness translation.

Source 07 is the base because it assumes less (signed rational speeds, no normalization interface) and proves more about the chamber; source 11 is printed in full as the second route together with its additions. Each Part keeps its source’s definitions, letters and numbering of statements. ([write] Added 2 October 2026: two further Parts, from batch 79, follow Part II; Section 1.7 describes them. They do not prove the common theorem of this subsection and the next again.) The common theorem is stated in both Parts, because its two statements have different inputs and witnesses; where a lemma of Part II coincides with one of Part I, a [write] note at the lemma says so, and source 11’s proof is kept as a marked second route.

1.2 One theorem by two routes

The common theorem. Fix a rational signal machine, an initial label word and a finite chronological collision history (a *skeleton* in Part I, a *schema* in Part II). There is an effective sum of squares of integer affine residuals, in the input coordinates and natural witnesses, whose natural

zeros are exactly the realizations of that history, with exactly one witness tuple for each realization; its degree is at most two. This is Theorems 5.3 and 6.1 of Part I and Theorem 18.3 of Part II (for prefix semantics).

The common mechanism. Both sources record each pair of adjacent signals once, over the maximal interval during which it stays adjacent, and impose a *typed* condition at each end of that interval: equality exactly when both segments are born (or die) at the same collision there, and strict positivity otherwise, including for a pair that survives the last listed batch. Affineness of the gap then excludes every interior meeting (Part I, Lemma 5.2; Part II, Lemma 21.1), and the strict conditions at a batch boundary make the declared blocks exactly the collision sites (the soundness part of the proof of Theorem 5.3; Lemma 21.2). The two sources then prove the same auxiliary statements: the lifetime count $(n - 1) + \sum_e (q_e + 1)$ ((5.2) and (18.2), where Part II writes $|\text{out}(e)|$ for q_e), causal-depth denominators dividing D^h (Theorem 7.1, Lemma 19.1), and semilinearity of the inputs accepted under a fixed event bound (Theorem 10.1, Proposition 26.1). Table 1 compares the two formats.

Table 1: The two routes compared.

	Part I (source 07)	Part II (source 11)
Speeds	arbitrary rationals, of either sign; time rescaled by $\tau = t/L$ to make them integers (Section 3)	natural numbers, after the external change of coordinates (19.1) (a Galilean shift and two scalings)
Input	the initial gap vector g , translation removed; real in the chamber theorem, positive integers in the certificate	the absolute initial positions $\mathbf{p}$, natural numbers in increasing order
Event times and positions	eliminated: rational linear forms in g , computed data (Section 4)	witnesses: natural T_i and X_e , scaled to the grid $B = D^K$ (18.1)
Witnesses	one slack $z_j = G_j g - 1$ per retained strict row; s in all	batch times T_i , time slacks d_i , event positions X_e , input-order slacks u_j , endpoint slacks $h^\pm$; $W \leq E + 2K + 2L + n_-$
Size	at most $3(n - 1) + (3m + 1)E$ raw conditions (5.3); a row may be dense in g	$R \leq I + K + 2L + n_- \leq (r + 2s + 3)E + 3n_-$ residuals, each of bounded width for a fixed machine (18.3)–(18.4)
Denominators	D^h at an event of causal depth h , for the coefficients of its forms	D^h for coordinates of depth h ; the compiler clears with D^K , a multiple of D^H
Convexity	convex, Hessian $2A^\top A \succeq 0$; unique slack by (6.3)	strictly convex in the witnesses for fixed input: full column rank over $\mathbb{R}$, even for unrealizable schemas (Lemma 22.1)
Semantics	prefixes: exactly the first B batches; nothing after them	prefixes, and halting realizations certified by terminal rays
Heights	row coefficients at most $2d(1+V)(DK)^h$ (Proposition 7.3); a realizing seed of size at most $d^{d/2}A^d$ (Theorem 8.1)	every canonical input and witness at most $60^K \max(1, M_0)$ for speeds in $\{0, 2, 3, 4\}$ (Theorem 25.2)
Shuttle	four-speed clock, speeds $0, 2, -1, -2$; denominator exactly $2 \cdot 3^k$ at event $2k$ for the seed $(1, 2)$; Zeno time $S = u + v$ (Section 12)	bounded shuttle, speeds $3, 2, 4, 0$; denominator exactly 3^{k-1} at the k th left event; Zeno time 2 (Proposition 25.3)
Missed simultaneity	two independent sites, word P, N, P, N , gaps $(2, 5, 2)$ (Section 11.3)	hidden collision at the final cut, word d, a, d, a at $(0, 4, 10, 14)$ (Section 24.2)

	Part I (source 07)	Part II (source 11)
Only here	chamber geometry, small realizations, real = rational = integer realizability, NP bound, robustness and codimension, first-hit quartic, Ehrhart counts, the tournament, the convex-quadratic barrier	halting semantics, strict convexity, the full-layer construction and its exact counts, the four-speed height bound, positive witnesses

Normalizations. The two definitions of D agree. Part I multiplies all speeds by an integer L and leaves their signs; Part II subtracts a common speed c and multiplies by an integer m . A common shift does not change speed differences, so with $m = L$ both take D to be the least common multiple of the same nonzero integer differences. Part II needs nonnegative speeds and an origin because its inputs are natural absolute positions; Part I needs neither because its input is the gap vector. Part I’s integer seeds come from homogeneity (scaling a rational chamber point); Part II fixes the scale outside the witness tuple and warns that a scale witness or variable denominators would not preserve its canonical quadratic claim without a new argument (Section 19.1).

Two shuttles, one mechanism. Part I’s clock and Part II’s shuttle are both a particle reflected between two converging walls, each round trip shrinking the remaining time to the accumulation by a factor three, so that the reduced denominators grow exponentially in the number of events while their bit lengths grow linearly. They are different machines: Part I’s has a stationary left wall and a right wall of speed -1 , and its every finite prefix has the whole positive quadrant as chamber (robust chronology with exponentially growing precision); Part II’s lives entirely in $0 < t < 2$, $0 < x < 6$ and is stated for its natural-speed interface. Neither source gives the accumulation point a continuation. Source 07 credits the accumulation phenomenon to Becker et al. [6].

Two missed simultaneities, one failure mode. In both examples a proposed batch omits a second, spatially separate collision that happens at the same time, and in both the omitted pair is an unchanged adjacent pair whose gap reaches zero exactly at the last listed boundary. The strict upper endpoint of a surviving pair rejects it: in Part I through the chamber row that must be positive there, in Part II through the residual $G(T_1) - h^+ - 1 = -h^+ - 1$. Part II adds the observation that weaker conditions (two nonnegative endpoint gaps with positive sum) would accept the false schema.

Two meanings of “sparse”. In Part I “event-sparse” counts geometric conditions: the bound (5.3) is linear in n and E , but a propagated intercept may depend on every initial gap, so the rows can be dense and storing them can take $O(dR_\sigma)$ coefficients (the paragraph after Theorem 5.4). In Part II the sparse certificate also has residuals of bounded width, because event coordinates are witnesses; its coefficients then contain the grid constant $B = D^K$. *Neither* size statement is a constant arithmetic-operation count for an unbounded computation, as both sources say.

1.3 Notions and letters

Each Part keeps its source’s letters. Many letters carry different meanings in the two Parts; Table 2 lists every letter used in both, so that a statement of one Part is never read with the other Part’s meaning. Inside a Part the meaning is always the one in that Part’s column. No symbol was renamed and no normalization was changed: source 11’s macro for the natural numbers is called

\NN in this file and still prints $\mathbb{N}$, and Part I prints $\mathbb{N}_0$; both denote $\{0, 1, 2, \dots\}$. ([write] The letters of Parts III and IV are listed in Section 1.8.)

Table 2: Letters used in both Parts.

Letter	Part I (source 07)	Part II (source 11)
$\mathbb{N}_0, \mathbb{N}$	$\mathbb{N}_0 = \{0, 1, 2, \dots\}$	$\mathbb{N} = \{0, 1, 2, \dots\}$ (the same set)
A	coefficient bound (7.2); a matrix in proofs	witness coefficient matrix (Lemma 22.1); birth-position expression A_σ ; start coordinates $A_{i,j}$
B	number of batches; a nonsingular matrix in the proof of Theorem 8.1; a label of the tournament	the grid modulus $B = D^K$ (18.1), <i>not</i> a number of batches; a label of the shuttle
C	the chamber $\mathcal{C}$; the count $C(H)$; a label	matrix of input coefficients; end coordinates $C_{i,j}$
D	lcm of nonzero speed differences (7.1)	the same lcm (18.1) (Section 1.2)
d	$d = n - 1$	time slacks d_i ; a label
E	number of collision sites; $\mathbf{E}$ the equality matrix	number of events (a site of Part I)
G	$\mathbf{G}$ the strict-row matrix	the gap function $G_{\sigma\tau}(T)$ (20.5)
g	the initial gap vector	number of blocks g_i of a slab (Section 23); a gap in proofs
H, h	h causal depth; H the side of a counting box	h a depth bound, H the maximal depth; $h, h^\pm$ endpoint slacks
K	$K = 2 + 4V$ (Lemma 7.2); number of first-hit chambers	number of batches, <i>not</i> a number of chambers
L	integer clearing the speeds; a label of the clock	number of recorded lifetimes; a label of the shuttle
M	the label set	largest position M_j ; $\mathcal{M}$ the machine
m	number of distinct speeds	the speed multiplier of (19.1); number of generators of a linear set
n	number of initial signals	the same; $n_- = \max(n - 1, 0)$
p, q	p_e, q_e incoming and outgoing counts at site e	initial positions p_j ; the position scale q
R	raw row count R_σ ; a label	residual count; a label
r, s	numbers of retained equality and strict rows	largest incoming arity and largest fanout; residuals r_a
S	the accumulation time $u + v$ of the clock	a label; $\mathcal{S}$ the schema
T	the site budget	scaled batch times T_i
u, v	initial gaps of the examples; $v(a)$ a speed	input-order slacks u_j ; $u(\ell)$ an original speed, $v(\ell)$ a normalized speed
V	$\max(1, \max v(a))$	the maximal speed (remark after Theorem 25.2)
W	$W = 3u + 2v$ of the clock	number of witnesses
z, y	z the slack witnesses	$\mathbf{y}$ the witnesses; $\mathbf{z}$ a test vector

Three words need the same care.

- **Skeleton** (Part I) and **schema** (Part II) carry the same information: an ordered list of batches, each a set of disjoint contiguous blocks of the live list. A skeleton lists only the

collision blocks (at least two members); a schema partitions the whole live list and marks singletons as persisting signals. A *site* of Part I is an *event* of Part II.

- **Realization.** Part I’s realization and Part II’s prefix realization are the same notion: exactly the listed batches as the first batches, nothing asserted afterwards. Part II’s *halting* realization additionally excludes every future collision. The “halting relation” of Part I’s Section 13 is the undecidable halting of a universal machine, a different thing.
- **Convex.** Part I’s certificate is convex (positive semidefinite Hessian) jointly in seed and slacks; Part II’s is *strictly* convex in the witnesses with the input held fixed, and Part II states that joint strict convexity is neither needed nor generally true.

1.4 Relation to the repository and to the literature

Neither source cites the related material below; this subsection supplies the cross-references (pins and dates in Appendix E). Reports of the collection are cited by label name, since their labels do not resolve in this file.

- **Report *canonical-diophantine-certificates*, Part XI [11]**, “Order-free self-assembly and a positivity-restricted degree threshold”. Its Lemma `cdc:of:lem:affinezero`, Theorem `cdc:of:thm:cubicsemilinear` and Theorem `cdc:of:thm:classification` ($D_2^+(k) = D_3^+(k) = \text{SL}(k)$) prove, for polynomials of degree at most three that are nonnegative only on the real nonnegative orthant, that the full natural zero set is a finite union of natural solution sets of rational affine systems, hence semilinear and effectively so. Part I’s Theorem 13.1 and Corollary 13.2 are the globally nonnegative quadratic special case; they are printed with source 07’s direct proof, as a second route, and credited in Remark 13.3. No novelty is claimed for them. The same Part’s Theorem `cdc:of:thm:singlefolds1` (a single-fold orthant-nonnegative quadratic for every semilinear set) bears on Part I’s first-hit quartic; see Remark 10.3. The bounded-search argument at the end of Part I’s Section 13 is the standard one of Proposition `cdc:bd:prop:nobound` there, for a different statement (site counts of accepting runs).
- **Report *liveness-beyond-halting* [12]**. Its Part V prints a question of its source 11, “Optimal degree under stronger geometric restrictions”, which asks for the minimal degree and dimension of a witness-free, binary-branching local polynomial that is globally nonnegative on all real inputs, and remarks that the globally nonnegative quadratic case “has a much more rigid affine geometry”. Part I’s Theorem 13.1 (and, more generally, Part XI of [11]) states that geometry: such a quadratic has a rational affine zero set. This report does not settle that degree question.
- **The Hilbert-tenth-problem programme has already reviewed both manuscripts.** The review of the delivery of 2 October 2026 [13], with its detailed signal-machine part [14], replayed both suites from the original archives, read both articles in full and reports “PASS for both reports’ stated finite-schema, exact-domain theorems and generated examples”, with no mandatory correction. It also proves and checks two reductions that neither source claims: deleting source 11’s identically zero same-birth rows and its same-death rows (differences of retained flight rows), and erasing the initial-order slacks when $K > 0$ or the certificate is halting, which takes the worked annihilation from six residuals and five witnesses to four and four (a bijection of natural zero sets, not of nonnegative rational ones, with a rational counterexample); and deleting redundant strict rows, with computed slacks, from the five

exported packets of source 07. Those reductions are the review’s, are not re-proved here, and do not change any theorem of either Part. This report’s intake is therefore not a first review; it adds the merge, the cross-references and the credit of Remark 13.3.

- **The programme’s audits of finite-history models.** Source 07 cites the reversible finite-history audit [2] for its distinction between cheap local relations and a complete computation certificate; that file is unchanged from source 07’s pin to the write. A sibling audit [15] notes that “finite particles can in principle store unbounded values in their separation” and that “a new construction would have to establish that interface”. Durand-Lose’s halting universal signal machine [3] uses a finite initial configuration and a halting event (Part II, Section 17), so signal machines are a candidate for such an interface, and both Parts compile its finite collision histories in principle; but neither transcribes its universal rule table or its input loader (Part I, Section 14; Part II, Question 1), so this report does not establish that interface. ([write] Added 2 October 2026: for signal machines Part III now establishes it, with a different universal machine and a prepared rational loader; see Section 1.9.)
- **The repository’s universal bounds.** The project README [1] still states the established bounds of 75 operations for a complete certificate and 87 for one universal polynomial. Neither Part improves or bears on them; both say so. ([write] Nor do Parts III and IV.)
- **Other reports of the category.** No result of *probabilistic-quantum-and-continuous-computation* or *group-theoretic-substrates* is re-proved here. Apart from the programme’s review described above, nothing in the repository treated signal machines before this report.

1.5 Status and what is not claimed

The report is AI-assisted and unrefereed, and **nothing in it is formalized in Lean or Rocq**. Both sources give conventional proofs together with finite exact-arithmetic checks that are regression evidence, not proofs. Every limitation stated by either source is kept in its Part; collected here, neither Part claims

- historical priority, or exhaustive novelty, for its constructions (Part I, Section 2; Part II, Section 17); Part II adds that a general linearizability of signal-machine evolution is already in Durand-Lose’s linear real-register simulation [4];
- one fixed-arity polynomial for unbounded halting, a single-fold or finite-fold Diophantine representation of c.e. sets, or a fixed-universal arithmetic-operation bound: the skeleton or schema, the machine and the input length are compiler data, and the row and residual counts are not operation counts;
- that the published universal machine (13 meta-signals, 21 non-blank rules) or its input loader has been transcribed, executed or certified;
- any semantics at or after an accumulation point; the clock and the shuttle are tests of finite prefixes, and the clock is not a claim to have discovered accumulation;
- NP-hardness (Corollary 8.3 is an upper bound), an efficient enumeration of first-hit chambers or of their volumes, or an optimized exhaustive enumerator in the code;
- that the denominator-growth examples are lower bounds for arbitrary Diophantine representations: Part II’s is a lower bound for common-grid encodings;

- that integer scaling of a rational seed preserves a prescribed physical length or deadline;
- that the finite checks are exhaustive, or that the shared simulator of Part II’s two compilers is independently verified;
- any Lean or Rocq verification; neither source’s code is a verified compiler.

The report itself claims no novelty for Part I’s convex-quadratic barrier (Remark 13.3) and none for the reductions of the programme’s review. ([write] The non-claims of Parts III and IV are collected in Section 1.10.)

1.6 Research questions

Both agendas are printed in full: the nine research directions of Part I (Section 15) and the five questions of Part II (Section 27.1), each in its Part and with its source’s wording. Four of Part II’s questions coincide with, or overlap, directions of Part I; Table 3 pairs them, so that the two formulations are read together. None of them is answered by the other source, by the repository, or by the programme’s review; the one partial overlap is that Part II’s Question 3 begins with causal-depth clearing, which both sources already prove (a [write] note there re-scopes it). ([write] Added 2 October 2026: this sentence describes the report before batch 79. Part III now answers Part I’s direction 15.2, except for an ordinary-integer loader, and Parts III and IV answer Part II’s Question 2 in part; Section 1.11 and dated notes at the questions give the details.)

Table 3: Concordance of the research questions.

Part I (source 07)	Part II (source 11)	Relation
15.2 An audited universal signal-machine frontend	Q1 Instantiate the universal table and input map	the same task; [write] answered by Part III for a different universal machine, except the ordinary-integer loader; the 13-label table of Q1 is still not transcribed
15.1 Formalizing the chronology theorem	Q5 Formalize geometric completeness	the same target (endpoint lemma and sweep invariant; typed endpoints and atomic replacement)
15.3 Sharper denominator invariants; 15.4 From few rows to small arithmetic circuits	Q3 Sharpen the denominator and cost ledger	overlapping: prime-valuation denominators, and a literal gate ledger
15.5 Avoiding exhaustive first-hit enumeration	Q4 Compress histories without losing canonicity	overlapping: both separate bounded compression from fixed-arity unbounded compression
15.6, 15.7, 15.8, 15.9	—	Part I only
—	Q2 Exploit conservative bounded populations carefully	Part II only; [write] answered in part, at a fixed horizon, by Parts III and IV
—	—	[write] Part IV’s five questions (Section 48); Part III states none

1.7 Parts III and IV: conservative frontends (batch 79)

[write] This subsection and the four that follow were written on 2 October 2026, when two manuscripts of batch 79 of ProveIt’s incoming reports were added to the report as Parts III and IV. They are called *source 12* and *source 13* after the file prefixes of their shipped programs and data, which continue this report’s local sequence after 07 and 11. Their batch-79 manuscript numbers are different: **source 12 is batch-79 manuscript 16, and source 13 is batch-79 manuscript 19.** Appendix E gives the full provenance. Neither manuscript cites Parts I and II, the other new manuscript, or any repository file.

- **Part III (source 12), *Conservative signal machines as Diophantine frontends.*** A fixed reversible, number-conserving universal signal machine $\mathcal{U}_{18}$, written out completely: 114 meta-signals, 445 explicit injective two-to-two collision rules, speeds $\{-12, -9, -4, 0, 4, 9, 12\}$ and exactly 18 live signals on every encoded input (Theorem 29.1). Its source is Morita’s 15-state, 6-symbol reversible universal Turing machine, compiled through Durand-Lose’s 2012 conservative rational stacks; the input of an explicitly halting cyclic-tag program is loaded into two rational distances. At event cuts the complete simultaneous-event dynamics of any fixed-population, equal-cardinality machine is a partial piecewise homogeneous integer-linear map with the pivot-scaled gap matrices $N_j = c_j I - ce_j^T$, $N_j^2 = c_j N_j$, plus one mode coordinate (Sections 34–35). A selector/copy construction then gives one explicit quadratic polynomial with a unique natural witness for each legal step, and for each fixed horizon (Theorem 36.1); for $\mathcal{U}_{18}$ the step polynomial has exactly 4,196,998 auxiliary variables over a closed family of 49,700 modes and 80,501 branches (Appendix H). A ten-signal binary version is proved to exist but not printed (Theorem 29.2).
- **Part IV (source 13), *Sparse Diophantine certificates for finite mass lattice dynamics.*** The discrete counterpart: one-dimensional partitioned number-conserving cellular automata with three lanes. A conserved mass M bounds the number of unit records but not the gaps between them, and a fixed horizon T of any supplied total conservative table compiles into one quartic sum of squares with exactly one natural zero, paying for collision detection, simultaneous redistribution, sorting, table lookup and empty space, with exact witness and residual ledgers (Theorem 38.1) and an option that is exact on the nonnegative real orthant. A source interface to Morita and Imai’s reversible conservative automaton corrects a printed sign, adds a reachable-legality hypothesis that an explicit false observer shows to be necessary, pays for a loader and a first-pulse observer, and works a fully specified, nonuniversal doubling example. Decidability results bound what any such frontend can do: under a bounded orbit diameter, and without it at total mass at most two (Theorem 44.1); at mass at most two the relation is effectively semilinear (Theorem 45.1).

How they relate to Parts I and II. Neither new Part proves the common theorem of Section 1.2 again; both certify a step, or a fixed horizon, of a *deterministic* dynamics rather than the realizations of a prescribed collision history. Part III meets Parts I and II in one lemma: its complete event selection (Lemma 34.1) is their typed-endpoint argument for a single batch, and its uniform lift is their common-denominator device ([write] notes at both places). Part III is also the instantiated universal frontend that Part I’s research direction 15.2 asks for, though for a different universal machine than the one named there (Section 1.11). Part IV shares no lemma with Parts I–III; it answers, for lattice dynamics, the conservative-population question of Part II.

Two Morita sources, no shared data. Both new Parts draw their universal source from Morita, but from different constructions: Part III from Morita’s 2008 reversible universal Turing

machine UTM(15,6), through Durand-Lose’s signal-machine stacks; Part IV from Morita and Imai’s 2001 number-conserving reversible partitioned automaton, built on Morita’s 1996 reversible two-counter machine. They share no table, program or data file. Part III prints its universal machine in full; Part IV deliberately leaves its safe universal source as an existence construction and prints only a nonuniversal example.

1.8 Notions and letters of Parts III and IV

[write] Parts III and IV keep their sources’ letters and numbering of statements, by section, like Parts I and II; no symbol was renamed and no normalization changed. Both sources’ macro for the natural numbers is written `\NN` in this file, as for source 11, and prints $\mathbb{N} = \{0, 1, 2, \dots\}$; Part I’s $\mathbb{N}_0$ is the same set. The table lists every letter of Parts III and IV that has another meaning in the other new Part or in Parts I–II (Table 2 gives the meanings in Parts I and II in full). Inside a Part the meaning is always that Part’s.

Letter	Part III (source 12)	Part IV (source 13)	Parts I–II
A	auxiliary-variable count $A = B(d + 1) + T + W$; loader numerators A_L, A_R ; the projection A_j	—	coefficient bound; witness matrix
B	number of branches of the step map, <i>not</i> batches	the control baseline $B = m + 16$ of the source interface	batches (I); grid modulus D^K (II)
b	base $\ell + 1$; selectors b_r ; the blank $\mathbf{b}$	comparison flag b ; output masses b_i ; baselines b_i	—
C	the set C_ℓ of stack codes	centre lane C ; number C of congruence atoms	chamber (I); input matrix (II)
c	gadget speed c ; speed-difference vector c , $c_i = v_i - v_{i+1}$; pivot speed c_j	output masses c_i ; a configuration c	—
D	lcm of the nonzero speed differences, <i>the same</i> D as in Parts I–II	the diameter of the initial support	lcm of speed differences
d	dimension of a branch system; a move direction	comparison slack d ; a modulus	$n - 1$ (I); time slacks (II)
E	number of equality guards; escape tokens $E_{q,a}$	residual count E_Φ of the Presburger compiler	sites (I); events (II)
F	the partial step map on $\mathbb{N}^d$	the global cellular-automaton map	—
g	rational gap vector	the local table $g : \{0, \dots, K\}^3 \rightarrow \{0, \dots, K\}^3$; an initial gap	gap vector (I); blocks (II)
H	halt labels $H_{q,a}$	the witness-height bound (38.4)	box side (I); depth (II)
h	integer gap vector h_k , loader h_0	comparison flags h_i ; a time	depth (I); depth, end-point slacks (II)
K	the horizon (number of steps)	the alphabet cap, $q = K + 1$; witness count K_Φ	$2 + 4V$, chambers (I); batches (II)
L	a guard form $L(X)$; the move direction L	left lane L ; an affine atom L ; the term count $\mathcal{L}$	speed-clearing integer (I); lifetimes (II)
M	uniform-lift matrices M_j ; branch matrices $\widehat{M}_r$	the conserved mass	labels (I); largest position (II)
m	the padded word length of the loader	number of source states	distinct speeds (I); speed multiplier (II)
n	the population (live signals)	a counter value; counters n_0, n_1	initial signals

Letter	Part III (source 12)	Part IV (source 13)	Parts I–II
p	timestamp numerators p_k ; productions p_i	the period of a type cycle	in-counts (I); positions (II)
Q	the loader scale ℓb^m , or a clearing denominator	a finite cellular-automaton alphabet	—
q	mode code; Morita’s states q_i	$q = K + 1$; source states q_i ; quotients $q^\pm$	out-counts (I); position scale (II)
R	the layout half-width $\ell + 2$, and the residual count $1 + 2d + E + T + W$	right lane R ; residual count; the right walker’s position $R(t)$	raw rows (I); residuals (II)
r	branch index; the length of a run in (33.1)	ranks r_i ; the radius r ; a table entry	retained rows (I); arity (II)
S	the span $S = \mathbf{1}^\top h$; the source alphabet	number of allowed rows of a restricted list	accumulation time (I)
s	guard slacks $s_{r,L}$; Morita’s target state	table selectors $s_{i,\tau}$; comparison slacks	strict rows (I); fanout (II)
T	number of strict guards, <i>not</i> a time	the horizon (number of lattice steps)	site budget (I); batch times (II)
u	mode coordinate $u = qS$	incoming masses u_i	gaps (I); slacks (II)
V	a bound on speed differences	witness count V	speed bound (both)
W	number of weak guards	—	$3u + 2v$ (I); witnesses (II)
w	a written symbol; the input word	incoming masses w_i ; the weight function	—
z	packet witnesses $\mathbf{z}$	channel index of a record	slacks (I); test vector (II)

Several words need the same care.

- **Horizon.** Part III’s horizon is K steps of the event map, Part IV’s is T lattice steps; Part II’s K is a number of collision batches of a prescribed schema. In Parts III and IV the horizon is a compiler parameter that fixes the number of variables; it is never a free unknown of one polynomial.
- **Batch.** Part III’s “signal batches” (Section 37) are the collision batches of Parts I–II, counted along a run. Its *mode* is an ordered label word, and a *germ* is a post-event state whose zero gaps separate.
- **Source.** “Source 12” and “source 13” name manuscripts; inside Parts III and IV “the source” is the simulated machine (Morita’s Turing machine, Morita’s two-counter machine).
- **Conservative.** Part III’s rules conserve the number of signals and a positive energy; Part IV’s local tables conserve the total lane mass. Both are reversible in the cases printed, but neither certificate theorem needs reversibility.
- **Sparse.** Part III’s “sparse scaled projections” have at most $2(n - 2)$ nonzero entries; Part IV’s “sparse” certificate keeps M records however large the gaps between them. Neither is Part I’s event-sparse row count, and none of them is a fixed-arity statement.
- **Loader.** Part III’s loader writes a prepared tape into two rational distances at a cost linear in the tape length (Section 33); Part IV’s uniform loader leaves two counter positions as free

natural parameters and pays $4 + 3M(M - 1)$ witnesses and residuals (42.2). Neither converts an ordinary binary integer input.

1.9 Relation of Parts III and IV to the repository

[write] Neither new source cites the material below; these cross-references are added. Results of other reports are cited by label name.

- **Re-derived results, printed as pointers or second routes.** Part III’s complete event selection (Lemma 34.1) is the one-batch case of Part I’s Theorem 5.3, and its uniform lift is the denominator device of Theorem 7.1 and Part II’s grid. Part IV’s comparison gadget is Lemma `cdc:mem:lem:compare` and its sorting comparator the two-field record comparator (`cdc:mem:prop:comparator-cost`) of Part V of *canonical-diophantine-certificates* [11]; Part IV’s horizon-free quartic (Corollary 46.1) is a degree-four second route to the single-fold degree-two representation of semilinear sets of that report’s Part XI (`cdc:of:thm:singlefoldsl`); and the finite-search step of Proposition 43.2 is `cdc:bd:prop:nobound`. Each place carries a [write] note; no novelty is claimed for any of them.
- **The degree-two orthant format.** Part III’s step polynomial is literally of the form $\sum A_i^2 + \sum B_j C_j$ of the spine of *quadratic-orthant-certificates* (`qoc:eq:spine`), nonnegative on the real orthant; its existence also follows from `cdc:of:thm:singlefoldsl`, since a step relation with finitely many linear branches is semilinear; source 12’s contribution is the explicit construction and its ledger. Its selector/copy construction is Balas’s disjunctive formulation [26, 27, 28], credited at the place. (Added 2 October 2026, batch 79: that report’s Parts V and VI use the same selector device with the selector added to the right factor of each product, which makes their nonnegative real fibres natural; the note after Theorem 36.1 compares the two gates and shows that the real fibre of Part III’s packet is natural at every natural input $X \neq 0$.) Part IV’s certificates are sums of squares of quadratic residuals, of degree four; its orthant-exact option restores exactness on the nonnegative real orthant with norm-one residuals.
- **The finite-input interface asked for by a programme audit.** The block-automaton audit [15] notes that finite particles could store unbounded values in their separations and that a new construction would have to establish that interface. For signal machines, Part III establishes it: a fixed machine with 18 live signals, finite support, stack values stored in two distances, and observable halt and null outcomes with an escape of eight crossings. It does so with a prepared rational loader, not from an ordinary integer, and not for billiard-ball or block automata, which remain as the audit leaves them. (This supersedes, for signal machines, the last sentence of the corresponding item of Section 1.4.)
- **Other reports.** Part IV’s mass-two theorem is related only loosely to Part I’s rigidity theorem (both end in semilinearity, for different reasons). Part IV bears on the question “Local certificates on adaptive support geometry” of Part XVI of *canonical-diophantine-certificates* without answering it (note at Part IV’s questions). The fixed-horizon form of Part III is the same kind of statement as the trace theorem `pqc:pm:thm:trace` of *probabilistic-quantum-and-continuous-computation*, for different dynamics.
- **The programme’s reviews.** The Hilbert-tenth-problem programme reviewed the earlier edition of source 12 (batch-79 manuscript 07) and wrote the patch that the corrected edition

printed here applies [29], and then reviewed the corrected edition itself, with a natural-domain projection of its packet [31]. It reviewed source 13 completely and wrote a constructor patch [30], which was not applied here; the corrected code edition of source 13 (batch 80) contains an equivalent repair of its own and is now shipped, and the programme’s correction audit [36] confirms it. It later reduced two of source 13’s ledgers [32, 33]. Its review index [34] lists all of these as complete, and its placement audit [35] authenticates every file of both sources placed by `a7ae02511` against the delivered archives (five of source 13’s have since been replaced by the corrected edition’s bytes). Sections 37 and 47 give the details. No review changes a theorem of this report, and the reductions are the programme’s, not re-proved here.

- **The repository’s universal bounds.** The 75- and 87-operation bounds stated in the project README [1] are untouched by Parts III and IV, which say so themselves (no fixed-arity polynomial for unbounded histories). The programme’s review of source 12’s earlier edition records the same conclusion.
- **Formal status.** Nothing in Parts III and IV is formalized in Lean or Rocq; the Hilbert-tenth-problem Lean development has no signal-machine, cellular-automaton or Presburger-compiler module that states any of their results. One classical ingredient that Part IV imports is formalized elsewhere in the repository: Cooper’s quantifier elimination for Presburger arithmetic, used in Section 45, is the Lean development `Logic/PresburgerArithmetic` (for instance `PresburgerArithmetic.Formula.holds_iff_quantifierEliminate` and the decision procedure `PresburgerArithmetic.Formula.presburgerArithmetic_decidable`). That formalizes the imported step only, over the integers, and none of Part IV’s own statements.

1.10 Status and what Parts III and IV do not claim

[write] Both new sources give conventional proofs and finite exact-arithmetic checks, which are regression evidence, not proofs; both are AI-assisted and unrefereed. Every limitation they state is kept in its Part. Collected here, they do not claim

- priority, or an exhaustive literature search, for the precise synthesis (Part III), the sparse constructions or the mass-two theorem (Part IV); a new universal computational mechanism or new classical universality; the stack, selector/copy and circuit-to-sum-of-squares devices as new;
- a fixed-arity polynomial for an unknown, unbounded horizon or for arbitrary halting histories; the horizon fixes the number of variables in both Parts, and “there exists some finite horizon” is an unbounded union that needs a separate compression theorem;
- for Part III: that the ten-signal binary interpreter is printed or instantiated; optimality of 18 or ten live signals, of the loader or of the dimension; injectivity of the integer event map; that every path of the 49,700-mode graph is realizable from one input (it is a mode-only overapproximation), or that the graph bounds the length of a universal run; a flat monomial expansion (the trillion-term count is derived, not emitted); a minimal gate count; any semantics for malformed rational configurations or at accumulation points; that the finite replays prove universality, which is imported from Morita’s theorem; that the polynomial is convex or a sum of squares over all reals;
- for Part IV: a printed universal source table or its state count; faithful interpretation of arbitrary counter pairs; intrinsic universality, an efficient simulation, exact-target undecidability, or

global halting (the pulse is an anchored observation); a minimum universal mass, or that three particles suffice; any statement over unrestricted reals; that the displayed counts are lower bounds; an arbitrary-rule quantifier eliminator; and anything about arbitrary c.e. sets from the decidable mass-two subclass;

- a proof-assistant verification; neither source’s code is a verified compiler.

1.11 Research questions answered or touched by Parts III and IV

[write] Dated notes at the questions themselves record the following; Table 3 carries the same information in its relation column.

- **Part I’s direction 15.2** (an audited universal signal-machine frontend) is *answered by Part III, except for the ordinary-integer loader*: Part III instantiates a published finite-input universal construction with its exact labels, speeds, all collision rules, forbidden (not transparent) undefined encounters, input encoding and marked acceptance, audits simultaneous and triple collisions, and proves the correspondence of its loader with the simulated machine. Two differences from the direction as worded remain. The machine is Morita’s UTM(15,6) through Durand-Lose’s 2012 stacks, not the 13-type construction of [3] named as a candidate; and the input is a prepared cyclic-tag tape loaded as two rational distances, so the equivalence with “an ordinary integer program input” would still need a digit-extraction and base-conversion loader, whose cost Part III leaves explicit (Section 33).
- **Part II’s Question 1** (transcribe the 13-label, 21-rule machine) stays open as stated: Part III transcribes a different universal machine.
- **Part II’s Question 2** (exploit conservative bounded populations) is *answered in part, at a fixed horizon*, by both new Parts. Part III instantiates every finite transition branch of a reversible conservative universal signal machine, pays for strict guards and for its loader, and excludes accumulation on encoded runs; Part IV does the same for lattice dynamics, where conserved mass bounds the records but not the gaps. Neither encodes unbounded histories in a fixed arity, which the question also mentions.
- **Part II’s Questions 3 and 4** are not answered; Part III’s literal ledger is a variable-and-residual count, not the gate ledger asked for.
- **Part IV’s own five questions** (Section 48) are open; Part III states no separate questions.

Part I

Collision geometry is linear (source 07)

[write] Part I prints batch-78 manuscript 07, *Collision Geometry Is Linear: Event-Sparse Quadratic Diophantine Certificates for Rational Signal Machines*, “research manuscript prepared with ChatGPT for Vladimir Reshetnikov’s ProveIt research program”, dated 2 October 2026. Its labels carry the prefix `smc:cg::`; its sections are Sections 2–16 here and its two appendices are Appendices A and B. Its abstract and keywords follow.

Abstract (source 07). A rational signal machine computes by the collisions of finitely many types of particles moving at fixed rational velocities on a line. Its continuous geometry seems

to require nonlinear arithmetic certificates. We show that, once a complete chronological collision skeleton is fixed, the geometry is instead governed by homogeneous rational linear equations and strict linear inequalities in the initial gaps. A sweep construction using maximal adjacency intervals proves that these conditions exclude both unrecorded earlier events and omitted simultaneous collisions. For n initial signals, m distinct velocities, and E collision sites, at most $3(n - 1) + (3m + 1)E$ raw linear conditions suffice.

After positive denominator clearing, one globally nonnegative convex quadratic polynomial certifies the skeleton over the nonnegative integers. Every accepted integer seed has exactly one slack-witness tuple. A finite disjoint first-hit family then yields one quartic with unique selector and slack witnesses. We also prove causal-depth denominator and coefficient bounds, a small integral realization theorem, a robustness–codimension criterion, and a finite-horizon quasi-polynomial counting theorem. Explicit examples include a first-collision tournament with an exact counting formula and inverse, and a four-speed accumulating clock whose denominator size grows exponentially in causal depth. Finally, we prove that globally nonnegative quadratic certificates cannot be compressed into a single fixed-arity universal halting representation without leaving their decidable linear zero-set class. ([write] This last result is a special case of Part XI of the collection’s report *canonical-diophantine-certificates* [11]; see Remark 13.3.)

These are manuscript-level results with executable exact-arithmetic checks, not proof-assistant-certified theorems. The generic compiler and regression suite accompany the article. No priority claim, resolution of finite-fold MRDP, or improvement to an existing fixed-universal operation bound is made.

Keywords. Diophantine representation; signal machines; collision chronology; rational polyhedral cones; Presburger arithmetic; Ehrhart quasi-polynomials; Zeno accumulation; exact computation.

2 The question and the contribution

2.1 A concrete gap between universality and arithmetic

The MRDP theorem guarantees that a computably enumerable acceptance relation has a Diophantine representation. That existence theorem does not itself give a transparent arithmetic description of a particular computational substrate. A useful substrate-specific compiler should explain what its witnesses mean, why they cannot describe a counterfeit computation, and which resource bound is actually being measured. The distinction is especially important when the substrate has a geometric, asynchronous, or apparently continuous dynamics. For background on the classical theorem and a machine-based formalization, see Matiyasevich [9] and Bayer et al. [10].

The **ProveIt** Hilbert-tenth-problem material motivates precisely this separation. Its reversible finite-history audit distinguishes cheap local relations from a complete finite-input computation certificate, and points to additional obligations concerning wiring, initialization, boundaries, and halting observations [1, 2]. This article develops an analogous complete *geometric frontend* for a different substrate: one-dimensional rational signal machines. It does not attempt another operation-count reduction for the repository’s universal polynomial.

[write] The repository has since reviewed this manuscript, and the related manuscript printed as Part II, in its Hilbert-tenth-problem research tree (Section 1.4). The reversible-history audit cited here is unchanged from this source’s pin to the write.

Signal machines are not merely collections of unrelated finite circuits. Durand-Lose gives a halting universal construction with 13 signal types and 21 collision rules, obtained through cyclic tag systems [3]. This established universality is the reason to study the substrate. We do not reproduce

or independently certify that universal machine's full rule table and input loader here. Instead we prove a generic finite-history compiler whose interface applies to the rational fixed-speed model.

The central question addressed is the following.

Can an explicitly proposed finite collision history be represented arithmetically without introducing arbitrary event coordinates, without overlooking an earlier collision, and without splitting a genuinely simultaneous event into a false sequence?

Our answer is affirmative. In fact, the required geometric residuals can be linear rather than quadratic, and their sum of squares can therefore be quadratic rather than quartic. The essential work is not squaring equations; it is showing that a small set of order constraints certifies the *complete* chronology.

2.2 Results and their logical scope

For a fixed initial label word and a fixed discrete skeleton σ , we construct integer matrices $\mathbf{E}_\sigma, \mathbf{G}_\sigma$ such that the real initial-gap vectors realizing that skeleton are exactly

$$\mathcal{C}_\sigma = \{g \in \mathbb{R}^{n-1} : \mathbf{E}_\sigma g = 0, \mathbf{G}_\sigma g > 0\}. \quad (2.1)$$

Initial gap positivity is included among the strict rows. Every event time and position is a rational linear form in g . Section 5 proves the event-sparse bound

$$R_\sigma \leq 3(n-1) + (3m+1)E \quad (2.2)$$

for the number of raw linear residuals and inequalities.

The corresponding natural-number certificate is

$$P_\sigma(g, z) = \sum_i (\mathbf{E}_{\sigma,i} g)^2 + \sum_j (\mathbf{G}_{\sigma,j} g - 1 - z_j)^2. \quad (2.3)$$

Its degree is at most two, and its entire auxiliary tuple is forced to be $z_j = \mathbf{G}_{\sigma,j} g - 1$. This uniqueness concerns a fixed externally supplied skeleton, not a fixed universal MRDP polynomial.

The same linearization gives several additional conclusions. A collision at causal depth h has coordinate denominators dividing D^h after integer speed normalization, where D is the least common multiple of the nonzero speed differences. Every nonempty chamber has a small positive integer point. A chamber's synchronization equalities determine its exact codimension and its perturbation behavior. For a fixed event budget, first-acceptance seeds form a finite disjoint union of such chambers, and counting them in growing integer boxes gives exact quasi-polynomials.

These conclusions solve specific representation, realizability, precision, and counting questions formulated here. They are not presented as the settlement of a named longstanding conjecture. The literature review establishes the underlying signal-machine model and related accumulation results; it does not establish historical priority for every combination of lemmas in this article. The proofs are intended to stand on their own and to be suitable for further independent review.

3 Rational signal machines and chronological skeletons

3.1 Particles, velocities, and collision rules

Definition 3.1 (Machine). A rational signal machine consists of a finite label set M , a velocity map $v : M \rightarrow \mathbb{Q}$, and a deterministic rule for every admissible incoming collision set under consideration.

An incoming set contains at least two labels with pairwise distinct velocities. Its output is a possibly empty set of labels with pairwise distinct velocities. A signal of label a moves along a straight line of slope $v(a)$ until it participates in a collision.

We use a transparent default rule when an explicit rule is absent: the incoming labels reappear as outgoing labels, ordered by increasing velocity. Such a crossing still counts as a collision. Alternatively, one may specify all relevant rules explicitly and reject a skeleton requiring an undefined rule. The attached implementation supports both conventions.

At a collision all incoming signals are consumed and the prescribed outputs are born at that space-time point. Newly born outputs are ordered by increasing velocity in the right-hand time limit. This makes them separate rather than immediately collide again. At each positive time, all spatially distinct collisions occurring at that time are processed together.

This is the fixed-speed straight-line model used in abstract geometrical computation [3]. Our arithmetic interface restricts initial configurations to finitely many signals at distinct positions and considers ordinary finite prefixes. An infinite sequence accumulating at a finite time has no automatically assigned continuation in this article.

3.2 Initial coordinates and normalization

Fix $n \geq 2$ initial labels $a_0, \dots, a_{n-1}$, in increasing spatial order, and write $d = n - 1$. Translation invariance lets us put

$$x_0 = 0, \quad x_i = g_1 + \dots + g_i \quad (1 \leq i \leq d), \quad g_i > 0. \quad (3.1)$$

Repeated labels at distinct positions are permitted. We call g the *seed geometry*; the label word is separate discrete input.

The principal chamber theorem allows real g . The Diophantine theorem uses integer g . A rational gap vector can be multiplied by a positive common denominator to become integral. Scaling all positions by $c > 0$ scales all event times by c without changing any collision rule or chronological skeleton. Thus integral seeds suffice for existence of a rational realization, but they need not preserve a separately specified physical length or time bound.

When all velocities are rational, choose an integer $L > 0$ such that $Lv(a)$ is integral for every label. In the time coordinate $\tau = t/L$, all speeds are integers $Lv(a)$. Denominator bounds below are stated in this normalized time coordinate. An arbitrary common translation in velocity is not needed.

3.3 Batches and stable signal identities

Definition 3.2 (Chronological skeleton). A finite skeleton is an ordered list of nonempty batches. Each batch is an ordered list of pairwise disjoint contiguous blocks in the current live signal list, each block having at least two members. The blocks are listed from left to right. Each block is a collision site, with its rule and outputs determined by the incoming labels. New signal identities are assigned by batch, then by site, then by increasing output velocity.

The initial identities are $0, \dots, n - 1$. A consumed identity cannot be used again; a noninitial identity must have been generated by an earlier batch. Within a single batch, outputs cannot serve as inputs of another site. These are discrete well-formedness conditions, not arithmetic constraints left for a witness to evade.

Let B be the number of batches and E the total number of collision sites. At site e , write p_e and q_e for the numbers of incoming and outgoing signals. Let m be the number of distinct velocities. Then

$$2 \leq p_e \leq m, \quad 0 \leq q_e \leq m, \quad B \leq E. \quad (3.2)$$

The quantities B , E , and causal depth are different. Many sites may share a batch; many batches may contain events unrelated to a given causal chain.

Definition 3.3 (Realization). A positive seed g realizes a skeleton σ when its actual execution has exactly the specified batches as its first B batches, with exactly the specified incoming blocks and outputs at every listed site. In particular, no earlier event and no simultaneous site or extra incoming signal may be omitted. Nothing is required about events after the final listed batch.

The empty skeleton is realized by every positive seed. It states only that no initial portion has yet been requested; it does not state that the machine never collides. A malformed skeleton is assigned the empty realization set. One can encode a statically rejected skeleton by the constant polynomial 1 if a total polynomial-producing interface is desired.

4 Eliminating every event coordinate

4.1 Intercept propagation

For each signal segment s , let v_s be its fixed velocity and write its supporting line as

$$x_s(t) = b_s + v_s t. \quad (4.1)$$

Its intercept b_s need not be its birth position: the birth may occur at a positive time. Initially, the b_s are the partial sums in (3.1), hence homogeneous integer linear forms in g .

Suppose an event contains two inputs s, r with $v_s \neq v_r$. Their formal meeting time and position are

$$t_e = \frac{b_r - b_s}{v_s - v_r}, \quad x_e = b_s + v_s t_e. \quad (4.2)$$

For an outgoing label of velocity w , its intercept is

$$b_{\text{out}} = x_e - w t_e. \quad (4.3)$$

The denominator $v_s - v_r$ is fixed by the machine, not a variable. Consequently these formulas preserve rational linearity.

Lemma 4.1 (Coordinate elimination). *For every well-formed finite skeleton, all formally reconstructed intercepts, event times, and event positions are homogeneous rational linear forms in the initial gap vector. An event with more than two incoming signals requires only additional homogeneous rational linear equalities.*

Proof. Induct on the ordered batches. Initial intercepts are linear. Equations (4.2) and (4.3) use only addition and multiplication or division by fixed rational constants. An additional incoming signal u requires $b_u + v_u t_e = x_e$, again a homogeneous linear equality. Every new intercept is then available for subsequent batches. No output of the current batch is used by another event in that batch, so this induction is acyclic. $\square$

All event coordinates are therefore *computed data*. They need not appear as free Diophantine witnesses. A certificate containing unconstrained coordinate variables would obscure both the low degree and the uniqueness that result from this elimination.

4.2 Local incidence does not imply a real computation

The incidence equations alone are insufficient. Three signals starting at $0, u, u + v$ with respective speeds $2, 0, -1$ formally give the first two a meeting time $u/2$. If $u = 3$ and $v = 1$, however, the second and third signals collide already at time 1, before $u/2 = 3/2$. An equation witnessing the later formal intersection cannot establish that the two original segments are still alive there.

There is also a boundary failure. When $u = 2v$, all three signals meet at once. A purported binary event containing only the first two signals has perfectly consistent incidence coordinates but the wrong incoming set. A compiler must distinguish strict separation from weak separation at precisely these boundaries.

For events in the same batch, choose the leftmost site's formal time as t_k . Equate all other site times in that batch to t_k , and require

$$t_k - t_{k-1} > 0, \quad t_0 = 0. \quad (4.4)$$

These conditions are necessary but still do not exclude an unlisted earlier collision. The missing ingredient is a complete spatial-order certificate.

5 A sparse certificate for complete collision chronology

5.1 Maximal adjacency intervals

Between collisions the live signals have a strict spatial order. The skeleton tells us how that order is updated: replace each incoming block by its output block, in increasing velocity order. This replacement changes only a bounded number of adjacent pairs.

Definition 5.1 (Adjacency interval). A maximal adjacency interval records an ordered pair (s, r) , left signal first, that remains adjacent in the live list from one batch boundary to a later batch boundary. The interval begins when the pair first becomes adjacent and ends when it ceases to be adjacent or when the requested prefix ends. An unchanged pair is not closed and reopened merely because an unrelated event occurs elsewhere.

Intervals created at the final batch have zero duration and need not be checked: their output order is already determined by the distinct velocities. Every recorded positive-duration interval has an affine gap

$$\Delta_{sr}(t) = b_r - b_s + (v_r - v_s)t. \quad (5.1)$$

At its initial boundary this gap is prescribed to be zero exactly when both segments are born at the same collision there. Otherwise it is prescribed to be strictly positive. At its final boundary it is prescribed to be zero exactly when both segments are consumed at the same collision there. Otherwise it is strictly positive.

Both endpoints cannot be prescribed zero for a positive-duration interval. Indeed, two distinct straight-line segments cannot separate after a common birth and then meet again without one undergoing an intervening collision. Such a proposal is rejected statically. Each surviving interval contributes exactly two linear conditions before redundant zero rows are removed.

Lemma 5.2 (Endpoint lemma). *Let $a < b$ and let f be affine on $[a, b]$. If $f(a), f(b) \geq 0$ and at least one of them is positive, then $f(t) > 0$ for every $a < t < b$.*

Proof. For $t = (1 - \lambda)a + \lambda b$ with $0 < \lambda < 1$, $f(t) = (1 - \lambda)f(a) + \lambda f(b)$ is a positive convex combination of nonnegative numbers, at least one of which is positive. $\square$

[write] Part II proves the same statement, with terminal rays added, as Lemma 21.1, and the boundary half of the soundness argument below as Lemma 21.2.

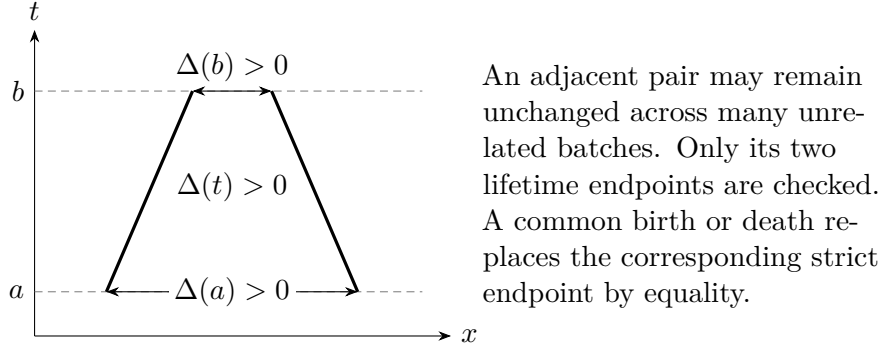


Figure 1: The affine gap between adjacent straight segments cannot become zero in the interior if its permitted endpoint conditions hold.

5.2 Soundness and completeness

Theorem 5.3 (Exact chronological chamber). *Fix a rational signal machine, an initial label word, and a finite skeleton σ . There is an effective construction of rational homogeneous linear equalities and strict inequalities whose solution set is exactly the set of real seed geometries realizing σ . All simultaneous collisions and all extra incoming signals are accounted for. Every nonempty solution set is a relatively open rational polyhedral cone in the kernel of its equality matrix.*

Proof. First reject malformed identities, overlapping or noncontiguous input blocks, undefined rules under the nontransparent convention, and incoming blocks whose velocities fail to decrease strictly from left to right. The latter condition is necessary because immediately before a multiple meeting, two incoming lines s, r ordered left to right have gap $(v_s - v_r)(t_e - t) > 0$ and hence $v_s > v_r$.

Propagate every intercept and event coordinate using (4.2)–(4.3). Add the additional incoming-incidence equalities, equal-time conditions within each batch, (4.4), initial gap positivity, and all adjacency-interval endpoint conditions described above. Every resulting expression is a homogeneous rational linear form in g .

For soundness, suppose all these conditions hold. On each positive-duration adjacency interval, the endpoint lemma gives strict separation in its interior. Hence no adjacent pair meets at an unlisted intermediate time. A meeting of nonadjacent signals would force an adjacent meeting: at the first failure of strict spatial order, at least one consecutive gap is zero. Thus no collision of any arity can occur between the listed batches.

At a batch boundary, the incidence equalities put the signals of each specified block at its reconstructed common site. Every adjacent gap not belonging to that block is strictly positive there. Consequently no additional incoming signal can join the site. The same strict conditions separate the sites of distinct blocks. They also exclude an omitted disjoint collision elsewhere in the configuration: an omitted site would have a zero gap at a boundary where strict positivity is

imposed, or would lie in the interior of an unchanged adjacency interval, where positivity has just been proved.

The deterministic rule is therefore applicable to exactly the listed incoming set at each site. Its outgoing order is increasing in velocity, so outputs separate in the right-hand time limit. Static identity validation ensures that every segment has a legitimate birth and is consumed at most once. Induction on batches now proves that the reconstructed execution is exactly the actual initial prefix.

For completeness, consider an actual execution realizing σ . The coordinate propagation agrees with its genuine intersection coordinates. The extra incidences and batch-time equalities hold, batch times strictly increase, and each endpoint gap is zero precisely at a common birth or common death of the pair. Every other endpoint gap is positive. A pair of straight segments with a common birth and a later common death cannot occur. Thus the execution satisfies every generated condition.

Finally, a finite intersection of homogeneous equality subspaces and open linear half-spaces is a relatively open convex cone in that subspace. If nonempty, its strict inequalities leave a neighborhood in the equality subspace around each point. All coefficients are rational. $\square$

5.3 The event-sparse bound

Theorem 5.4 (Linear number of chronology conditions). *Let J be the number of positive-duration maximal adjacency intervals used by the construction. Then*

$$J \leq (n - 1) + \sum_{e=1}^E (q_e + 1) \leq (n - 1) + (m + 1)E, \quad (5.2)$$

$$R_\sigma \leq 3(n - 1) + (3m + 1)E. \quad (5.3)$$

These bounds apply before deduplication or removal of identically zero equalities.

Proof. Initially there are $n - 1$ adjacent pairs. Replacing a collision block by q_e outputs creates at most $q_e - 1$ internal output pairs and two external pairs, for a total at most $q_e + 1$ when $q_e \geq 1$. If $q_e = 0$, it creates at most one bridge pair between the surviving neighbors. At a boundary or when neighboring blocks change simultaneously, fewer pairs may be created. Existing unchanged pairs need no new interval. This proves (5.2).

There are $d = n - 1$ initial gap inequalities, B positive batch-time increments, $\sum_e (p_e - 2)$ additional incoming-incidence equalities, $E - B$ equalities synchronizing the sites within batches, and $2J$ endpoint conditions. Therefore

$$\begin{aligned} R_\sigma &\leq d + B + \sum_e (p_e - 2) + (E - B) + 2J \\ &= d + \sum_e p_e - E + 2J \\ &\leq d + (m - 1)E + 2d + 2(m + 1)E \\ &= 3d + (3m + 1)E. \end{aligned}$$

$\square$

[write] The first inequality of (5.2) is Part II's Lemma 20.1; Part II's own size ledger (18.3) counts witnesses and residuals in its event-coordinate format.

The word *sparse* here refers to the number of geometric conditions, not to the number of nonzero coefficients in each eliminated linear form. A deeply propagated intercept may depend on every

initial gap. Storing all rows densely can take $O(dR_\sigma)$ coefficient slots; compiling them may have additional bookkeeping cost. Neither (5.3) nor degree two is a constant arithmetic-operation bound for an unbounded computation.

An implementation may scan the live list at each batch for simplicity. That does not change the mathematical row bound. A linked-list implementation can update adjacencies locally, while the attached small implementation prioritizes clarity and exact arithmetic over this engineering optimization.

6 Quadratic Diophantine certificates with unique slacks

6.1 Positive denominator clearing

Every nonzero rational row can be multiplied by a positive common denominator and then divided by the positive greatest common divisor of its integer coefficients. For equality rows the sign may subsequently be normalized; for strict inequality rows it may not. Negating a strict row would reverse its meaning.

Delete identically zero equality rows. An identically zero strict row makes the chamber empty. Remove duplicate rows of the same kind. Denote the resulting integer matrices by $\mathbf{E}$ and $\mathbf{G}$, with r equality rows and s strict rows. Their matrix presentation is a complete, explicit description of the polynomial, even when its expanded quadratic has many monomials.

Theorem 6.1 (Unique-slack quadratic representation). *For a nonrejected skeleton, define*

$$P(g, z) = \|\mathbf{E}g\|_2^2 + \|\mathbf{G}g - \mathbf{1} - z\|_2^2, \quad (g, z) \in \mathbb{N}_0^{d+s}. \quad (6.1)$$

Then P is an integer polynomial of total degree at most two, globally nonnegative and convex on $\mathbb{R}^{d+s}$. Moreover,

$$\exists z \in \mathbb{N}_0^s \ P(g, z) = 0 \iff g \in \mathbb{Z}^d \cap \mathcal{C}_\sigma. \quad (6.2)$$

For every accepted g , the auxiliary tuple is unique:

$$z = \mathbf{G}g - \mathbf{1}. \quad (6.3)$$

Proof. Each summand is the square of an affine linear form with integer coefficients. Thus P is a quadratic sum of squares and has Hessian $2A^\top A \succeq 0$, where A is the combined coefficient matrix of these affine forms. Over the reals, and hence over the integers, a sum of squares vanishes precisely when every squared form vanishes. Its zeros therefore require $\mathbf{E}g = 0$ and $\mathbf{G}_j g = 1 + z_j \geq 1$ for every strict row. Because these expressions are integers, this is equivalent to $\mathbf{G}g > 0$. The chamber theorem proves (6.2). The same equations force (6.3), which is a nonnegative integer tuple whenever g is accepted. $\square$

[write] Theorems 5.3 and 6.1 together are the common theorem of the two Parts (Section 1.2). Part II proves it again as Theorem 18.3, with batch times and event positions kept as natural witnesses, adds halting semantics, and strengthens uniqueness to full column rank of the witness matrix (Lemma 22.1).

All event coordinates are reconstructed from g and are absent from the auxiliary tuple. This avoids arbitrary rational encodings with multiple numerator–denominator representatives. If the seed itself is given as a rational geometry in a different interface, one must separately choose and validate an encoding or a scale; the uniqueness theorem should not be silently transferred to a noncanonical rational encoding.

6.2 What uniqueness does not assert

The polynomial depends on the externally supplied skeleton. Its number of rows and slack variables is not fixed independently of the requested history. Even though each member of this family has unique auxiliary witnesses, placing an arbitrary unbounded skeleton inside one fixed Diophantine equation is a different task. An unspecified application of MRDP to a skeleton verifier does not preserve this uniqueness for free.

The present result is therefore not a proof that all computably enumerable sets admit single-fold or finite-fold Diophantine representations. Its stronger-than-usual feature is narrower and fully explicit: for each fixed collision skeleton, projection of *all* natural zeros of (6.1) onto the initial gaps is a bijection onto the actual integer realizations. There are no additional zeros corresponding to incomplete local histories.

7 Causal depth controls denominators and coefficient size

7.1 A denominator invariant

Work in the integer-speed normalization of Section 3, and set

$$V = \max(1, \max_{a \in M} |v(a)|), \quad D = \text{lcm}\{|v(a) - v(b)| : v(a) \neq v(b)\}. \quad (7.1)$$

The least common multiple of an empty set is taken as 1; in that case no collision is possible from a simple initial configuration.

Assign depth zero to initial segments. The depth of an event is one plus the maximum depth of its incoming segments. Every output segment inherits the depth of its birth event. This is causal depth, not the chronological index of the batch.

Theorem 7.1 (Causal denominator bound). *At an event of depth h , each coefficient of its time and position forms belongs to $D^{-h}\mathbb{Z}$. The same holds for every outgoing intercept. At an integer seed, the event coordinates therefore have reduced denominators dividing D^h .*

Proof. Initial intercept coefficients are integral. Suppose an event has depth h . Its incoming intercepts have denominators dividing powers of D no larger than D^{h-1} . Put them over the common denominator D^{h-1} . Formula (4.2) divides their difference by a nonzero integer velocity difference dividing D . Its time coefficients thus belong to $D^{-h}\mathbb{Z}$. The position and all output intercepts use only addition and multiplication by integer velocities, preserving that lattice. Evaluation at integer gaps preserves the same denominator divisibility. $\square$

[write] Part II proves the same divisibility for event coordinates as Lemma 19.1, with its own proof.

Only primes already present in the speed differences can appear in these coordinate denominators at an integer seed. At a rational seed, primes in its initial common denominator may also occur. Parallel collisions elsewhere do not increase the denominator exponent along an unrelated causal chain.

This is an upper bound, not an assertion that all factors of D^h occur. Cancellations and special velocity configurations can make the actual precision requirement much smaller. Section 12 exhibits an explicit case in which exponential denominator magnitude is nevertheless unavoidable.

7.2 An explicit coefficient bound

For a rational linear form, use the ℓ^1 norm of its coefficient vector. Put $K = 2 + 4V$. Initial intercepts have norm at most d .

Lemma 7.2 (Growth of propagated forms). *Every intercept of depth at most h has coefficient norm at most dK^h . Every event time and event position of depth at most h satisfies the same bound.*

Proof. Assume the input intercept norms at a depth- h event are at most $M = dK^{h-1}$. A nonzero integer speed difference has absolute value at least one, so (4.2) gives $\|t_e\|_1 \leq 2M$. Consequently

$$\|x_e\|_1 \leq (1 + 2V)M, \quad \|b_{\text{out}}\|_1 \leq (1 + 4V)M.$$

Each factor is at most K . Induction proves the claim; forms of smaller depth obey the larger bound as well. $\square$

Proposition 7.3 (Integer row height). *If the maximum event depth is h , each primitive integer chamber row can be chosen with every coefficient of absolute value at most*

$$A = 2d(1 + V)(DK)^h. \tag{7.2}$$

In particular, for a fixed machine, row coefficient bit lengths are $O(h + \log d)$.

Proof. An endpoint gap is a difference of two intercepts plus a velocity difference times a batch time. Its coefficient norm is at most $2dK^h + 2VdK^h = 2d(1 + V)K^h$. Additional incidence residuals and differences of event times satisfy the same bound. Initial gap rows also do. All these coefficients have denominators dividing D^h by Theorem 7.1. Multiplication by D^h bounds every resulting integer coefficient by (7.2). Passing to a primitive integer row can only decrease this bound. $\square$

The exponent in (7.2) bounds coefficient *magnitude*; its logarithm is linear in depth. Exact rational arithmetic can therefore remain polynomial in the externally bounded history length even when ordinary fixed-precision floating-point simulation becomes unreliable. This is not an assertion that the total number of events before acceptance has a computable bound in the initial input length.

8 Small integral realizations and finite-horizon synthesis

Theorem 8.1 (Small positive integer seed). *Let $\mathcal{C} = \{g \in \mathbb{R}^d : \mathbf{E}g = 0, \mathbf{G}g > 0\}$ be a nonempty chamber whose strict rows include $g_i > 0$. Suppose all matrix coefficients are integers of absolute value at most $A \geq 1$. Then $\mathcal{C}$ contains a positive integer point satisfying*

$$\max_i g_i \leq d^{d/2} A^d. \tag{8.1}$$

Thus every realizable collision skeleton has a realization with a seed encoding of polynomial bit length in the explicit skeleton and initial label word, for a fixed machine.

Proof. Homogeneity permits a positive scaling of any point in $\mathcal{C}$ so that every strict expression is at least 1. Hence the closed rational polyhedron

$$Q = \{g : \mathbf{E}g = 0, \mathbf{G}g \geq \mathbf{1}\}$$

is nonempty. It lies in $[1, \infty)^d$. Minimizing $\sum_i g_i$ over Q attains an optimum: intersecting with a nonempty objective sublevel gives a compact set. The optimal face is a nonempty compact polytope and has a vertex; that point is also a vertex of Q .

At such a vertex choose d independent active constraints, including sufficient equality rows. They give a nonsingular integer matrix B and a right-hand side c with entries in $\{0, 1\}$. The vertex is $B^{-1}c$. Let $\delta = |\det B|$, a positive integer. Cramer's rule shows that $g = \delta B^{-1}c$ is integral. It is positive, satisfies the homogeneous equalities, and has every strict row at least $\delta \geq 1$.

Each coordinate of g is, up to sign, a determinant in which one column of B has been replaced by c . Hadamard's inequality bounds its magnitude by $d^{d/2}A^d$; using the sharper bound on the replaced column is unnecessary. This proves (8.1). Combining it with (7.2) gives the stated bit-length conclusion. $\square$

Corollary 8.2 (Real, rational, and integer realizability coincide). *A fixed skeleton is realizable by a positive real seed if and only if it is realizable by a positive rational seed, if and only if it is realizable by a positive integer seed. Realizability is decidable by rational linear feasibility, using $Eg = 0, Gg \geq 1$.*

The equivalence is specific to homogeneous rational chambers. It should not be transferred to arbitrary nonlinear geometry. Nor does the scaled integer seed necessarily satisfy extra nonhomogeneous requirements such as an absolute prescribed distance or a fixed deadline.

Corollary 8.3 (An NP upper bound for bounded synthesis). *Fix a rational signal machine and a distinguished output label. Given an initial label word of length n and a unary bound T , the problem of whether some positive seed produces the distinguished label within a complete prefix containing at most T collision sites is in NP.*

Proof. A skeleton of at most T sites has at most T batches and at most $n + mT$ created signal identities. Since m is fixed, its discrete description has polynomial length in $n + T$. Guess such a skeleton, check its identities and rules, and check that the distinguished label appears in its prefix.

If it is realizable, Theorem 8.1 supplies a positive integer seed with polynomial bit length: $d = n - 1$, $h \leq T$, and $\log A = O(T + \log n)$ for the fixed machine. Guess that seed as well. Propagating rational forms and verifying every chamber row then takes a polynomial number of operations on polynomial-length integers. This is a polynomial-time verifier. No search over exponentially long real numbers is required. $\square$

Only an upper bound is claimed. There is no NP-hardness result here. The budget counts total sites, including all sites in the accepting batch, not elapsed physical time. A physical-time bound can contain arbitrarily many events and even an accumulation, so it is not interchangeable with T .

9 Robustness and the rank of synchronization constraints

The chamber description distinguishes two different sources of numerical difficulty. A trace can be sensitive because it demands an exact equality among initial gaps, or its exact coordinates can require large denominators even while its combinatorial pattern is robust. The latter phenomenon will occur in the accumulating clock.

Theorem 9.1 (Certified perturbation radius and codimension). *Let $\mathcal{C} = \{g : \mathbf{E}g = 0, \mathbf{G}g > 0\}$ be nonempty, with zero strict rows removed, and let $g^* \in \mathcal{C}$. Define*

$$\rho(g^*) = \min_j \frac{\mathbf{G}_j g^*}{\|\mathbf{G}_j\|_1} > 0. \quad (9.1)$$

Every perturbation $\eta \in \ker \mathbf{E}$ with $\|\eta\|_\infty < \rho(g^)$ preserves the skeleton. The chamber has relative dimension $d - \text{rank } \mathbf{E}$. It has nonempty ambient interior if and only if $\text{rank } \mathbf{E} = 0$; otherwise it has ambient Lebesgue measure zero.*

Proof. The equality constraints persist for $\eta \in \ker \mathbf{E}$. For every strict row, Hölder’s elementary norm inequality gives

$$\mathbf{G}_j(g^* + \eta) \geq \mathbf{G}_j g^* - \|\mathbf{G}_j\|_1 \|\eta\|_\infty > 0.$$

Hence the perturbed point stays in the chamber. This proves relative openness in the full equality kernel, whose dimension is $d - \text{rank } \mathbf{E}$. A proper linear subspace has empty interior and measure zero in $\mathbb{R}^d$. Conversely, when the rank is zero, the finitely many strict inequalities define an open neighborhood of each accepted point. $\square$

The radius in (9.1) is a certified lower bound, not necessarily the largest possible radius in the equality subspace. Restricting perturbations to that subspace can increase the true distance to a forbidden boundary.

One must also distinguish genuinely independent synchronization equations from identities already forced by previous dynamics. Some formal coincidences yield an identically zero row and impose no additional codimension. Therefore the statement “every multiple collision is fragile” is too strong. What matters is the rank of the *nontrivial* conditions on the input geometry. When a loader or problem specification already restricts the admissible inputs to a subspace, robustness should be measured relative to that allowed input space.

The theorem provides a useful diagnostic for a proposed geometric computer. A simulator can report not merely a diagram, but also an exact equality matrix, a synchronization rank, and a rational certified tolerance at an integer seed. Those objects separate an intended robust mechanism from an accidental coincidence created by a particular numeric sample.

10 Finite-horizon acceptance, semilinearity, and counting

10.1 Disjoint first-acceptance chambers

Fix a machine, an initial label word, a marked accepting label, and a site budget T . Acceptance means the first appearance of that label in a batch such that the total number of processed sites through that entire batch is at most T . If the initial word already contains the marked label, every seed accepts with zero events.

There are only finitely many candidate skeletons with at most T sites. Indeed, at every stage there are at most $n + mT$ identities; the rule table and possible contiguous input blocks are finite. Enumerate those skeletons that end at the first accepting batch and reject all statically invalid ones. Distinct nonempty chambers in this first-acceptance list are disjoint: for a fixed seed, deterministic complete chronology has just one first accepting prefix.

Theorem 10.1 (Finite-horizon semilinearity). *The positive integer seed geometries accepted within site budget T form an effectively constructible semilinear set. More specifically, they are a finite disjoint union of the integer points of rational homogeneous chambers of the form (2.1).*

Proof. The enumeration above is finite and effective. For each skeleton, Theorem 5.3 gives a complete linear characterization. On integer points, a strict integer inequality is equivalent to being at least 1, so each chamber is definable in Presburger arithmetic. Finite union remains Presburger-definable. The effective equivalence with semilinear sets is the classical theorem of Ginsburg and Spanier [7]. $\square$

This use of the classical equivalence is not a new proof of that theorem. The new substrate-specific input to it is the exact finite union of collision chambers, including chronology and simultaneous-event boundaries.

[write] Part II proves the same statement for its fixed schemas and event bounds, prefix and halting, as Proposition 26.1.

10.2 A single quartic for an unknown bounded first-hit skeleton

The skeleton need not remain externally supplied when the event budget is fixed. Enumerate the nonrejected first-acceptance skeletons $\sigma_1, \dots, \sigma_K$ and let their matrices be $\mathbf{E}_k, \mathbf{G}_k$. Empty chambers may remain in this list without harm. Introduce natural selectors $b_1, \dots, b_K$ and a separate slack vector z_k for each chamber. Define

$$\begin{aligned} Q_T(g, b, z) = & \left(\sum_{k=1}^K b_k - 1 \right)^2 + \sum_{k=1}^K \sum_i (b_k \mathbf{E}_{k,i} g)^2 \\ & + \sum_{k=1}^K \sum_j (b_k (\mathbf{G}_{k,j} g - 1) - z_{k,j})^2. \end{aligned} \quad (10.1)$$

For an empty family, use $Q_T = 1$.

Theorem 10.2 (Canonical first-hit quartic). *Equation (10.1) is a globally nonnegative integer polynomial of degree at most four. Its natural zeros project bijectively onto the integer seeds accepted within the fixed event budget. No Boolean equations for the selectors and no additional inactive-slack constraints are needed.*

Proof. At a natural zero, $\sum_k b_k = 1$ forces exactly one selector to be 1 and all others to be 0. For the selected chamber, the remaining equations are precisely $\mathbf{E}_k g = 0$ and $\mathbf{G}_k g - \mathbf{1} = z_k \geq 0$. For an unselected chamber, each slack residual becomes $-z_{k,j}$, forcing all of its slacks to zero. Thus the polynomial accepts exactly the union of the chamber integer points.

Those chambers are disjoint by canonical first-acceptance chronology. Therefore an accepted seed determines exactly one selected skeleton, its active slacks are uniquely determined, and every inactive slack is zero. Every residual has degree at most two before squaring, proving the quartic bound. The construction is a sum of squares and hence globally nonnegative, but it is not in general convex. $\square$

The same construction works for any finite disjoint family of chambers. If chambers overlap, it still represents their union, but a seed in several chambers has several selector witnesses. First-hit canonicalization is therefore mathematically important, not cosmetic.

Remark 10.3 ([write] Relation to a single-fold quadratic of another report). The accepted set of Theorem 10.2 is a finite union of chamber integer points, hence semilinear (Theorem 10.1). The collection's report *canonical-diophantine-certificates* [11] proves in its Part XI (Theorem `cdc:of:thm:singlefolds1`) that every semilinear set has a representation by an integer polynomial of degree at most two that is nonnegative on the real nonnegative orthant, with exactly one natural witness for each member, effectively from a Presburger formula. So, if only orthant nonnegativity is required, the first-hit set has a single-fold quadratic certificate. Theorem 10.2 gives something different: an explicit construction from the chamber matrices, globally nonnegative on all real inputs, whose selector and slacks have the chamber meaning. Within the globally nonnegative class a quadratic has a rational affine zero set (Theorem 13.1), and the selector products of (10.1) leave that class, as the paragraph after Corollary 13.2 says. Whether a globally nonnegative quadratic suffices for a given first-hit family is not decided here.

This theorem removes external skeleton choice for a *bounded* problem; it does not remove the bound. The number of candidate skeletons, and hence the number of selectors and slacks, may be very large and grows with T . The attached implementation provides the matrix-based quartic constructor, and checks a two-chamber first-hit example. It does not include an optimized exhaustive enumerator for all skeletons of an arbitrary event budget.

10.3 Exact quasi-polynomials rather than merely leading asymptotics

Let $A_T(H)$ count the accepted seeds in $\{1, \dots, H\}^d$, with $H \geq 1$. For a single chamber put

$$\Pi_\sigma = \mathcal{C}_\sigma \cap (0, 1]^d.$$

Its closure is a bounded rational polytope in its equality subspace. The original set is obtained from that closed polytope by excluding finitely many rational faces where one or more required strict expressions vanish. The upper box facets remain included.

Theorem 10.4 (Exact finite-horizon counting). *For fixed machine, initial word, accepting label, and event budget T , $A_T(H)$ is an exact quasi-polynomial in H for all positive integers H . It has degree at most d . Its ordinary generating function is rational and has poles only at roots of unity. Furthermore,*

$$\lim_{H \rightarrow \infty} \frac{A_T(H)}{H^d} = \sum_{\sigma} \text{vol}_d(\Pi_\sigma) \in \mathbb{Q}, \quad (10.2)$$

where the sum ranges over the first-acceptance chambers, and lower-dimensional terms have zero d -dimensional volume.

Proof. Homogeneity gives $\mathcal{C}_\sigma \cap (0, H]^d = H\Pi_\sigma$. For a nonempty chamber, the closure of Π_σ is obtained by replacing its strict homogeneous inequalities by weak ones. To see equality with this weak closure, take a small strictly feasible point with every coordinate less than 1 and form convex combinations with any weakly feasible point. Those combinations converge to the latter while preserving all strict conditions and the box.

A forbidden equality $G_j g = 0$ cuts out a face of the weak closure, since $G_j g \geq 0$ throughout it. Inclusion–exclusion over these faces expresses the lattice-point count in $H\Pi_\sigma$ as a finite integer combination of counts in dilates of closed rational polytopes. Ehrhart's rational-polytope theorem makes each term a quasi-polynomial; see Beck and Robins [8]. This also covers lower-dimensional faces. Disjointness of first-acceptance chambers permits summing these exact counts.

A quasi-polynomial of degree at most d and period p has a rational generating function whose denominator divides a power of $1 - z^p$. This proves the generating-function assertion. The normalized leading coefficient of a full-dimensional rational polytope count is its volume; removing proper faces does not change it. Equivalently, the same limit follows from Riemann sums for a bounded polyhedral set. Rational polytopes have rational volume by a rational triangulation and determinant formulas. Summing over the disjoint chambers proves (10.2). $\square$

Thus a finite-horizon seed-counting problem has a particularly rigid asymptotic form: a polynomial in H with periodic coefficient functions, not an arbitrary irregular error term. Its leading density is rational. If the initial gaps are instead sampled independently and uniformly from $(0, 1]$, the probability of finite-horizon acceptance is exactly the same rational volume sum.

The theorem does not promise an efficient procedure for enumerating all chambers or computing all their volumes. Their number can grow rapidly with T . It also does not justify interchanging an unbounded-time limit with the box-size limit. Removing the event bound can leave an infinite union of chambers, outside the finite quasi-polynomial argument.

Because each integer seed in one chamber has one slack tuple, its chamber count is also the number of complete natural zeros of its quadratic certificate with $1 \leq g_i \leq H$, even when no separate bound is placed on z . Across different skeletons this statement uses a tagged disjoint family of certificates, not a single unqualified polynomial.

11 An exact first-collision tournament

11.1 Three chambers and a complete polynomial

Consider labels A, B, C with respective speeds $2, 0, -1$, initially at $0, u, u + v$, where $u, v > 0$. Introduce two stationary output labels H, R and rules

$$\{A, B\} \longrightarrow \{H\}, \quad \{B, C\} \longrightarrow \{R\}, \quad \{A, B, C\} \longrightarrow \{R\}.$$

Use transparent rules otherwise. The question here is whether H is produced at the *first* collision batch.

The possible first events are exactly

event	condition	time
AB	$u < 2v$	$u/2$
BC	$u > 2v$	v
ABC	$u = 2v$	$v = u/2$.

(11.1)

For instance, the formal AC meeting time is $(u + v)/3$. If $u < 2v$, it is later than $u/2$; if $u > 2v$, it is later than v . It cannot be a different first event. The equality case is the triple collision.

The complete accepting chamber has the particularly small certificate

$$P_{AB}(u, v, z_1, z_2, z_3) = (u - 1 - z_1)^2 + (v - 1 - z_2)^2 + (2v - u - 1 - z_3)^2. \quad (11.2)$$

All five variables range over $\mathbb{N}_0$. At $u = v = 1$, for example, the unique auxiliary tuple is $(0, 0, 0)$. At $(u, v) = (3, 1)$, the third strict condition fails, correctly rejecting the incidence-only counterfeit. At $(2, 1)$ it also fails, correctly rejecting an omitted simultaneous third input.

For the triple-collision skeleton the compiler instead produces

$$P_{ABC}(u, v, z_1, z_2) = (u - 2v)^2 + (u - 1 - z_1)^2 + (v - 1 - z_2)^2. \quad (11.3)$$

Its equality rank is one. The two binary-event chambers have rank zero and are open in the positive quadrant; the triple-collision chamber is one-dimensional and requires exact synchronization.

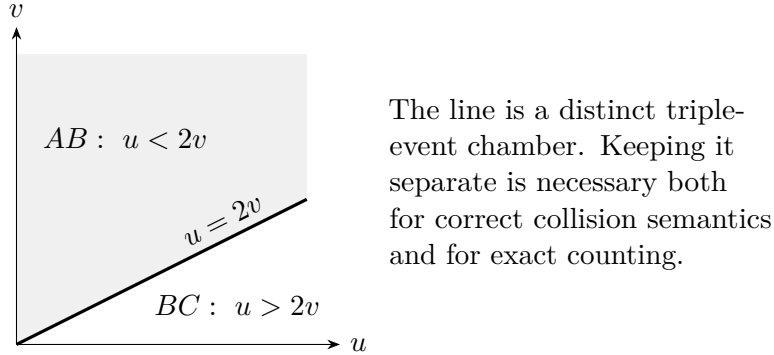


Figure 2: The first-event geometry is partitioned into two open cones and a synchronization ray. The coordinate axes are excluded.

11.2 Counting, generating function, and inversion

Let $C(H)$ count the accepting gap pairs with $1 \leq u, v \leq H$, and set $C(0) = 0$. Direct summation gives

$$\begin{aligned} C(H) &= \sum_{v=1}^H \min(H, 2v - 1) \\ &= \begin{cases} 3m^2, & H = 2m, \\ 3m^2 + 3m + 1, & H = 2m + 1, \end{cases} \\ &= \left\lfloor \frac{3H^2 + 1}{4} \right\rfloor = \frac{3H^2}{4} + \frac{1 - (-1)^H}{8}. \end{aligned} \quad (11.4)$$

For $H = 2m$, split the sum after $v = m$; the odd-number sum contributes m^2 and the remaining m terms contribute $2m^2$. The odd case follows by the same split after $m + 1$.

The first values are

$$0, 1, 3, 7, 12, 19, 27, 37, 48, 61, 75, 91, 108, 127, 147, 169, \dots$$

No assertion about an OEIS identification or novelty of this elementary sequence is needed. It is included as an exactly solvable counting test for the compiler and as an illustration of the general quasi-polynomial theorem. Its ordinary generating function is

$$\sum_{H \geq 0} C(H) z^H = \frac{z(1 + z + z^2)}{(1 - z)^3(1 + z)}. \quad (11.5)$$

This follows from the standard geometric-series derivatives applied to (11.4), or by expanding the displayed rational function.

There is also an exact inverse threshold. For every positive integer y ,

$$\min\{H \geq 0 : C(H) \geq y\} = \left\lceil \sqrt{\frac{4y-1}{3}} \right\rceil. \quad (11.6)$$

Indeed, because y is integral, $\lfloor (3H^2 + 1)/4 \rfloor \geq y$ is equivalent to $3H^2 + 1 \geq 4y$. The smooth expression inside the ceiling has the expansion

$$\sqrt{\frac{4y-1}{3}} = \frac{2}{\sqrt{3}}y^{1/2} - \frac{1}{4\sqrt{3}}y^{-1/2} - \frac{1}{64\sqrt{3}}y^{-3/2} - \frac{1}{512\sqrt{3}}y^{-5/2} + O(y^{-7/2}). \quad (11.7)$$

The binomial series supplies all further orders. The ceiling in (11.6) is essential: omitting it would incorrectly replace an integer inverse's bounded rounding oscillation by a decaying error.

11.3 Independent simultaneous sites

Take the transparent two-speed machine with $v(P) = 1$, $v(N) = -1$, and initial word P, N, P, N . With gaps $(2, 5, 2)$, the two disjoint pairs $(0, 1)$ and $(2, 3)$ collide simultaneously at time 1. A correct first batch must contain both sites. A proposed batch containing only $(0, 1)$ is rejected by the unchanged adjacency interval of $(2, 3)$, whose gap would vanish at the final boundary where positivity is required.

For general positive gaps (g_1, g_2, g_3) , this simultaneous first-batch skeleton has the single nontrivial equality $g_1 = g_3$. Changing the last gap from 2 to 3 destroys that equality. This example tests a different failure mode from the omitted third input of a multiple collision: the missing event is spatially separate, not an extra signal at the same site.

[write] Part II's hidden collision at the final cut (Section 24.2) is another instance of this failure mode, rejected by the same strict endpoint of a surviving pair (Section 1.2).

12 A four-speed clock with exponentially growing denominators

12.1 Rules and a closed-form execution

Consider the four labels

$$v(L) = 0, \quad v(P) = 2, \quad v(R) = -1, \quad v(N) = -2$$

with rules

$$\{P, R\} \longrightarrow \{N, R\}, \quad \{L, N\} \longrightarrow \{L, P\}, \quad (12.1)$$

and transparent defaults. Initially put L at 0, P at u , and R at $u + v$, with $u, v > 0$. The left boundary is stationary, the right boundary moves left at speed 1, and a particle alternately travels right at speed 2 and left at speed 2, reflecting at the boundaries via the two rules.

Write $S = u + v$ and $W = 3u + 2v$. The collision times are

$$t_{2k+1} = S - \frac{W}{3^{k+1}}, \quad k \geq 0, \quad (12.2)$$

$$t_{2k} = S - \frac{W}{2 \cdot 3^k}, \quad k \geq 1. \quad (12.3)$$

Odd collisions occur on the right boundary at position $S - t_{2k+1}$; even collisions occur at position zero.

Proposition 12.1 (Clock correctness and common chamber). *For every $u, v > 0$, equations (12.2)–(12.3) describe the actual alternating execution. The collision times strictly increase to S . Every finite alternating prefix has the whole positive (u, v) quadrant as its seed chamber.*

Proof. The first rightward meeting occurs at $t_1 = v/3$, which equals $S - W/3$. At a right-boundary collision at time t , the remaining boundary distance is $S - t$. The new left-moving particle takes $(S - t)/2$ to reach zero, leaving time-to-boundary-meeting $(S - t)/2$. From there the right-moving particle closes the remaining gap at relative speed 3, leaving two thirds of that remaining time when it next hits the right boundary. Hence a full right-to-right bounce multiplies $S - t$ by $1/3$, with the intermediate left collision having half the previous remaining time. This proves the formulas inductively.

The first time is positive. Every subsequent difference is positive, and all times are less than S . Between collisions the moving particle lies strictly between the two boundaries. The boundaries themselves meet only at S , not at any finite indexed event. There is therefore no unlisted collision in any finite prefix. The labels alternate exactly as specified by (12.1), independently of the positive gaps. This proves both correctness and the common-chamber statement. $\square$

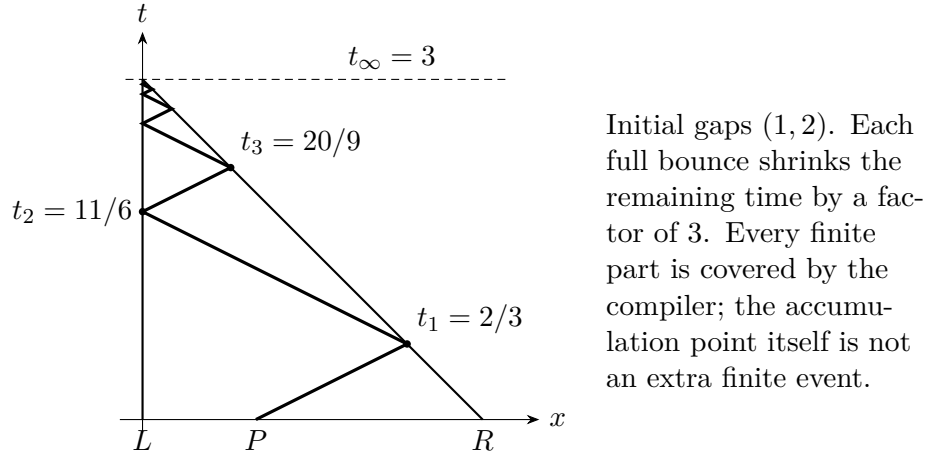


Figure 3: A four-speed accumulating clock. The drawing displays only a finite number of segments; the formulas specify all finite prefixes.

12.2 A denominator lower bound without fragile synchronization

For the particular integer seed $(u, v) = (1, 2)$, $S = 3$ and $W = 7$, giving

$$\frac{2}{3}, \frac{11}{6}, \frac{20}{9}, \frac{47}{18}, \frac{74}{27}, \frac{155}{54}, \frac{236}{81}, \frac{479}{162}, \dots$$

At event $2k$, the reduced denominator is exactly $2 \cdot 3^k$:

$$t_{2k} = \frac{6 \cdot 3^k - 7}{2 \cdot 3^k},$$

and the numerator is odd and is not divisible by 3. The causal depth of event j is j , since the reflected particle links every event to its predecessor. Thus denominator magnitude is exponential in causal depth, while its bit length is linear in depth.

Here the integer speed differences have least common multiple $D = 12$, so Theorem 7.1 correctly gives a divisible upper bound 12^j . The example does not make that upper bound sharp in its base, but it rules out a depth-independent denominator bound or a polynomial-magnitude bound for general rational signal machines.

At the same time, every finite prefix has equality rank zero and remains valid throughout the positive quadrant. Growing exact precision is therefore not evidence of a fragile collision pattern. Robust symbolic chronology and large rational denominators are compatible.

Accumulations in signal machines are established phenomena. Becker et al. [6] distinguish the behavior of small speed sets and show, in particular, that four speeds permit accumulation whereas the three-speed case with rational speed and initial-distance ratios does not. The clock is an explicit test case and a denominator-growth illustration, not a claim to have discovered accumulation. Our compiler verifies every ordinary finite prefix before S ; it assigns no transition or computational power to the limiting point at S .

[write] Part II's bounded shuttle (Proposition 25.3) uses the same mechanism on a different four-speed machine, with reduced denominators exactly 3^{k-1} (Section 1.2).

13 Why a fixed universal convex quadratic cannot follow

A natural question is whether the quadratic frontend can be compressed into one fixed polynomial with a fixed number of witnesses for arbitrary unbounded computations. Within the globally nonnegative quadratic class, there is an elementary obstruction stronger than a failure of the present construction.

Theorem 13.1 (Linear zero-set rigidity). *Let $Q(w)$ be a quadratic polynomial with rational coefficients that is nonnegative for every $w \in \mathbb{R}^N$. Its real zero set is either empty or a rational affine subspace. The projection of its nonnegative integer zeros onto any subset of coordinates is an effectively semilinear, decidable set.*

Proof. Write

$$Q(w) = w^\top A w + 2b^\top w + c$$

with A rational and symmetric. Global nonnegativity implies $A \succeq 0$. It also implies $b \perp \ker A$: otherwise the restriction to an appropriate kernel direction would be a nonconstant affine function unbounded below. Thus $b \in \text{range } A$ and the consistent rational system $A w_0 = -b$ has a rational solution.

Completing the square gives

$$Q(w) = (w - w_0)^\top A (w - w_0) + \mu, \quad \mu = c + b^\top w_0 \geq 0.$$

If $\mu > 0$, there are no zeros. If $\mu = 0$, positive semidefiniteness gives $Q(w) = 0$ exactly when $A(w - w_0) = 0$, equivalently $A w = -b$. This is a rational affine subspace.

Clearing denominators gives finitely many integer linear equations. Intersecting with $\mathbb{N}_0^N$ and projecting coordinates is an existential Presburger definition. Effective semilinearity and decidability follow from Ginsburg–Spanier and Presburger arithmetic [7]. $\square$

Corollary 13.2 (Convex-quadratic compression barrier). *An undecidable halting relation cannot be represented by one fixed globally nonnegative rational quadratic with a fixed finite number of natural parameters and witnesses, even after a computable input-loading map.*

Proof. The projected zero relation would be decidable by Theorem 13.1. Its preimage under a computable input-loading map would then be decidable as well, contradicting the assumed undecidability. $\square$

This theorem applies to global nonnegativity on *all real variables*, not merely nonnegativity on a special natural-number input slice. It does not classify arbitrary quadratic equations, arbitrary nonconvex polynomials, or cubic Diophantine representations. For example, $x - y^2 = 0$ is a quadratic equation with a nonlinear zero set, but its polynomial is not globally nonnegative and is outside the theorem.

The quartic first-hit construction in (10.1) leaves the convex-quadratic class by multiplying selectors and input-dependent linear forms. Its finite-horizon relation remains decidable for other reasons. A truly unbounded compiler must additionally encode an unbounded amount of discrete chronology into finitely many integer variables. That compression, not the local collision geometry, is where the universal Diophantine problem re-enters.

Remark 13.3 ([write] Credit). Theorem 13.1 and Corollary 13.2 are a special case of results already in Part XI of the collection’s report *canonical-diophantine-certificates* [11], which source 07 does not cite. There, Lemma `cdc:of:lem:affinezero` shows that a real polynomial p of degree at most three that is nonnegative on an open convex set O and vanishes at $a \in O$ vanishes on O exactly along $a + \ker \nabla^2 p(a)$; with $O = \mathbb{R}^N$ this is the affine zero set of Theorem 13.1. Theorem `cdc:of:thm:cubicsemilinear` shows that the full natural zero set of a *rational* polynomial of degree at most three that is nonnegative only on the real nonnegative orthant is a finite union of natural solution sets of rational affine systems, and Theorem `cdc:of:thm:classification` that the coordinate projections of such zero sets are exactly the semilinear sets, $D_2^+(k) = D_3^+(k) = \text{SL}(k)$, effectively. Those results assume less (nonnegativity on the orthant only, degree up to three, no convexity or sum-of-squares hypothesis) and give the same decidable, semilinear conclusion; that report itself notes that its theorem is stronger than the observation that a globally nonnegative polynomial cannot have odd highest degree. Corollary 13.2 therefore also holds for orthant-nonnegative polynomials of degree at most three. No novelty is claimed here for either statement. Source 07’s proof by completing the square is kept as a direct second route for the globally nonnegative quadratic case, and its clause “even after a computable input-loading map” is its own: a computable preimage of a decidable set is decidable.

13.1 The classical unbounded existence result remains available

For a finite rational initial configuration, an exact event-driven simulator can find the next collision batch by computing all positive pairwise meeting times and taking the smallest. It can then apply all rules at the sites of that batch. Repeating this procedure semidecides the appearance of a marked signal in an ordinary finite prefix. An infinite accumulating run simply causes the simulator to execute indefinitely when no earlier acceptance has occurred; no post-accumulation convention is needed for semidecidability.

Consequently the pre-accumulation finite-prefix acceptance relation, with machine and rational input encoded by natural numbers, is computably enumerable. MRDP supplies a fixed-arity Diophantine representation of that encoded relation [9, 10]. This last existence step is classical. Neither MRDP nor the present frontend automatically supplies unique witnesses, an explicit small expanded universal polynomial, or the repository’s particular operation ledger.

For a fixed universal machine with an undecidable marked-halting interface, there can be no total computable function of the encoded seed that bounds the number of sites needed by every

accepting run. Such a function, followed by exact bounded simulation, would decide halting. This explains why the bounded chamber theorems and the universal existence theorem coexist without contradiction.

[write] The last argument is the standard bounded-search argument, used for other statements in Proposition `cdc:bd:prop:nobound` of [11] and elsewhere in the collection.

14 Implementation, reproducible checks, and trust boundaries

14.1 What the attached code computes

The accompanying standard-library Python implementation uses exact `Fraction` arithmetic for every geometric calculation. The essential interfaces are:

```
Machine(speeds, rules, transparent_default=True)
simulate(machine, initial_labels, gaps, max_batches=20)
compile_skeleton(machine, initial_labels, skeleton)
UnionCertificate(tuple_of_chamber_certificates)
```

A skeleton is a tuple of batches; each batch is a tuple of input-ID tuples. For example, `((0, 1), (2, 3)),` denotes one batch with two independent simultaneous sites. Fresh identities are generated canonically by increasing outgoing velocity.

The simulator uses an all-pairs minimum-time calculation. The symbolic compiler uses intercept propagation and adjacency intervals. Their different chronology mechanisms provide a useful cross-check: the regression suite compares exact numerical prefixes against the regions predicted by the symbolic compiler on many other initial seeds.

A compiled `Certificate` contains the complete integer matrices, exact rational event forms, causal depths, interval count, and raw row ledger. Its `witnesses(gaps)` method returns the unique natural slack tuple, while `evaluate(gaps, witnesses)` evaluates the full polynomial. The `UnionCertificate` implements (10.1). Its constructor does not prove that a supplied family is globally disjoint; that is a mathematical precondition for uniqueness. At a supplied seed, its witness routine rejects multiple accepting chambers rather than silently claiming a unique answer.

The machine constructor copies and freezes the speed and rule tables. This prevents a caller's later dictionary mutation from changing the meaning of an already constructed machine. Input validation rejects unknown labels, repeated-speed outputs, malformed groups, use of dead signals, noncontiguous blocks, overlapping sites, and inappropriate incoming speed orders. These checks are part of the semantic contract.

The exported JSON matrices specify full polynomials, not just sample zeros. For example, `tournament_left.json` contains all rows of (11.2), while `first_hit_quartic_union.json` supplies every matrix and the exact selector formula for a two-chamber union. Expanding these expressions into monomials is optional and provides no extra mathematical information.

14.2 Run and reproduce

From the extracted archive's root directory, run

```
python code/run_checks.py
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

Python 3.10 or newer and a standard LaTeX installation are sufficient. No Python third-party dependencies are needed. The test command overwrites the generated JSON data with deterministic results, including `data/checks.json`.

[write] These are the delivered names. In this report the programs are shipped as `code/07-collision-geometry-run_checks.py` and `code/07-collision-geometry-signal_certificates.py` (with the delivered `Makefile` as `code/07-collision-geometry-Makefile`), and the seven JSON files of `data/` as `data/07-collision-geometry-*.json`. The test program imports `signal_certificates` by its delivered name and rewrites all seven JSON files in place, relative to its own directory's parent; it must therefore be run on a copy with the delivered layout (see the README). It was so run at the write, under Python 3.14.4 on Windows: it passed with 34,560 assertions, and the seven regenerated files equal the shipped ones apart from line endings. The delivered article PDF is not shipped; this report's PDF replaces it.

The completed run accompanying this manuscript passed **34,560 exact assertions**. Its principal coverage is summarized in Table 4. These are finite regression tests, not exhaustive verification of all machines or all input geometries.

Check family	Instances
Tournament seeds $(u, v) \in \{1, \dots, 40\}^2$	1,600
Exact seed-count boxes $H = 0, \dots, 80$	81
Exact inverse thresholds $y = 1, \dots, 2000$	2,000
Four-speed clock batches at $(1, 2)$	24
Other positive clock seeds with the same 24-batch skeleton	25
Pseudorandom rule tables (fixed random seed 20261002)	6
Distinct nonempty prefix skeletons obtained from those tables	24
Independent seed-skeleton chronology comparisons	1,536
Two-chamber quartic-union seed checks	1,600

Table 4: Finite exact test coverage. Individual assertions also check witness mutations, denominators, malformed inputs, and table immutability.

The pseudorandom cases include creation, annihilation, and transparent rules. The checked skeletons are generated from exact sample executions and then tested against other seed geometries. The suite also specifically rejects an earlier omitted collision, an omitted third input, and an omitted independent simultaneous site. It tests the clock's exact closed-form times and denominator divisibility. For the quartic union, it checks selector validity and that inactive slacks are forced to zero.

14.3 An auditable resource ledger

Table 5 separates retained conditions from raw generated rows. The generic bound is deliberately loose; redundancy elimination can be substantial. The 24-batch clock is a useful example: its entire seed chamber reduces to only three retained strict rows, even though its geometric coordinates become increasingly complicated. One of those rows is redundant with positive gaps, but the implementation does not attempt full linear redundancy minimization.

Certificate	E	B	r	s	Raw rows	Bound
Tournament AB	1	1	0	3	7	16
Tournament BC	1	1	0	3	7	16
Tournament ABC	1	1	1	2	8	16
Two simultaneous sites	2	1	1	4	11	23
Clock, 24 batches	24	24	0	3	122	318

Table 5: Exact compiler ledger: r is the retained equality count and s the retained strict-row count. The rightmost column is $3(n-1) + (3m+1)E$, not an arithmetic-operation count.

14.4 What has and has not been verified

The prose proofs establish general claims by mathematical argument. The code and data test finite instances with exact arithmetic. No Lean or other proof-assistant verification of this article is included, and the implementation is not a formally verified compiler. The universal 13-type machine’s complete published rule table and finite-input loader have not been reimplemented in the package. The code supplies a generic machine interface and the specific worked examples only.

The repository was inspected on 2 October 2026. The observed main-branch head was

`f1edb38f93aa0a362a34b531a600909f6e74608a.`

The relevant reversible-history audit was fetched as blob

`cb31d0a108b9a19fc33da6fa89a73b604e15e260.`

No repository-wide build was performed and no repository files were changed. The current README and the older audit have different historical operation ledgers; none is used as a theorem in the present proofs. The new row counts are not comparable to a fixed-universal arithmetic count without a separate input and history compression analysis.

[write] The pin `f1edb38f9` is an ancestor of this report’s placement commit. The audit blob `cb31d0a10` is still the current `Computability/HilbertTenthProblem/Papers/1980/REVERSIBLE_FINITE_HISTORY_MODELS.md` at the write; the project README [1] has gained text since the pin and still states the 75- and 87-operation bounds. The repository’s own review of this package [14] is described in Section 1.4.

15 Further research questions and a concrete development path

[write] Table 3 pairs these directions with the questions of Part II. The first two coincide with Part II’s Questions 5 and 1.

15.1 Formalizing the chronology theorem

The most direct formalization target is the endpoint lemma followed by the sweep invariant: the live list is in strict spatial order on every open interval between batches, and its equal-position blocks at each boundary are exactly the declared sites. Stable IDs make consumption and birth finite-map properties. A Lean development should keep the real semantic statement separate from rational coefficient computation, then prove that evaluation commutes with the intercept compiler.

Finally, the sum-of-squares zero lemma and natural slack uniqueness can be linked to the repository’s Diophantine infrastructure.

A practical first milestone is a small checker for exported matrices and rational event forms. A second is a proof-producing compiler rather than a trusted Python translator. Tests that compare two unverified programs should remain auxiliary evidence, not be promoted to kernel verification.

15.2 An audited universal signal-machine frontend

Instantiate a published finite-input universal construction with its exact labels, velocities, all required collision rules, transparent defaults, input encoding, and marked acceptance convention. The 13-type construction of Durand-Lose [3] is a concrete candidate. The task is not merely to copy a picture: simultaneous three-input cases, finite workspace supply, and the loader’s correspondence with the simulated machine must be audited. The eventual theorem should state an exact equivalence between an ordinary integer program input and the existence of a compiled accepting prefix.

This would turn the present generic geometry theorem into a fully explicit end-to-end substrate result. A fixed-arity universal polynomial would still require a separate outer compression step and its own complete resource ledger.

[write] **Added 2 October 2026 (batch 79).** This direction is answered by Part III, except for the ordinary-integer loader. Part III instantiates a published finite-input universal construction literally: Morita’s 15-state, 6-symbol reversible universal Turing machine compiled through Durand-Lose’s 2012 conservative stacks, with 114 labels, their speeds, all 445 collision rules, undefined encounters forbidden rather than completed, the input encoding, and marked acceptance distinguished from null termination (Theorem 29.1); simultaneous and triple collisions are treated exactly (Lemma 34.1; a triple collision is undefined), the workspace is a fixed population of 18 signals, and the loader’s correspondence with the simulated machine is proved (Sections 32–33). The machine is not the 13-type construction named above, and the input is a prepared cyclic-tag tape loaded as two rational distances: the exact equivalence with an *ordinary integer* program input asked for here still needs a digit-extraction and base-conversion loader, whose cost Part III states but does not pay (Section 33). As the paragraph above says, no fixed-arity universal polynomial follows; Part III’s certificates are for fixed horizons.

15.3 Sharper denominator invariants

The factor D^h treats all speed differences uniformly. Can one use a velocity-labelled dependency graph to compute exact prime-valuation bounds or a smaller denominator ideal for each event? Which speed sets force uniformly bounded denominators on all integral seeds, and which permit exponential growth? The clock supplies one lower-bound mechanism, while the small-speed accumulation results [6] suggest that the interaction between arithmetic precision and global dynamics is subtler than a least-common-multiple bound alone.

A particularly concrete project is to propagate prime-valuation vectors through (4.2) and detect systematic cancellation at each rule. Any claimed lower bound must use reduced denominators, not merely the unreduced fractions produced by a particular implementation.

15.4 From few rows to small arithmetic circuits

The event-sparse theorem controls row count but permits dense coefficient vectors. Can common subexpressions in intercept forms give a substantially smaller arithmetic circuit than dense matrix evaluation? An appropriate cost model must distinguish fixed machine numerals, input-dependent arithmetic, equality tests, and the conversion from a residual system to one polynomial. The original intercept dependency graph already provides a sharing structure; the challenge is to preserve its certified semantic meaning while optimizing its arithmetic schedule.

A useful result would be a proved tradeoff among witness count, maximum degree, coefficient height, and literal addition/multiplication count. No one-dimensional comparison such as “quadratic beats a degree-169 polynomial” is meaningful when arity and unbounded-history coding differ.

15.5 Avoiding exhaustive first-hit enumeration

The quartic in (10.1) is explicit but can contain an enormous number of chambers. Can a local selector circuit encode a bounded unknown skeleton with polynomially many variables while retaining canonical first-hit uniqueness, including all simultaneous events? A promising direction is to encode changes of adjacency directly rather than a rectangular array of all particle positions. The main obstruction is that a locally plausible selection must still describe one globally ordered and complete execution.

Even a polynomial-size bounded compiler would not solve fixed-arity unbounded compression. Keeping those two improvements distinct will make future operation claims interpretable.

15.6 Robust universality and minimal synchronization rank

For a given computational simulation, determine the rank of its required input-geometry equalities. Can the same computation be realized on a full-dimensional chamber, or does its interface genuinely require exact ratios among certain gaps? The answer must be relative to the legal input space rather than to coordinates the loader never permits to vary.

One possible invariant is the minimum independent synchronization rank among equivalent signal-machine implementations with a fixed speed set. The chamber theorem makes that invariant concrete for each finite trace, but extending it uniformly over a universal family needs a precise notion of simulation and allowed perturbation.

15.7 Counting complexity and asymptotic structure

Determine when first-hit chamber counts can be computed without enumerating all skeletons. Are there useful machine classes for which a recurrence on adjacency types produces the Ehrhart generating function directly? When do quasi-polynomial periods collapse, and which restrictions imply polynomial counts? The exact inverse in the tournament example is simple; in higher degree, inverse asymptotics must retain residue-class corrections and rounding effects rather than replacing them by an unjustified smooth error.

For unbounded acceptance, investigate convergence of finite-horizon rational densities. The increasing accepted sets give monotonicity, but a rational limit is not automatic. Quantitative tail estimates would be needed for a useful approximation theorem and for interchanging input-box and time limits.

15.8 Algebraic velocities and nonlinear motion

With velocities in a fixed algebraic number field, formal event coordinates remain linear over that field. However, positivity is no longer a rational half-space condition in the same elementary form, and integer realization by scaling is not automatic. A number-field sign certificate might extend part of the method, but the rational-cone, Presburger, and Ehrhart conclusions would require new arguments or additional hypotheses.

If signals accelerate or their velocities depend on variable parameters, (4.2) ceases to divide only by a fixed constant. This identifies a precise boundary of the present linearization. Studying which broader motion laws still admit low-degree complete chronological certificates is a more informative problem than assuming every geometric model behaves like the fixed-speed case.

15.9 Accumulation limits without fictitious infinite witnesses

The clock gives a finitely described accumulation time, but a generic finite prefix certificate cannot assert all events of an infinite run. A separate notion of a certified self-similar tail could perhaps prove a limit formula or a no-collision property beyond a given recursive pattern. Such a notion must specify the dynamics and limit semantics explicitly. It must not silently treat an infinite sequence of successful finite checks as one ordinary existential Diophantine witness.

16 Conclusion

For rational fixed-speed signal machines, the continuous coordinates of a finite computation are not the difficult nonlinear part. Once a discrete chronological skeleton is fixed, they are rational linear forms in the initial gaps. The nontrivial certificate obligation is completeness of chronology: excluding unlisted earlier meetings and accounting for every signal and every site at a simultaneous event. Maximal adjacency intervals solve that obligation with a linear number of endpoint conditions.

The resulting exact chamber description yields quadratic natural-number certificates with unique slack tuples, and finite disjoint first-hit unions yield quartics with unique selector and slack tuples. It also yields useful results about denominator growth, small integral realizations, perturbation stability, and seed-counting quasi-polynomials. The executable examples test these claims against independent exact simulation and expose the boundary cases that incidence-only encodings miss.

The unresolved universal task is sharply separated: finitely encode an unbounded discrete skeleton together with the ordinary input interface. The convex-quadratic rigidity theorem explains why this cannot be achieved by merely hiding more geometry inside one fixed globally nonnegative quadratic. ([write] The rigidity theorem is a special case of Part XI of *canonical-diophantine-certificates*; Remark 13.3.) The present contribution is a complete and auditable geometric frontend, together with a clear account of which further arithmetic and formalization steps remain.

Part II

Direct Diophantine certificates (source 11)

[write] Part II prints batch-78 manuscript 11, *Direct Diophantine Certificates for Rational Signal Machines: Finite collision schemas with unique quadratic witnesses*, “research article and exact

arithmetic replay”, dated 2 October 2026; its document metadata name the author as “Research report prepared with OpenAI”. Its labels carry the prefix `smc:sd:`; its sections are Sections 17–28 here and its two appendices are Appendices C and D. It proves the theorem of Part I by a second route (Section 1.2); it is printed in full, and its lemmas that coincide with lemmas of Part I carry [write] notes. Its abstract and keywords follow.

Abstract (source 11). A prescribed finite collision history of a rational signal machine admits an explicit Diophantine certificate that retains the machine’s geometry. After external normalization, we compile each finite schema into a sum of squares of integer affine residuals. Natural zeros are exactly its realizations, and every admissible input has exactly one witness tuple. The witness coefficient matrix has full column rank, so the polynomial is strictly convex in the witnesses. A sparse construction uses event coordinates, batch times, and typed endpoints of maximal adjacency lifetimes; for bounded collision arity and fanout its size is linear in the initial support and number of events. The endpoint types enforce complete collisions, simultaneous events, and the absence of a hidden collision at the final cut. We prove denominator divisibility and an explicit witness-height bound for the four-speed interface of a known universal signal machine. The construction is a uniform compiler for externally supplied finite schemas, not a fixed-arity representation of unbounded halting. Exact standard-library replays and fully worked examples accompany the proofs.

Keywords. Diophantine representation; signal machine; abstract geometrical computation; finite trace; unique witness; convex quadratic; semilinear set.

17 Purpose and relation to prior work

Turing-complete substrates can store and transform information in very different ways. Signal machines use a finite collection of particle types moving continuously on a line and interacting at collisions. A direct arithmetization of their geometry should explain both the prescribed events and the absence of all other events. Merely writing down line intersections leaves the latter obligation unresolved.

This article establishes that obligation for each fixed finite collision schema. Its contribution is an explicit integer certificate, including denominator control, canonical witnesses, sparse size bounds, and exact endpoint conditions. No general claim that signal-machine evolution is newly linearizable is intended. Durand-Lose’s linear real-register simulation already computes the next delay, updates gaps, and processes simultaneous events by finitely many linear branches [4, Section 4]. Our arithmetic constraints begin after a particular finite branch history has been fixed.

The substrate itself is already Turing universal. Durand-Lose gives a halting construction with 13 meta-signals and 21 *non-blank* collision rules; its four speeds are $-2, 0, 1, 2$ [3, Theorem 5 and Figure 4]. The construction uses a finite initial configuration and is separate from the paper’s weaker, non-halting cellular-automaton simulation with an infinite background. After a Galilean shift the speeds become $0, 2, 3, 4$. The rational normalized placements used in the halting arrangement can be scaled to integer positions, as proved generally below. We have not transcribed or executed that universal collision table. The accompanying replay uses small exact test machines.

The results are deliberately bounded in a precise sense. The machine, input length and labels, schema, and number of batches are compiler data. The polynomial’s arity grows with the schema. Nothing here packs unbounded execution into one fixed-arity quadratic, resolves the single-fold Diophantine representation problem, improves a universal-polynomial arithmetic-operation bound, or formalizes the proof in a proof assistant. A targeted literature check supports the stated relation to earlier work; it does not establish exhaustive novelty.

[write] Source 11 does not know source 07, printed as Part I, which proves the same theorem with all event coordinates eliminated and allows signed rational speeds; Section 1.2 compares the two routes. The repository has since reviewed this manuscript in its Hilbert-tenth-problem research tree (Section 1.4).

18 Model and exact theorem

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Inputs and witnesses are natural numbers unless explicitly stated otherwise.

Definition 18.1 (Normalized signal machine). A machine $\mathcal{M}$ consists of a finite set of labels, a fixed speed $v(\ell) \in \mathbb{N}$ for each label, and a deterministic collision table. A legal incoming set has at least two labels with pairwise distinct speeds. Its outgoing set also has pairwise distinct speeds and may be empty. An initial configuration consists of labels $\ell_0, \dots, \ell_{n-1}$ at positions

$$0 \leq p_0 < p_1 < \dots < p_{n-1}, \quad p_j \in \mathbb{N}.$$

Between collisions a signal follows a straight line of its label's speed. At a collision every incident signal is consumed and the designated outgoing set is emitted at that point.

If an incoming set has no rule, the collision is forbidden, unless the model explicitly completes the table by a transparent rule. Such a rule consumes the incident segments and emits the same labels, ordered by speed. The theorem works for either convention after the machine is fixed. The replay scripts use identity completion, so an omitted rule in their input is transparent, not forbidden.

We distinguish labels from *segment identities*. Each initial signal has its own identity, and each emitted signal receives a fresh one, even if a transparent rule reproduces the same label. A segment has one birth and at most one death. Initial signals are born at time zero. This distinction is essential for endpoint typing.

Definition 18.2 (Finite collision schema). Fix the machine and the initial label list. A schema $\mathcal{S}$ specifies K nonempty chronological batches. In batch i , a partition of the ordered pre-batch live list into consecutive blocks designates each nonsingleton block as an event and each singleton as a persisting signal. Every event uses a legal rule, and its incoming speeds strictly decrease from left to right. Its outputs are ordered by strictly increasing speed. All replacements in a batch are performed atomically. Segment births precede their deaths by at least one batch.

The schema fixes no event coordinates or durations. Its events within a batch are ordered from left to right by their incoming blocks. Zero-output events are retained as events. A schema with $K = 0$ has no events. No initial collision markers, infinite initial support, or accumulation events are included.

A *prefix realization* has exactly these first K nonempty collision batches and ends immediately after batch K . The $K = 0$ prefix is the initial configuration alone. A *halting realization* additionally has no future collision. Halting does not require all signals to disappear or stop moving. There is no arbitrary terminal observation time.

Let E be the number of events, $I = \sum_e |\text{in}(e)|$ the number of incoming incidences, r the largest incoming arity, and s the largest fanout. Set $n_- = \max(n - 1, 0)$ and

$$D = \text{lcm}\{|v - w| : v, w \text{ are machine speeds, } v \neq w\}, \quad B = D^K, \quad (18.1)$$

with the empty least common multiple equal to one.

[write] Theorem 18.3 is, for prefix semantics, the theorem of Part I's Theorems 5.3 and 6.1, in a different witness format: natural absolute positions as input, and batch times, event positions and slacks as witnesses on the grid B . Its halting version, the full column rank and the size ledger (18.3)–(18.4) are not in Part I. Source 11's proof is printed in full as the second route.

Theorem 18.3 (Sparse canonical certificate). *There is an effective compiler which, for every such fixed schema, produces integer affine residuals $r_1, \dots, r_R$ in the input coordinates $\mathbf{p}$ and witnesses $\mathbf{y} \in \mathbb{N}^W$, and a polynomial*

$$P_S(\mathbf{p}, \mathbf{y}) = \sum_{a=1}^R r_a(\mathbf{p}, \mathbf{y})^2.$$

For prefix semantics, a natural zero exists exactly when $\mathbf{p}$ realizes the schema. For halting semantics the same statement holds after the terminal conditions below are included. For every admissible input the witness is unique. The witness coefficient matrix has full column rank; consequently, if $W > 0$, the Hessian in the witness variables is positive definite.

If L denotes the number of recorded adjacency lifetimes, including terminal rays when present, then

$$L \leq n_- + \sum_e (|\text{out}(e)| + 1), \quad (18.2)$$

$$W \leq E + 2K + 2L + n_-, \quad R \leq I + K + 2L + n_-. \quad (18.3)$$

In particular,

$$W \leq (2s + 5)E + 3n_-, \quad R \leq (r + 2s + 3)E + 3n_-. \quad (18.4)$$

The polynomial has degree at most two, and degree exactly two whenever its witness vector is nonempty.

Syntactically illegal schemas are rejected before compilation. For the halting version, a final live order with a negative adjacent relative velocity is likewise rejected as impossible. One can instead output the constant polynomial one for these cases; the stated resource ledger concerns the structurally admitted construction. At $K = 0$, the prefix compiler may use only input-order checks. For $n \leq 1$ it has no witnesses and returns zero. This harmless degenerate case is not a strictly convex function on a positive-dimensional space.

19 Rational normalization and denominator control

19.1 Normalization is an external interface

Suppose the original speeds are rational numbers $u(\ell)$. Choose a rational $c \leq \min_\ell u(\ell)$ and a positive integer m such that every $m(u(\ell) - c)$ is an integer. For a fixed rational input choose a rational translation a and positive integer q so that every $q(x_j + a)$ is a nonnegative integer. The invertible coordinate change

$$X = q(x + a - ct), \quad T = \frac{q}{m}t \quad (19.1)$$

preserves simultaneous incidence, spatial order at each time, positive time direction, and the collision rules. Its speeds are

$$\frac{dX}{dT} = m(u(\ell) - c) \in \mathbb{N},$$

and its initial coordinates are natural numbers. Thus every finite rational configuration can be put into the theorem's interface.

The constants a, c, m, q are chosen *outside* the existential witness tuple. For a varying input family with fixed denominator and shift, the natural input $p_j = q(x_j + a)$ is the declared coordinate encoding. Allowing an arbitrary additional scale as a witness produces many proportional encodings; allowing raw variable numerators and denominators introduces products when denominators are cleared. Neither change preserves the present canonical, jointly quadratic claim without a new argument. Inputs at arbitrary real coordinates are outside this interface.

[write] Part I needs no Galilean shift: it allows signed integer speeds after the time rescaling $\tau = t/L$ (Section 3) and takes the gap vector as input. With $m = L$ the two normalizations give the same speed differences, hence the same D (Section 1.2).

19.2 Causal depth and a fixed grid

Assign depth zero to initial births. The depth of event e is one plus the maximum depth of the birth events of its incoming segments, with initial births contributing zero. Every birth lies in an earlier batch, so the maximum event depth H satisfies $H \leq K$.

Lemma 19.1 (Denominator divisibility). ([write] *The same as Part I, Theorem 7.1, there stated for the coefficient forms in the initial gaps; source 11's proof is kept as a second route because it works with event-coordinate witnesses.*) *Every event time and position of depth at most h has reduced denominator dividing D^h . In particular D^H , and hence $B = D^K$, clears every event coordinate and time. It also clears the position of any live segment sampled at an event time.*

Proof. Initial birth positions and times are integers. Suppose an event of depth h has incoming lines

$$x = b_1 + v_1(t - a_1), \quad x = b_2 + v_2(t - a_2), \quad v_1 \neq v_2.$$

All birth coordinates have denominators dividing D^{h-1} by induction. The intersection time is

$$t = \frac{b_2 - v_2 a_2 - b_1 + v_1 a_1}{v_1 - v_2}. \quad (19.2)$$

The numerator's denominator divides D^{h-1} , while $|v_1 - v_2|$ divides D . Thus the reduced denominator of t divides D^h . Substitution into either line gives the same assertion for its position. A multi-signal event is pinned by any two incoming speeds; the remaining incidences are additional equalities. Finally a sampled position is an integer-linear combination of a birth position, birth time and sample time, all cleared by D^H . $\square$

This lemma bounds denominator divisibility, not numerator magnitude. Its practical advantage is that B is a computable constant of the schema. The compiler can use the sharper D^H if desired; the supplied implementation uses D^K .

20 The sparse construction

20.1 Geometry variables and flight equations

Use a natural variable T_i for the scaled time of batch i , with $T_0 = 0$, and a natural variable X_e for the scaled position of each event. Introduce $d_i \in \mathbb{N}$ and require

$$T_i - T_{i-1} - d_i - 1 = 0 \quad (1 \leq i \leq K). \quad (20.1)$$

Thus each batch occurs at a strictly later time on the B -grid.

For a segment σ of speed v_σ , let $b(\sigma)$ be its birth batch. Define its scaled birth-position expression A_σ to be Bp_j for initial segment j , and X_f if it is emitted at event f . Its position at any relevant batch time is the affine expression

$$Z_\sigma(T_i) = A_\sigma + v_\sigma(T_i - T_{b(\sigma)}). \quad (20.2)$$

If σ is consumed by event e in batch i , impose the flight residual

$$X_e - Z_\sigma(T_i) = 0. \quad (20.3)$$

There are I such residuals. All coefficients are integers because the speeds and B are fixed integers. No product of two unknowns appears.

Input order is encoded, rather than silently assumed, by natural slacks u_j and residuals

$$p_{j+1} - p_j - u_j - 1 = 0 \quad (0 \leq j < n - 1). \quad (20.4)$$

An explicitly restricted ordered-input domain permits these residuals and witnesses to be omitted. We keep them in every count.

20.2 Maximal adjacency lifetimes

Maintain the ordered live list combinatorially. A consecutive ordered pair of segment identities has a lifetime beginning at the first boundary at which it becomes adjacent and ending at the boundary at which it ceases to be adjacent. A pair that remains adjacent across several batches has one lifetime, not a new copy in every slab. Perform all replacements in a simultaneous batch before creating new pairs; pairs appearing only during an arbitrary sequential processing order are never recorded.

For a prefix, close each still-active positive-duration lifetime at the final cut T_K . A pair newly born at batch K has no positive-duration finite lifetime and is omitted. For halting semantics, still-active pairs receive infinite terminal lifetimes instead, including pairs born at the final batch. The initial $K = 0$ prefix needs no finite lifetimes.

For a pair (σ, τ) , left to right, its scaled gap is the affine function

$$G_{\sigma\tau}(T) = Z_\tau(T) - Z_\sigma(T), \quad (20.5)$$

whose slope is the fixed relative velocity $v_\tau - v_\sigma$.

Lemma 20.1 (Lifetime count). (*[write] The same as the first inequality of (5.2) in Part I, Theorem 5.4; source 11's proof is kept as a second route because it works with event-coordinate witnesses.*) The number L of recorded lifetimes satisfies (18.2).

Proof. There are at most n_- initial adjacent pairs. Replacing one consecutive event block by k outputs creates at most $k + 1$ new pairs: at most $k - 1$ internal output pairs and two outer-neighbor pairs when $k \geq 1$, or at most one bridge pair when $k = 0$. Boundary events can create fewer. A sequential accounting of the disjoint replacements in a batch therefore counts at most $\sum_e (|\text{out}(e)| + 1)$ creations. Some counted pairs may be transient and disappear within the same batch; atomic processing removes those and cannot increase the count. Every lifetime has exactly one creation, including any terminal lifetime. Closing a lifetime does not create an additional one. $\square$

20.3 Typed endpoint constraints

For a finite lifetime from batch a to batch $b > a$, impose two constraints:

1. At the lower endpoint, impose $G(T_a) = 0$ if and only if both segments were emitted by the *same event* in batch a . Otherwise impose $G(T_a) = h^- + 1$ with a fresh $h^- \in \mathbb{N}$.
2. At the upper endpoint, impose $G(T_b) = 0$ if and only if both segments are consumed by the *same event* in batch b . Otherwise impose $G(T_b) = h^+ + 1$ with a fresh $h^+ \in \mathbb{N}$.

The “if and only if” here specifies which residual the compiler emits from the schema; it is not a new logical connective in the polynomial. Permitted equality means an equality residual, not an arbitrary nonnegative gap. In particular, a surviving pair at the final cut receives a *strict* upper gap.

For a terminal ray starting at batch a , require its relative velocity to be nonnegative, a check on machine constants. Impose a zero initial gap only for two same-event outputs at that starting boundary; impose $G(T_a) = h + 1$ otherwise. Same-event outputs have strictly increasing speeds, so a zero-gap terminal pair automatically has strictly positive relative velocity. A terminal pair with zero relative velocity must therefore start with positive gap.

Every stated condition is either an integer affine residual or a finite compile-time structural check. Define P_S by squaring and adding the residuals (20.1), (20.3), (20.4), and all endpoint residuals.

21 Proof of geometric correctness

21.1 Why endpoint types suffice

Lemma 21.1 (Positive interior gap). ([write] *The same as Part I, Lemma 5.2 (the endpoint lemma) together with the static rejection of a common birth and a common death that precedes it; source 11’s proof is kept as a second route because it works with event-coordinate witnesses, and it adds terminal rays.*) *Every finite lifetime satisfying its endpoint residuals has strictly positive gap in its open time interval. Every admitted terminal lifetime has strictly positive gap at every time after its start.*

Proof. A finite gap is affine. Its endpoint values are nonnegative. If either is strictly positive, every proper convex combination of the endpoint values is strictly positive. If both are zero, the endpoint types say that the pair shares its birth event at the lower boundary and its death event at the upper boundary. Birth ordering gives $v_\sigma < v_\tau$, so

$$G(T_b) = G(T_a) + (v_\tau - v_\sigma)(T_b - T_a) > 0,$$

a contradiction. Thus at least one endpoint is strict, and no extra endpoint-sum residual is needed. On a terminal ray the nonnegative slope preserves a strict initial gap; a zero initial gap has strictly positive slope and becomes positive immediately. $\square$

It is important that this argument uses segment identities and the exact start and end boundaries. Merely allowing two nonnegative endpoint gaps, even with positive sum, does not exclude an unrecorded meeting at the upper endpoint.

Lemma 21.2 (Complete boundaries). *At every batch satisfying the residuals, each designated incoming block meets at its event point, distinct designated events have distinct ordered positions, and no persisting signal or signal of another event also meets that point. The post-batch order is valid; its only zero gaps are between outputs of the same event.*

Proof. Before the batch, every adjacent pair belongs to a lifetime that has existed for a positive duration. For a pair in one event block, the flight equations place both segments at that event. Any pair across two distinct pre-batch blocks is not consumed by the same event. If its lifetime ends at this boundary, the strict upper endpoint gives a positive gap. If it persists across the boundary and this is an interior point of its lifetime, Lemma 21.1 gives a positive gap. If the boundary is the final cut, a surviving pair has a strict upper endpoint, or lies on an admitted terminal ray. These cases exhaust the pre-batch neighboring pairs.

All positions in an incoming block are equal by the flight equations, whereas successive blocks are strictly separated. Hence the designated blocks are precisely the maximal equality classes at that time. In particular a third incident signal cannot be omitted, two designated events cannot coincide, and a completely erased event still has a position in the strict block order. Replacing each event block at its own point by the prescribed speed-ordered outputs therefore preserves weak spatial order. Across distinct surviving blocks the gap stays strict; within one output fan the gap is zero and the increasing speeds separate it immediately. If erased blocks disappear, the new bridge gap spans one or more former strict block separations, so is positive. The replacements are atomic and introduce no additional intermediate configuration. $\square$

This proof also explains why final-born prefix pairs can be omitted. Their boundary order is already certified by the incoming block partition and flight equations. Immediately after that boundary, a common output fan separates and all other gaps remain positive for a sufficiently small interval. A prefix does not assert the absence of collisions later in that interval's future.

21.2 Soundness and completeness

Proof of the existence assertion in Theorem 18.3. For soundness, suppose $P_S(\mathbf{p}, \mathbf{y}) = 0$ with natural inputs and witnesses. Each square is nonnegative, so every residual is zero. The input packet gives strict initial order. Time residuals give strictly increasing batch times. Divide all scaled times and event positions by B and draw the prescribed segments using (20.2).

On each open slab, the combinatorial live list is fixed and every neighboring pair is in an active lifetime. Lemma 21.1 excludes all adjacent meetings. A meeting of nonadjacent signals would force an adjacent equality in the ordered list at or before that meeting, so there are no other meetings either. At the next batch, Lemma 21.2 proves that the designated events are exactly all collisions and contain every incident signal. Their legal rules are the fixed schema rules. Induction across the K batches gives precisely the prescribed prefix. For halting semantics every final adjacent gap remains positive for all future time by the terminal-ray argument, so no future collision occurs.

For completeness, start with an actual realization. Lemma 19.1 puts every event time, event position and endpoint sample on the B -grid. Assign their scaled values to T_i and X_e , and set $d_i = B(t_i - t_{i-1}) - 1$. All are natural. Input gaps determine natural u_j . Each lower or upper endpoint at which the schema prescribes equality is an actual common birth or death point. Every other endpoint is strictly separated by the definition of a complete collision history; its positive scaled integer gap minus one supplies the unique natural slack. An actual halting configuration has nonnegative adjacent relative velocities, since a negative one would eventually cause a meeting, and its terminal initial gaps have exactly the required types. All residuals vanish. $\square$

The argument covers simultaneous multi-signal collisions, disjoint events in the same batch, annihilation, transparent collisions and empty final configurations. Its dependence on complete blocks and typed boundaries is substantive, not a cosmetic choice of encoding.

22 Canonical witnesses and strict convexity

Uniqueness of integer witnesses alone would not prove strict convexity. We establish the stronger linear-algebra statement directly.

Lemma 22.1 (Full column rank). *Write the residual vector as*

$$\mathbf{r}(\mathbf{p}, \mathbf{y}) = A\mathbf{y} + C\mathbf{p} + \mathbf{c}.$$

For every structurally admitted schema, the matrix A has trivial kernel over $\mathbb{R}$, even if the schema has no realization for the chosen input.

Proof. Let $A\Delta\mathbf{y} = 0$ and hold the input fixed. Initial birth positions have zero variation. Induct on batches. Assume all earlier times and event positions have zero variation. Choose any event e in batch i and two of its incoming segments σ, τ with distinct speeds. The homogeneous flight equations are

$$\Delta X_e - v_\sigma \Delta T_i = 0, \quad \Delta X_e - v_\tau \Delta T_i = 0,$$

because both segments were born in earlier batches. Their difference gives $(v_\sigma - v_\tau)\Delta T_i = 0$, hence $\Delta T_i = 0$ and $\Delta X_e = 0$. Every other event in the batch then has zero position variation from any one of its flight equations. Equation (20.1) gives $\Delta d_i = 0$. Induction fixes all geometry variables and time slacks.

Each remaining witness is an input-order or endpoint slack with coefficient -1 in its own defining residual. With the geometry and input fixed, that residual forces its variation to be zero. Thus $\Delta\mathbf{y} = 0$. $\square$

Corollary 22.2 (Canonical zero and Hessian). *For fixed input there is at most one real solution of the residual system, and hence at most one natural zero of P_S . The Hessian in witness coordinates is $2A^\top A$, positive definite when $W > 0$.*

Proof. Two solutions differ by a vector in $\ker A$, which is zero by Lemma 22.1. Moreover $\mathbf{z}^\top (2A^\top A)\mathbf{z} = 2\|A\mathbf{z}\|^2 > 0$ for every nonzero $\mathbf{z}$. Existence for admissible inputs was proved above. $\square$

Strict convexity is asserted in the witness variables with the input held fixed. Joint strict convexity in input and witness coordinates is neither needed nor generally true. The compiler introduces no witness for a schedule choice, denominator choice or arbitrary final observation time. Adding any of those could destroy the canonical property.

Proof of the size assertions in Theorem 18.3. Geometry uses E event positions, K batch times and K time slacks. A finite lifetime needs at most two slacks and two residuals; a terminal lifetime needs at most one of each. Input order uses n_- slacks and residuals. Flight and time residual counts are I and K . This proves (18.3). Lemma 20.1, $I \leq rE$, $K \leq E$ and $|\text{out}(e)| \leq s$ give (18.4). Squaring affine residuals gives degree at most two. If $W > 0$, full column rank forces a nonzero quadratic part. $\square$

The ledger counts variables and affine residuals. With a fixed machine, every residual has bounded width: an endpoint gap uses only the birth positions and times of two segments, one endpoint time, and possibly a slack. A linear-size arithmetic circuit follows if integer constants are treated as available coefficients. This does not give a literal gate ledger under a model that

charges separately for constructing large constants, input loading, multiplication conventions or natural-to-positive conversion. In particular, no improvement to previously studied 75-operation or 87-operation constructions is asserted here.

23 A full-layer reference construction

A denser construction is useful for cross-checking. In slab i , let N_i be the number of pre-batch signals and g_i the number of blocks, including singleton through-signals. Introduce natural scaled end coordinates $C_{i,j}$, a duration slack d_i , and one gap slack between successive blocks. The scaled start coordinate $A_{i,j}$ is Bp_j initially or an alias of an earlier endpoint. Require

$$\begin{aligned} C_{i,j} - A_{i,j} - v_j(d_i + 1) &= 0 && \text{for every live signal,} \\ C_{i,j} - C_{i,\text{first}(b)} &= 0 && \text{inside each collision block } b, \\ C_{i,\text{first}(b')} - C_{i,\text{first}(b)} - h - 1 &= 0 && \text{for successive blocks } b, b'. \end{aligned}$$

All blocks, including zero-output collisions, participate in the last row. The exact counts, with input-order checks, are

$$W = n_- + \sum_i (N_i + g_i), \quad R = n_- + \sum_i (2N_i - 1). \quad (23.1)$$

Indeed each slab uses $N_i + 1 + (g_i - 1)$ witnesses, and $N_i + (N_i - g_i) + (g_i - 1)$ residuals.

The two boundary lists have nonnegative adjacent gaps, with strict separation across ending blocks. A pair with zero gap at its starting boundary is a same-event output pair with increasing speeds; it cannot meet again at the ending boundary after positive time. Affineness therefore excludes all interior collisions. The full ending partition supplies completeness at each boundary. Conversely the denominator lemma gives a witness from every realization. The same two-incoming-speed induction fixes each duration, then all endpoints and gap slacks, and proves full witness rank. A halting certificate additionally checks that the final live speeds are nondecreasing; the final boundary order is already certified.

This reference compiler stores coordinates for every live signal at every layer, even if that signal survives unchanged through many layers. The sparse compiler avoids these repetitions by storing only event coordinates and maximal adjacency lifetimes. Both implementations share an exact simulator and small arithmetic utilities; agreement between them is useful evidence, but is not independent formal verification of the common simulator.

24 Worked examples

24.1 One annihilation with its complete quadratic

Consider labels R, S with speeds $2, 0$ and the rule $\{R, S\} \mapsto \emptyset$. Let the initial positions be $p_0 < p_1$, and specify one event. Here $D = B = 2$. Write T, X, d for its scaled time, position and duration slack, u for input order and h for the lower adjacency slack. The compiler gives

$$\begin{aligned} P &= (p_1 - p_0 - u - 1)^2 + (T - d - 1)^2 \\ &\quad + (X - 2p_0 - 2T)^2 + (X - 2p_1)^2 \\ &\quad + (2p_1 - 2p_0 - h - 1)^2 + (2p_1 - 2p_0 - 2T)^2. \end{aligned} \quad (24.1)$$

The last row is the zero upper gap of the two segments consumed by the same event. Its redundancy with flight equations is harmless. Every $p_0 < p_1$ has the unique witness

$$T = p_1 - p_0, \quad X = 2p_1, \quad d = u = p_1 - p_0 - 1, \quad h = 2(p_1 - p_0) - 1.$$

For $(p_0, p_1) = (0, 1)$ the tuple (T, X, d, u, h) is $(1, 2, 0, 0, 1)$, corresponding to physical time $1/2$ and position 1. There are five witnesses and six residuals. The final state is empty, so the prefix is also halting. Changing any one witness while fixing the others cannot keep all residuals zero.

24.2 A hidden collision exactly at the final cut

Use the four-speed interface $\{0, 2, 3, 4\}$, with labels a, b, c, d respectively, and the annihilation rule $\{d, a\} \mapsto \emptyset$. Start with

$$(\ell_0, \ell_1, \ell_2, \ell_3) = (d, a, d, a), \quad (p_0, p_1, p_2, p_3) = (0, 4, 10, 14).$$

There are two simultaneous collisions at physical time 1, at positions 4 and 14. Now submit a false one-batch schema containing only the first event and marking the last two signals as through-signals. Since $B = 12$, the first event's flight equations force $T_1 = 12$ and $X_0 = 48$.

The omitted pair's gap is

$$G(T) = 12(14 - 10) - 4T = 48 - 4T.$$

It is positive before the cut, has lower value 48, and reaches zero at the cut. Thus conditions of the form $G(0) \geq 0$, $G(T_1) \geq 0$, and $G(0) + G(T_1) > 0$ would accept its endpoints. Our upper endpoint is strict because the pair survives rather than dying in one designated event. Its residual becomes

$$G(T_1) - h^+ - 1 = -h^+ - 1,$$

which cannot vanish for $h^+ \in \mathbb{N}$. The false schema has no certificate. The correct simultaneous schema consumes both pairs atomically and has event positions 48 and 168 on the scaled grid.

[write] Part I's independent simultaneous sites (Section 11.3) are another instance of this failure mode: an omitted, spatially separate simultaneous collision, rejected by the strict endpoint of the surviving pair (Section 1.2).

24.3 An outgoing fan at a finite cut

Replace the annihilation rule in the preceding setup by $\{d, a\} \mapsto \{b, c\}$. Each collision emits speeds 2 and 3. At the final cut the two members of each fan have zero gap and immediately separate; they need no zero-duration finite lifetime. In a halting certificate they instead receive terminal rays with zero starting gap and positive relative velocity. The gap between the two different fans is strict at the cut. Its relative velocity is $2 - 3 = -1$, so that terminal ray is rejected: the two fans will interact later. This separates valid finite-prefix geometry from the stronger no-future-collision requirement.

25 Witness height for the four-speed interface

Let the normalized machine speeds lie in $\{0, 2, 3, 4\}$, and let M_j be the largest live position immediately after batch j , with $M_0 = \max_j p_j$ and $\max \emptyset = 0$. Let t_j be elapsed physical time, with $t_0 = 0$.

Lemma 25.1 (Geometric growth). *Whenever batch $j + 1$ exists, its delay satisfies $\delta_{j+1} \leq M_j$. All positions at that boundary, including erased event positions, are at most $5M_j$. Consequently*

$$M_j \leq 5^j M_0, \quad t_j \leq M_0 \frac{5^j - 1}{4}.$$

Proof. A next collision must be approached by an adjacent closing pair. The pair has a strictly positive gap g and positive integer closing speed $c \geq 1$. Because all live positions are nonnegative, $g \leq M_j$; its meeting delay is $g/c \leq M_j$. The earliest collision delay is no larger. During this interval every signal advances by at most $4M_j$, so every pre-collision position is at most $5M_j$. Outputs remain at those event positions and deletion cannot increase the maximum. Iterate this bound and sum $\delta_i \leq 5^{i-1} M_0$ to obtain the stated geometric series. $\square$

The argument concerns an existing next batch. Empty or singleton configurations have none. Nor does a valid configuration whose maximum position is zero: any coincident outgoing fan has strictly increasing speeds and separates. This avoids dividing by a nonexistent positive gap.

Theorem 25.2 (Canonical height). *For $B = 12^K$, every canonical input coordinate and witness of the sparse or full-layer compiler is at most*

$$60^K \max(1, M_0). \quad (25.1)$$

Each such integer therefore has bit length $O(K + \log(M_0 + 1))$.

Proof. For the full speed set, the nonzero absolute differences are 1, 2, 3, 4, whose least common multiple is 12. The same 12 is a valid clearing modulus for a subset. Up to batch K , every live sample and every event position is at most $5^K M_0$, including positions subsequently erased. Multiplication by 12^K bounds them by $60^K M_0$. Every endpoint gap is the nonnegative difference of two positions in this range, so its slack is smaller. Input-order slacks are at most M_0 . The time bound is

$$12^K t_j \leq 12^K M_0 \frac{5^K - 1}{4} \leq 60^K M_0,$$

and the scaled duration minus one is no larger than the corresponding scaled time. The empty and all-zero cases are covered by $\max(1, M_0)$; if $K = 0$, all remaining slacks are initial-gap slacks bounded by M_0 . Taking binary logarithms proves the bit bound. $\square$

This bounds actual zero witnesses, not arbitrary trial assignments. Degree two alone would supply no such height guarantee. Likewise, the bound is exponential in batch count K as a numerical magnitude while linear in K as a bit length. For a fixed machine with maximal speed V , the same argument gives $M_j \leq (1 + V)^j M_0$; the four-speed constant above is the concrete interface relevant here.

[write] Part I bounds the chamber's row coefficients (Proposition 7.3) and the size of *some* realizing seed (Theorem 8.1); Theorem 25.2 bounds the canonical witnesses of every realizing input. The two bounds measure different things and neither implies the other.

25.1 Exponential denominator growth can be necessary

Proposition 25.3 (A bounded geometric shuttle). *There is a fixed four-label machine with integer initial positions whose $2k$ -batch prefix contains a time of reduced denominator 3^{k-1} , although every event lies in $0 < t < 2$ and $0 < x < 6$. Thus a common integer grid for exact event geometry may require exponentially large scale in the number of batches.*

Proof. Give labels L, R, F, B speeds 3, 2, 4, 0, respectively, and use rules

$$\{F, R\} \mapsto \{B, R\}, \quad \{L, B\} \mapsto \{L, F\}.$$

Start with L, F, R at positions 0, 1, 2. The two walls follow $x_L(t) = 3t$ and $x_R(t) = 2 + 2t$, so their separation is $2 - t$ until they meet at $t = 2$. The first right collision occurs at $t_{R,1} = 1/2$ and the first left collision at $t_{L,1} = 1$. At a left collision at time $a < 2$, the outgoing speed-four shuttle catches the right wall after $(2 - a)/2$, giving right time $r = 1 + a/2$. Its speed-zero replacement is then caught by the left wall after $(2 - r)/3$, giving the next left time

$$a' = r + \frac{2 - r}{3} = \frac{4}{3} + \frac{a}{3}.$$

These positive delays place the shuttle strictly between the walls between alternating collisions; there are no other earlier meetings. The recurrence and first collision give

$$t_{L,k} = 2 - 3^{1-k}, \quad t_{R,k} = 2 - \frac{3}{2} 3^{1-k} \quad (k \geq 1).$$

The first six batch times are $1/2, 1, 3/2, 5/3, 11/6, 17/9$. At the k th left event the numerator of $(2 \cdot 3^{k-1} - 1)/3^{k-1}$ is coprime to three, so the reduced denominator is exactly 3^{k-1} . Event positions lie on a wall before time two and hence are below six. Every finite prefix is valid. $\square$

This example supplies a lower bound for common-grid geometry encodings: after $2k$ batches a clearing scale is divisible by 3^{k-1} , and its scaled last time requires $\Omega(k)$ bits. It makes the linear event-count dependence in the witness-bit bound qualitatively sharp for this type of encoding; it is not a lower bound on arbitrary Diophantine representations. The collision times approach a Zeno accumulation at $t = 2$. We assert nothing about a continuation at or beyond that accumulation, which is outside the finite-schema theorem. The replay checks 64 batches with exact fractions.

[write] Part I's four-speed clock (Section 12, Proposition 12.1) is the same shuttle mechanism on a different machine, with speeds 0, 2, -1 , -2 and reduced denominators exactly $2 \cdot 3^k$ at event $2k$ for the seed (1, 2). The worked-example receipt shipped as `data/11-signal-certificates-expected-worked_examples_receipt.json` records the last left denominator of the 64-batch prefix as 617673396283947 = 3^{31} , which is 3^{k-1} for $k = 32$.

26 Fixed-schema arithmetic and the unbounded boundary

Proposition 26.1 (Semilinear feasible inputs). ([write] *The same as Part I, Theorem 10.1, for prefix semantics; source 11's proof is kept as a second route because it works with event-coordinate witnesses, and it adds the halting case.*) *For a fixed normalized machine, fixed initial label list and fixed schema, the set of natural input positions admitting a prefix certificate is semilinear. The same holds for the halting certificate. For a fixed total-event bound, the union over all schemas with that bound is also semilinear, provided the initial length and labels are fixed.*

Proof. A zero of a sum of squares is exactly a common zero of its affine residuals. Existence of natural witnesses is therefore an existential formula using only integer constants, addition, order and equality. Such Presburger-definable sets are semilinear [7, Theorem 1.3]. Terminal slope checks depend only on the fixed schema; they either remove it or leave the same argument.

For a total-event bound E_0 , at most $n + sE_0$ segment identities can have been created. A finite label set and bounded finite list admit finitely many consecutive partitions, legal rule choices and

batch assignments with at most E_0 events. Thus there are finitely many schemas. Finite unions of semilinear sets are semilinear. $\square$

More concretely, a semilinear set is a finite union of sets $\mathbf{a} + \sum_{j=1}^m \mathbb{N}\mathbf{b}_j$. The proposition describes a strong boundary: a fixed externally supplied finite schema reduces to integer linear feasibility despite its quadratic packaging. Strict convexity does not import undecidability into that fixed finite system.

Enumerating all finite schemas gives an enumerable family of quadratics covering finite-event halting, with varying arities and coefficients. In a deterministic machine an actually halting run has a definite finite sequence of complete nonempty batches. That fact does not encode its arbitrary length into one fixed tuple. A single fixed-arity representation of unbounded halting requires an additional history-packing construction, with its own decoder, arithmetic costs and witness-multiplicity proof. Generic Diophantine representability results do not automatically preserve this compiler’s quadratic degree, sparsity, or single witness. Nor does the construction certify continuation through a Zeno accumulation; no finite schema contains infinitely many events.

[write] For a fixed event budget, Part I’s Theorem 10.2 packs the finite union over all first-hit skeletons into one quartic with unique selector and slack witnesses; it does not remove the budget either.

26.1 Strictly positive witnesses

If a convention requires witnesses in $\mathbb{Z}_{>0}$, replace every y_j by $Y_j - 1$ and define

$$Q_S(\mathbf{p}, \mathbf{Y}) = P_S(\mathbf{p}, \mathbf{Y} - \mathbf{1}).$$

This is an integer polynomial of the same degree. The coordinatewise translation is a bijection from positive witness tuples to natural witness tuples and preserves uniqueness and the Hessian. The input-coordinate convention remains unchanged unless it is translated separately and explicitly. This substitution is exact but is not an assertion that conversion is free in an arithmetic-operation ledger.

27 Verification evidence and research questions

The package supplies two exact-arithmetic compilers, a rational event simulator, and deterministic test suites. Arithmetic uses Python integers and `fractions.Fraction`; no floating-point solver or Diophantine oracle is involved. Table 6 summarizes reproduced checks. Counts of certificates include both prefix and halting variants when both are generated; they are not counts of distinct universal-machine programs.

The full-layer suite additionally enumerates all 625 natural tuples in a small witness box for a two-signal example and finds its unique zero. The sparse suite’s original height test uses a factor-two safety margin; the additional suite checks the sharper constant-one bound (25.1). A release-specific example check verifies the complete annihilation polynomial, the omitted-final-collision obstruction, empty inputs, positive-witness translation, simultaneous events, output fans and the denominator-growth shuttle. Appendix C describes how to reproduce every reported receipt.

These checks can catch coefficient, endpoint and bookkeeping defects. They do not prove an infinite theorem, establish exhaustive novelty, or replace an independent formalization. The mathematical proofs above establish the stated scope.

Check	Full layers	Sparse	Additional
Certificates	122	444	878
Exact full-rank tests	30	40	12
Single-coordinate mutations	4 229	10 102	—
One-layer candidate schedules	832	832	—
Terminal certificates	—	198	372
Height checks	—	444	878

Table 6: Finite implementation evidence. The sparse suite includes 1 114 layers, 1 186 events, 3 982 lifetimes and a dedicated hidden-final-collision regression. The additional suite covers 506 fixtures and 2 152 local growth inequalities.

[write] The counts in Table 6 equal the four expected receipts shipped as `data/11-signal-certificates-expected-*_receipt.json`. The replay was run at the write on a copy with the delivered layout, under Python 3.14.4 on Windows: all four receipts and both coefficient-example files matched. The repository’s review [14] replayed it as well and adds independent finite checks (Section 1.4).

27.1 Questions with concrete next steps

[write] Table 3 pairs these questions with the research directions of Part I: Questions 1 and 5 coincide with Sections 15.2 and 15.1, and Questions 3 and 4 overlap Sections 15.3, 15.4 and 15.5.

1. **Instantiate the universal table and input map.** Transcribe the 13-label, 21-non-blank-rule machine with an explicit convention for every blank collision. Verify the cyclic-tag input encoding and rational halting gadget, then compile and replay complete finite examples. This would connect the present implementation to the cited universal machine rather than only to its speed interface. ([write] Added 2 October 2026: still open as stated. Part III instantiates a different universal machine, Morita’s UTM(15,6) through Durand-Lose’s 2012 stacks; the 13-label table has not been transcribed.)
2. **Exploit conservative bounded populations carefully.** A reversible conservative rational construction is already Turing universal [5]. Its positive energy assignment bounds the number of simultaneously live signals for fixed total energy, but not the number of collisions. A direct unbounded representation would need to instantiate its finite transition branches, encode unbounded histories of rational distances, and pay for strict guards and input loading. Conservation alone neither bounds runtime nor excludes accumulation. ([write] Added 2 October 2026: answered in part, at a fixed horizon. Part III instantiates every finite transition branch of a reversible conservative universal signal machine with 18 live signals, pays for the strict guards and the loader, and excludes accumulation on encoded runs (Section 32.4); Part IV does the same for conservative lattice dynamics. Neither encodes unbounded histories of rational distances in a fixed arity.)
3. **Sharpen the denominator and cost ledger.** Causal depth can replace batch count in B . Further denominator cancellation may depend on the actual branch structure. A useful next result would make such improvements canonical and give a literal arithmetic-gate ledger, including the construction of coefficients and any conversion of input conventions. ([write] Causal-depth clearing itself is proved: Lemma 19.1 shows that D^H suffices, and Part I, Theorem 7.1, gives D^h event by event. What remains open is a canonical choice compatible

with the witness format, and the gate ledger.)

4. **Compress histories without losing canonicity.** The sparse certificate removes repeated geometry, not the unbounded number of events. Can a restricted family of repeated collision motifs be packed with a unique decoder witness and explicit degree, height and operation bounds? Any answer should separate verification of a supplied history from existence of an outcome.
5. **Formalize geometric completeness.** The most valuable proof-assistant target is the typed-endpoint soundness theorem, including atomic simultaneous replacement and disappearing event blocks. The replay data provide regression fixtures, but the formal objects must model the real semantics, not only the executable’s output.

28 Conclusion

A finite signal-machine schema admits a direct, canonical Diophantine encoding because its collision geometry can be expressed by affine integer constraints on a predetermined rational grid. Sparse adjacency lifetimes make the certificate linear in event count and initial support. Correct endpoint types are what turn line-incidence equations into a complete execution certificate. The resulting quadratic is strictly convex in its witness variables, and the four-speed case has an explicit linear witness-bit bound. The remaining unbounded problem is to encode arbitrary schema length without losing the quantitative and multiplicity properties that make this finite construction useful.

Part III

Conservative signal machines as Diophantine frontends (source 12)

[write] Part III prints batch-79 manuscript 16, *Conservative signal machines as Diophantine frontends*, an AI-assisted research note dated 2 October 2026, from the archive `Conservative_Signal_Frontend_Corrected.zip`. That archive is the corrected edition of the batch’s manuscript 07, which has the same article and is not printed separately (Section 37 and Appendix E). It is called source 12 after the prefix of its shipped files. Its labels carry the prefix `smc:cs::`; its sections are Sections 29–37 here, and its three appendices are Appendices F–H, after the provenance appendix of the report. Its statements keep their numbering by section; equations are numbered by section (the source numbered them consecutively). Source 12 was written without Parts I and II and names no repository commit; [write] notes at Lemma 34.1, in Section 35 and after Theorem 36.1 relate it to Parts I–II and to the collection, and Section 1.7 describes the Part. Its abstract follows.

We instantiate a fixed reversible, number-conserving universal signal machine with 114 meta-signals, 445 explicitly enumerated binary collision rules, and exactly 18 live signals on every encoded input. The source is Morita’s 15-state, 6-symbol reversible universal Turing machine, compiled through the stack construction of Durand-Lose. Rational finite-support tape encodings occupy a fixed spatial interval; an exact replay checks the published 184-step example at every Turing-machine read. For any fixed-population, equal-cardinality signal machine, complete simultaneous-event dynamics admits the integer branch normal form $N_j = c_j I - c e_j^T$. These gap matrices satisfy $N_j^2 = c_j N_j$ and have coefficients of absolute value at most 24 in the literal

machine. Encoding finite label control in one additional coordinate yields an 18-dimensional partial piecewise homogeneous integer-linear map for the literal machine. A separate binary-alphabet existence construction gives dimension and population ten. Finally, a direct branch-copy construction produces a nonnegative quadratic polynomial with a unique natural witness for each legal step, and canonical quadratic certificates for every fixed finite horizon. No fixed-arity compression of unbounded computation is claimed. The release contains the complete collision table, exact arithmetic code, receipts, and reproducible article source.

29 Results and scope

Signal machines are an unusually direct esoteric substrate for arithmetic descriptions of computation: a finite set of point particles moves at fixed velocities, and only collisions change their labels. Their geometry is continuous, but every finite collision history starting from rational data is rational. The issue is to expose that arithmetic without silently assuming generic position, constant-time tape loading, or a fixed number of witnesses for an unbounded history.

The universal computational mechanisms here are prior work. Durand-Lose supplies reversible conservative stacks and a reversible Turing-machine compiler [16]; Morita supplies the particular universal transition table [17]. Event simulation by linear real-register operations is also established background [4]. This note makes their composition literal, derives a population-dimensional integer normal form with full tie semantics, and gives its canonical quadratic finite-step certificate. Priority of this precise synthesis has not been established by an exhaustive literature search.

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. A *meta-signal* is a label equipped with a speed; a *live signal* is one instance at one position. A fixed finite set of labels is compatible with arbitrarily long computations. Population and alphabet size are different resources.

Theorem 29.1 (Literal universal frontend). *There is a fixed partial signal machine $\mathcal{U}_{18}$ whose complete finite data are included with this note, with the following properties.*

- (i) *It has 114 meta-signals and 445 collision rules. Every rule has exactly two inputs and two outputs, with distinct speeds on each side. Distinct input sets have distinct output sets.*
- (ii) *Its speeds are $\{-12, -9, -4, 0, 4, 9, 12\}$. The least common multiple of all positive speed differences is $D = 65520$.*
- (iii) *Each well-formed finite input to Morita’s explicitly halting cyclic-tag interpreter is loaded into exactly 18 rational-position signals. Before halt these signals remain in $[-8, 8]$. They simulate its reversible Turing machine exactly. The signal table does not depend on the input program or word.*
- (iv) *Designated terminal pairs produce a halt label and an escape token. The token undergoes exactly eight further crossings on these encoded configurations, then no further collision occurs. Infinite encoded computations have unbounded elapsed time.*
- (v) *At event cuts, 17 nonnegative integer gaps and one integer mode coordinate define an exact partial piecewise homogeneous integer-linear map on $\mathbb{N}^{18}$. Finite reachability of its designated accepting modes is universal under the explicit input recoding described below.*

Here “universal” imports the halting-preserving simulation theorem for Morita’s source machine; the finite replay is an implementation check, not a replacement proof of universality. The source distinguishes its phase-zero halt from exhaustion of the current word. Only $(q_1, \mathbf{b})$ is accepting;

$(q_2, \$)$ is nonaccepting null termination. Both outcomes are observable separately in $\mathcal{U}_{18}$. Unspecified source instructions and collisions remain undefined.

Theorem 29.2 (Binary existence bound). *There exists a fixed reversible number-conserving universal signal machine with ten live signals on every effectively encoded input. It uses speeds $\{-6, -3, -2, 0, 2, 3, 6\}$ and has a nine-gap integer lift using the current positive pivot speed c_j as its scale multiplier. Including label control gives a partial piecewise homogeneous integer-linear map on $\mathbb{N}^{10}$ with universal finite accepting reachability. Each gap branch is a rank-eight scaled projection with at most 16 nonzero entries, all of absolute value at most 12. A uniform alternative multiplier is $D = 360$.*

Theorem 29.2 is an existence-by-compiler statement using established binary reversible Turing-machine simulation [18, 17]. This release does not print or instantiate the resulting universal binary interpreter. The fully literal result is Theorem 29.1. Neither bound is claimed optimal.

Theorem 29.3 (Canonical quadratic finite-step certificate). *For either fixed integer map, one can effectively construct a polynomial of total degree at most two which vanishes over natural arguments precisely on legal one-step input/output pairs. Its full auxiliary witness is unique whenever it exists. For each externally fixed horizon K , the same construction gives a quadratic certificate for K steps with size linear in K , the machine being fixed.*

For the literal machine, the release enumerates a forward-closed family of 49,700 modes and 80,501 branches, and supplies the complete numeric mode/event data, an exact sparse coefficient exporter, and a factored polynomial circuit. The one-step packet has exactly 4,196,998 auxiliary variables. It is not expanded into a flat monomial list; that representation is much larger. The binary existence result remains specified by a compiler. The horizon K determines the arity of the certificate. It is not another free variable in a single fixed-arity quadratic polynomial.

30 Primary sources and the literal interpreter

30.1 The source machine and its terminal convention

Morita's Theorem 3.3 and Table 5 specify a 15-state, 6-symbol reversible universal Turing machine with 62 quintuples [17]. Its states are $q_0, \dots, q_{14}$ and its tape alphabet is

$$S = (\mathbf{b}, \mathbf{Y}, \mathbf{N}, *, \$, 1),$$

with $\mathbf{b}$ the blank. A quintuple (q, a, b, d, s) means: in state q scanning a , write b , move one cell in direction $d \in \{-1, +1\}$, and enter s . The complete table is reproduced as mathematical data in Appendix F and in the release JSON.

Two cells have designated terminal meaning:

$$(q_1, \mathbf{b}) = \text{halt}, \quad (q_2, \$) = \text{null}.$$

The other 26 empty cells are unspecified. In particular, q_1 and q_2 are not unconditional halting states: they also have ordinary transitions. Treating the entire states as final would change the computation.

For each target state s , all incoming transitions have one common direction d_s , and their written symbols are distinct. These local conditions imply reversibility: the target state fixes the previous head location; the symbol there identifies the unique predecessor transition, prior state, and overwritten symbol. All 62 transitions were checked against these conditions.

30.2 The finite input language

The source interpreter uses the explicitly halting cyclic-tag convention of Morita’s Definition 3.3. A program has $k \geq 1$ cyclic productions

$$(\text{halt}, p_1, \dots, p_{k-1}), \quad p_i \in \{\mathbf{Y}, \mathbf{N}\}^*.$$

An empty current word gives the outcome null. Reading $\mathbf{N}$ deletes it and advances the phase. Reading $\mathbf{Y}$ in a nonzero phase deletes it, appends that phase’s production, and advances the phase. Reading $\mathbf{Y}$ in phase zero halts. The simulation theorem applies to this convention; no unproved identification with every other cyclic-tag convention is used.

Let $\text{rev}(p)$ denote reversal. At phase zero the finite program region is

$$P = \text{rev}(p_{k-1}) * \dots * \text{rev}(p_1) * \mathbf{b}. \quad (30.1)$$

The tape is $P\$w$, surrounded by infinite blank tails, and the head initially scans the first cell of w , or a blank if w is empty, in state q_0 . Equivalently, start with a delimiter for every production including halt, then replace the rightmost delimiter by the blank phase marker. Other initial phases replace the appropriate delimiter counted from the right. There is finite nonblank support on both sides of the head; there is no nonblank periodic background.

For $(\text{halt}, \mathbf{YN}, \mathbf{YYN})$ and $w = \mathbf{NYY}$, the source Figure 28 starts with

$$\mathbf{NYY} * \mathbf{NY} * \mathbf{b\$NYY}.$$

The printed prose following Table 5 reverses the letters of p_1 relative to the example and figure. We use the general reversal convention and the figure, which agree with the full 184-step table replay. The release documents this discrepancy rather than silently changing the sample.

30.3 What is imported and what is checked

Durand-Lose’s 2012 Sections 3–4 give the stack construction and reversible-TM translation used here. The later shrinking construction in that paper is not used. The 2006 result also establishes reversible conservative universality, but omits some compiler detail and is not an enumerated fixed universal collision table [5]. The 2007 linear-BSS comparison already treats next-event arithmetic [4].

The literal collision table is a new finite instantiation of these ingredients, not a claim to a new universal computational mechanism. Its structural checks, source-tape comparison, and integer event identities are exact. They do not constitute proof-assistant formalization or an exhaustive audit of every source reduction.

31 Finite-support stacks with a fixed number of particles

31.1 Exact rational encoding

For an alphabet of $\ell \geq 2$ symbols, number the digits $1, \dots, \ell$, with digit 1 blank, and put $b = \ell + 1$. Encode a finite digit word followed by infinitely many blanks by

$$\sigma(d_1 \dots d_m) = \sum_{i=1}^m \frac{d_i}{b^i} + \frac{1}{\ell b^m}. \quad (31.1)$$

Let C_ℓ be the set of all such values. Trailing blank digits give the same eventually blank tape, as they should. Delimiters in a source input language determine any required finite-word boundary.

Lemma 31.1 (Stack arithmetic). $C_\ell \subset [1/\ell, 1)$ is closed under

$$\text{push}_v(\sigma) = \frac{v + \sigma}{b}, \quad \text{pop}(\sigma) = (v, b\sigma - v), \quad v = \lfloor b\sigma \rfloor.$$

The pop digit belongs to $\{1, \dots, \ell\}$, the remainder belongs to C_ℓ , and no admissible value lies on a digit boundary v/b .

Proof. Equation (31.1) is the base- b expansion of an infinite digit stream in $\{1, \dots, \ell\}$ eventually equal to 1. Its minimum is the all-1 stream, $1/\ell$; its value is strictly below the all- ℓ stream, which has value 1. Removing or prepending a digit leaves an eventually-1 stream. Thus $b\sigma = v + \rho$ with $\rho \in [1/\ell, 1)$, proving the floor formula and excluding boundaries. $\square$

This is an invariant on the encoded domain. Arbitrary rationals in the same interval are not substitutes: for example $2/3$ is a forbidden binary boundary. The gadget has exact rational dynamics, not a rounding convention for malformed stack values.

31.2 The complete parametric collision gadget

There are stationary marks $m_0, \dots, m_\ell$ at positions $0, \dots, \ell$, one stationary memory signal at σ , and one operation token arriving from the right. Choose a positive speed c and put $a = \ell/(\ell + 2)$. The labels and speeds are

Labels	Speeds
m_i, mem	0
$\text{ml}, \text{sl}_v, \text{gl}, \text{vl}_v$	$-c$
$\text{mr}, \text{sr}_v, \text{ack}, \text{vr}_v$	c
$\text{catch}, \text{fix}, \text{gr}, \text{set}$	$c/3, -ac, ac, -c/3$

The nontransparent rules are, for $1 \leq v \leq \ell$,

$$\{m_0, \text{ml}\} \mapsto \{m_0, \text{mr}\}, \quad \{\text{mem}, \text{sl}_v\} \mapsto \{\text{ml}, \text{sr}_v\}, \quad (31.2)$$

$$\{\text{sr}_v, m_v\} \mapsto \{m_v, \text{catch}\}, \quad \{\text{mr}, \text{catch}\} \mapsto \{\text{ml}, \text{fix}\}, \quad (31.3)$$

$$\{\text{mr}, \text{fix}\} \mapsto \{\text{mem}, \text{ack}\}, \quad \{\text{mem}, \text{gl}\} \mapsto \{\text{ml}, \text{gr}\}, \quad (31.4)$$

$$\{\text{mr}, \text{gr}\} \mapsto \{\text{ml}, \text{set}\}, \quad \{m_v, \text{set}\} \mapsto \{\text{vl}_v, m_v\}, \quad (31.5)$$

$$\{\text{mr}, \text{vl}_v\} \mapsto \{\text{mem}, \text{vr}_v\}. \quad (31.6)$$

For every $1 \leq i \leq \ell$, add identity crossings of m_i with

$$\text{ml}, \text{mr}, \text{fix}, \text{ack}, \text{gl}, \text{gr}, \text{sl}_v, \text{vr}_v \quad (1 \leq v \leq \ell),$$

and with sr_v, vl_v only when $i < v$. All other encounters are undefined. This last clause is essential: completing every unspecified pair by an identity rule can create an output-set collision with a nontransparent rule and destroy reversibility.

There are $5\ell + 10$ labels and

$$5 + 4\ell + 6\ell + 2\ell^2 + \ell(\ell - 1) = 3\ell^2 + 9\ell + 5 \quad (31.7)$$

rules. Thus one binary module has 20 labels and 35 rules, while one six-symbol module has 40 labels and 167 rules. The gadget uses only $\ell + 3$ live signals, including its operation token.

Every displayed rule is two-to-two, with distinct input and output speeds. Its output set determines which displayed case and, where applicable, which v occurred. The transparent outputs are the corresponding untouched inputs, and the listed transparent cases exclude the nontransparent outputs. This proves local rule injectivity; the release also checks it mechanically.

31.3 Geometric proof of push and pop

For push, take time zero at the encounter of mem with sl_v at $x = \sigma$. The memory moves left at speed c , reflects at zero, then moves right at speed c . Meanwhile sr_v goes right to mark v and produces a token of speed $c/3$. The reflected memory and that token meet at

$$x_0 = v + \sigma.$$

They produce leftward memory of speed c and a fix token of speed ac . After the memory reflects once more, their encounter has coordinate

$$x_0 \frac{1-a}{1+a} = \frac{v+\sigma}{\ell+1}.$$

The memory becomes stationary there and the acknowledgement leaves to the right. This is exactly push.

For pop, at the first memory/get encounter the memory moves left and reflects while the get token moves right at speed ac . Their meeting coordinate is

$$\sigma \frac{1+a}{1-a} = (\ell+1)\sigma = v + \rho, \quad \rho \in C_\ell.$$

The new set token moves left at speed $c/3$, so its first lower integer mark is exactly v . That mark turns it into the leftward value token vl_v of speed c . The reflected memory meets this token at ρ , becomes stationary, and returns vr_v to the right.

All additional marker crossings are precisely those in the transparent list. The values $\sigma, \rho \in [1/\ell, 1)$ keep the critical meetings away from integer marks. Each construction has a fixed finite number of moving segments and finitely many marker crossings, hence a uniform finite collision bound for fixed ℓ . Its largest local coordinate is less than $\ell + 1$. The memory and operation token remain the same two live resources throughout.

32 Reversible control and the fixed population

32.1 Alternating write and move form

Let a reversible Turing machine have disjoint write states Q_W and move states Q_M , with partial maps

$$\delta_W : Q_W \times S \longrightarrow Q_M \times S, \quad \delta_M : Q_M \longrightarrow Q_W \times \{L, R\}.$$

Require δ_W to be injective and the projection of δ_M onto Q_W to be injective. Thus each target write state has a unique preceding move and a fixed incoming direction. Require also that every move-state in the image of δ_W is in the domain of δ_M , and every used noninitial write or terminal state has an incoming move. The initial read side must agree with any existing incoming direction. A designated terminal *pair* (q, a) has no writing transition; other undefined pairs remain forbidden.

The compiler may alternatively be given states whose every symbol is terminal, but this stronger condition is not imposed on Morita's table.

For a reversible quintuple table with nonzero movements, replace each transition (q, a, b, d_s, s) by the write $(q, a) \mapsto (r_s, b)$ followed by $r_s \mapsto (s, d_s)$, using one fresh move state for each target s . The local reversibility conditions from Section 30 give exactly the two injectivity requirements. Morita's table yields 15 move states and 62 writing transitions. Its q_0 has incoming direction right, so the initial read can use the right module consistently even though q_0 has predecessors.

32.2 Layout and tape invariant

Put $R = \ell + 2$. The left module has marks at $-R + i$ and memory at $-R + \sigma_L$; the right module is its reflected, disjointly labelled copy, with marks at $R - i$ and memory at $R - \sigma_R$. Put the current controller state at zero. There is one travelling operation/value token shared by the modules. The population is

$$2((\ell + 1) \text{ marks} + 1 \text{ memory}) + 1 \text{ controller} + 1 \text{ token} = 2\ell + 6. \quad (32.1)$$

In particular, $\ell = 6$ gives 18 and $\ell = 2$ gives ten. The available state labels depend on the fixed interpreter, but the number of live controller signals is always one.

At a controller read, the arriving value token holds the scanned symbol; the two stationary memories hold the cells strictly left and strictly right of the head, nearest cell first. Suppose the current write sends (q, a) to (r, b) and the next move is $r \mapsto (s, d)$. The controller sends a push- b token to the side opposite d . On acknowledgement it changes to s and sends a pop request to side d . The returned value is the new scanned symbol. This preserves the tape invariant and performs exactly one source transition.

Distinct writing rules cannot share an output unless they share both (r, b) , which injectivity forbids. Distinct acknowledgement rules cannot share an output unless they share the target s , also forbidden. The writing and acknowledgement output phases have disjoint controller-state sets. Gadget outputs contain no controller state. These observations prove injectivity of the compiled control rules and their separation from the gadget rules.

32.3 Terminal exits and conservation

For each designated terminal pair (q, a) , replace its arriving controller/value pair by a fresh stationary label $H_{q,a}$ and a fresh token $E_{q,a}$ moving right at maximum speed. Add transparent crossings of this escape label with every stationary marker and memory label. All these rows are two-to-two and their fresh labels preserve output injectivity.

At that time both memories and all marks are stationary. On the right there are $\ell + 1$ marks and one memory, all at distinct positions. Hence exactly $\ell + 2$ further crossings take the token to the rightmost mark; thereafter it has no future encounter. This is eight crossings for the literal machine and four for the binary construction. The larger bound of all $2\ell + 4$ stationary module signals is also immediate but unnecessary.

A terminal mode is recorded by the presence of $H_{q,a}$. Only the specified accepting subset is a reachability target: in the literal instance this is $H_{q_1,b}$, while $H_{q_2,s}$ records nonaccepting termination. Such a cut can precede the last escape crossing. If a collision-free cut is desired, additionally require the token to be rightmost and every adjacent velocity difference nonpositive. For an accepting label the two acceptance cuts differ by a fixed finite suffix. Acceptance is never a Zeno-limit convention.

Giving every signal energy one makes every rule conserve both number and this positive energy. Local reversibility refers to the partial collision table and finite well-defined flow; no uniqueness assertion about continuation through singular accumulation points is needed.

32.4 No accumulation on encoded runs

All active stack geometry remains outside $(-1, 1)$, apart from the controller token travelling between the center and a gadget. A center-to-gadget-to-center exchange covers distance at least two at speed at most $c_{\max}$, so takes at least $2/c_{\max}$ time. Each exchange contains only uniformly finitely many collisions, by Section 31.3. Thus an infinite encoded computation requires infinitely many exchanges and unbounded time. A finite number of source steps has finitely many events and finite duration. This excludes Zeno acceleration on the encoded domain without making a claim about every malformed rational configuration of the partial machine.

The outer marks remain at $-R, R$. At every event cut on an encoded run, including the final escape crossing, the span is exactly $2R$. The token later flies beyond R , but those later non-event observation times are not states of the event map.

32.5 The ten-signal existence specialization

Morita, Shirasaki and Gono give a simulation by a one-tape two-symbol reversible Turing machine [18]; the blank is included in that two-symbol alphabet [17]. Fix one universal ordinary interpreter and apply that conversion. If its reversible description uses separate write/move quadruples, writing becomes a zero-move quintuple and a movement becomes one identity-writing quintuple per symbol. Alphabet size does not change.

Factor transitions by their target state as above. A zero-move entry into s can be replaced by a right move into a fresh write state z_s , an identity write into a fresh move state t_s , and a left move into s . Both new maps satisfy the same injectivity conditions. The detour restores the head and tape. Initial entry is chosen consistently with the existing incoming direction, using the two orientations of the loader below; if there is no incoming move either entry can be chosen. Halting can be designated by the finitely many undefined terminal pairs. Applying the binary gadget proves Theorem 29.2; this is a compiled-existence argument, not a second printed universal table.

33 The exact input loader and its cost

33.1 Both finite tape halves

At the initial head position, list the cells strictly to its left in decreasing index order to form w_L . List the scanned cell and the cells to its right in increasing order to form w_R . Include every nonblank cell; omitted tails are blank. This works whether finite data lie on one side or both sides of the head, as happens in Morita's program/data layout.

Let $m = \max(|w_L|, |w_R|)$, append blank digits to the shorter word, and define

$$Q = \ell b^m, \quad A_\epsilon = \ell \sum_{i=1}^m d_{\epsilon,i} b^{m-i} + 1 \quad (\epsilon \in \{L, R\}).$$

Then $\sigma_\epsilon = A_\epsilon/Q$. Place the initial pop token at $x = 1$ pointing toward the right module. All initial labels and their order are fixed. Only two memory coordinates vary.

The ordered gap numerators, with a run of r equal entries denoted $Q^{[r]}$, are

$$h_0 = (A_L, Q - A_L, Q^{[\ell-1]}, 2Q, Q, Q, Q^{[\ell-1]}, Q - A_R, A_R). \quad (33.1)$$

There are $2\ell + 5$ entries, whose sum is $2(\ell + 2)Q$. For the binary case this is

$$h_0 = (A_L, Q - A_L, Q, 2Q, Q, Q, Q, Q - A_R, A_R). \quad (33.2)$$

For the literal machine it gives 17 gaps and span $16Q$. The initial mode coordinate, introduced below, is its fixed mode code times this sum.

If the initial state instead has left incoming direction, include the scanned cell in w_L and exclude it from w_R . Place a leftward pop token at -1 . The binary gap vector becomes

$$(A_L, Q - A_L, Q, Q, Q, 2Q, Q, Q - A_R, A_R).$$

For general ℓ , the vector is

$$(A_L, Q - A_L, Q^{[\ell-1]}, Q, Q, 2Q, Q^{[\ell-1]}, Q - A_R, A_R).$$

These are two fixed entry conventions, each with a fixed initial label order. The literal six-symbol interpreter uses right entry.

33.2 Arithmetic and representation cost

The rational loader uses $O(m)$ multiply-by- b and add-digit operations on $O(m)$ -bit integers for fixed alphabet. This is linear in the prepared tape-word length in an arithmetic-operation model; it is not a constant-cost conversion from a raw base-two integer or a claim of linear bit complexity with arbitrary multiplication primitives. Ordinary binary input integers require digit extraction, source-interpreter recoding, length/power construction, and base conversion. Those costs must be paid either in preprocessing or by a separate arithmetic representation theorem.

The source program in (30.1) has length linear in its explicit production description, and its two tape halves can be obtained effectively. The separate universal reduction into that source language is imported from the cited simulation theorem, not counted as free computation.

Keeping rational positions gives a fixed physical box. Scaling all positions and times by Q instead gives integer initial positions with the same fixed speed and rule tables, at the price of input-dependent physical scale. These two conventions have the same live population and event ordering.

34 The complete simultaneous-event normal form

The next results do not depend on stacks or Turing machines. Fix any finite partial signal machine with rational speeds and fixed population $n \geq 2$, whose rules preserve the cardinality of each collision block and whose input and output sets each have pairwise distinct speeds. Rescale the time unit once so all speeds are integers.

34.1 Post-event germs and first-collision guards

A state at an event cut is an ordered label word $a_1, \dots, a_n$ and a gap vector $g \in \mathbb{Q}_{\geq 0}^{n-1}$. Put $v_i = v(a_i)$ and $c_i = v_i - v_{i+1}$. We require

$$g_i = 0 \implies v_i < v_{i+1}. \quad (34.1)$$

Thus signals emitted at the same point are sorted by strictly increasing outgoing speed. This is a post-event *germ*, not an arbitrary coincident snapshot. It prevents zero-time recollision and equal-speed overlap. Strictly ordered initial slices satisfy the condition automatically.

If every $c_i \leq 0$, there is no future collision. Otherwise each closing edge has $g_i > 0$ by (34.1); its putative encounter time is g_i/c_i . Let J be the nonempty set attaining the smallest such time and fix the canonical pivot $j = \min J$. For a fixed mode and candidate J , the exact branch guards are

$$c_j > 0, \quad g_j > 0, \quad (34.2)$$

$$c_j g_i - c_i g_j = 0 \quad (i \in J), \quad (34.3)$$

$$c_j g_i - c_i g_j > 0 \quad (i \notin J). \quad (34.4)$$

The first condition is a compile-time test on speeds. These guards imply that every $i \in J$ is closing; equality would otherwise contradict nonnegativity and the germ condition.

Lemma 34.1 (Complete event selection). *Under the germ condition and guards (34.2)–(34.4), the next positive collision time is $\tau = g_j/c_j$. Exactly the adjacent edges in J have zero gap then. The maximal connected components of J are the complete collision blocks.*

Proof. Every gap evolves affinely as $g_i - c_i t$. At $t = \tau$, multiplication by $c_j > 0$ gives exactly the expressions in (34.3)–(34.4). The endpoint gaps are nonnegative, zero precisely for $i \in J$. An initially positive gap remains positive for $0 < t < \tau$, as a convex combination of its positive initial value and nonnegative endpoint. An initially zero gap has $c_i < 0$ and is strictly positive for every $t > 0$. Hence no collision is earlier. Zero endpoint gaps join exactly the contiguous signals occupying the same endpoint position. $\square$

[write] Lemma 34.1 is the one-batch case of the chamber theorem of Part I, Theorem 5.3, with its typed endpoints (Lemma 5.2); Part II proves the same step as Lemmas 21.1 and 21.2. The guards (34.3)–(34.4) are exactly Part I’s equality and strict rows for one batch, written after multiplication by the positive pivot speed, and a batch with several components of J is a batch of several sites in Part I’s terms. The one addition is the germ condition (34.1): Parts I and II start from strictly ordered positions, while an event cut here may carry zero gaps between diverging outputs, and the proof’s sentence on initially zero gaps disposes of them. Source 12 was written without these Parts; its proof is kept as a second route. What is new in this Part is the use made of the lemma: the mode-indexed integer map of Section 35, the pivot-scaled lift, and the literal universal machine to which they are applied.

For every nontrivial collision block, its exact incoming set must be a defined rule input. Apply all such rules atomically and sort each output by speed. An undefined block has no branch. Thus a triple arrival cannot be replaced by a preferred pairwise rule, and simultaneous collisions at distinct locations cannot be processed with an accidental temporal priority. The literal table has only two-input rules, so a genuine triple collision is undefined.

34.2 Why no coordinate permutation is needed

The endpoint gap vector before label replacement is

$$\bar{g} = A_j g, \quad A_j = I - \frac{c e_j^T}{c_j}. \quad (34.5)$$

Each collision replaces k signals at one position by exactly k signals at that position. Therefore the ordered multiset of positions is unchanged by replacement. Reordering outgoing labels at a coincident point changes the mode, but not the gap vector. Consequently $g' = \bar{g}$: equation (34.5) is the entire branch map even when several collision blocks occur simultaneously.

The matrix is a rank-one *correction* to the identity, not a rank-one matrix. Equal-cardinality at each block is important. Merely conserving the total population across a collection of events would not by itself imply this coordinate identity.

35 Integer lifting and internalized finite control

35.1 The pivot-scaled integer lift

For a branch with canonical pivot j , the rational projection from (34.5) has only the denominator c_j . Multiplying by that very denominator gives

$$h_{k+1} = N_j h_k, \quad N_j = c_j I - c e_j^T \in \mathbb{Z}^{(n-1) \times (n-1)}. \quad (35.1)$$

Choose Q clearing the initial gap denominators and let

$$\lambda_0 = Q, \quad \lambda_{k+1} = c_{j_k} \lambda_k, \quad h_k = \lambda_k g_k. \quad (35.2)$$

Here c_{j_k} is the positive closing speed of the uniquely selected branch at step k . All guards are homogeneous, so they apply unchanged to h_k . The multiplier is determined by the branch, and no extra scale coordinate, division, gcd reduction, or denominator witness is needed in the integer map.

Conversely, a legal integer orbit determines its branch sequence, hence all λ_k . Set $g_k = h_k / \lambda_k$ and reconstruct the event delay as

$$\tau_k = \frac{h_{k,j_k}}{\lambda_k c_{j_k}}.$$

Lemma 34.1 and induction reconstruct exactly the rational geometry, up to global translation. This proves finite-orbit equivalence in both directions, including simultaneous events.

Proposition 35.1 (Sparse scaled projections). *For every branch with $c_j > 0$,*

$$A_j^2 = A_j, \quad N_j^2 = c_j N_j, \quad \text{rank } N_j = n - 2.$$

Row j vanishes. Every other row has a diagonal entry c_j and possibly one further entry in column j . There are at most $2(n - 2)$ nonzero entries. If all speed differences have absolute value at most V , every matrix entry has absolute value at most V .

Proof. Since $e_j^T c = c_j$, multiplication expands to $A_j^2 = A_j$. Its kernel is exactly the span of the nonzero vector c : $A_j c = 0$, while $A_j x = 0$ implies $x = c x_j / c_j$. Its rank is $(n - 1) - 1$. Since $N_j = c_j A_j$, its square and rank follow. For $i \neq j$ the only possible nonzero row entries are c_j in column i and $-c_i$ in column j , proving the sparse and coefficient bounds. $\square$

For the literal machine, the 17-dimensional gap matrices have rank 16, at most 32 nonzero entries, and entries of absolute value at most 24. The binary existence construction has nine-dimensional gap matrices of rank eight, at most 16 nonzero entries, and entries of absolute value at most 12. These bounds do not depend on the source interpreter's number of states.

35.2 A uniform alternative for comparison

It is also valid to use one fixed multiplier

$$D = \text{lcm}\{|v - v'| : v \neq v' \text{ are available integer speeds}\},$$

with $D = 1$ if the set is empty. This gives $\tilde{h}_k = QD^k g_k$ and

$$\tilde{h}_{k+1} = M_j \tilde{h}_k, \quad M_j = DI - \frac{D}{c_j} c e_j^\top, \quad M_j^2 = DM_j. \quad (35.3)$$

The literal machine has $D = 65520$, with coefficient bound $8D = 524160$ because the minimum positive speed difference is 3. The binary speeds give $D = 360$ and coefficient bound 4320. This alternative is exact and useful for cross-checking the replay, but is unnecessary for a pure integer reachability frontend: the pivot-scaled matrices are much smaller. Both conventions have the same homogeneous guards and mode sequence.

[write] The multiplier D is the least common multiple of the nonzero speed differences used in Parts I and II ((7.1); (18.1)), and the uniform lift is their device: after k events every gap is an integer once multiplied by QD^k , which is the event-by-event denominator bound of Theorem 7.1 (with the event count bounding the causal depth) and the grid $B = D^K$ of Part II on which Lemma 19.1 places the event coordinates. The pivot-scaled lift (35.2) sharpens that device for one branch sequence: it multiplies by the actual pivot speed c_j instead of D and needs no extra coordinate, and on the primary example it ends with a largest gap numerator of 39,320 bits against 179,449 under the uniform QD^k lift (Section 37).

35.3 Control uses one coordinate

There are finitely many ordered n -label modes; enumerate them injectively by positive integers q . This can be done lexicographically, including modes later discarded as invalid. Put

$$S = \mathbf{1}^\top h, \quad u = qS, \quad X = (h, u) \in \mathbb{N}^n. \quad (35.4)$$

Require $S > 0$. This mode encoding is used on the positive-span invariant domain of the compiled machines. An arbitrary signal machine can collapse its entire population to one point; at such a zero-span cut this particular coordinate does not distinguish outgoing modes and needs a separate convention. The generic gap theorem itself does not require positive span. In fact, positive span is invariant under every legal step of either compiled table, not only its encoded runs: zero output span would mean that all n particles form one complete collision block, requiring an n -input rule, whereas these tables have only two-input rules and $n > 2$. Each branch for mode q includes the homogeneous equality $u - qS = 0$, applies (35.1), and sets

$$u' = q' \mathbf{1}^\top N_j h, \quad (35.5)$$

where q' is the uniquely determined outgoing mode. These equations define a homogeneous integer-linear matrix $\widehat{M}_r$ on all n coordinates. Its last input column is zero; the mode is read through its guard. The small coefficient bounds 24 and 12 apply to the gap block; the added mode row contains the fixed, potentially large codes q' .

The guard also includes the germ conditions. On natural gap variables these are particularly simple: whenever $c_i \geq 0$, require $h_i > 0$; when $c_i < 0$, no extra constraint is needed beyond $h_i \geq 0$. The remaining guards are the first-collision equalities and inequalities. They are homogeneous

rational polyhedral-cone conditions, with strict positivity interpreted on natural arguments as value at least one.

For the literal release, give the 114 labels their zero-based lexicographic indices $\iota(a)$ and use the explicit code

$$q(a_1, \dots, a_{18}) = 1 + \sum_{i=1}^{18} \iota(a_i) 114^{18-i}.$$

The initial and final collision-free modes have codes

$$\begin{aligned} q_{\text{initial}} &= 473631179350110573467332809270905830, \\ q_{\text{accept}} &= 473631179350110518412162422097252939, \\ q_{\text{null}} &= 473631179350110521664110943253890124. \end{aligned}$$

Their exact label orders and decimal codes are included in the machine JSON. Thus $u_0 = 16q_{\text{initial}}Q$, while the final accepting target is the single equality $u = q_{\text{accept}} \sum h$ on admissible post-event germs. The null code is a different nonaccepting target. The code values are serialized as strings to avoid loss of precision in JSON consumers using floating-point numbers.

Distinct mode codes cannot overlap when $S > 0$. For a fixed valid mode and gap vector, the set of minimizing edges is unique and its pivot is fixed. The branch guards are consequently disjoint. Enumerating every mode and every nonempty subset of edges, discarding impossible or undefined cases, is a complete finite branch compiler. This proves the pure n -coordinate claims of Theorems 29.1 and 29.2.

35.4 Limits of the normal form

No injectivity of the integer event map is asserted. Reversing a signal flow, quotienting global translation, forgetting elapsed time, taking an event section, and changing the common scale are different operations. The original partial collision table is reversible; that does not automatically make (35.1) invertible.

For a general configuration, summing the pivot-scaled update gives

$$\mathbf{1}^\top N_j h = c_j \mathbf{1}^\top h - (v_1 - v_n) h_j.$$

It is not valid in general to discard the second term. On encoded runs, the extreme markers are stationary at the input of every event, including the last escape crossing. Their fixed event-cut span gives

$$S_k = \sum_i h_{k,i} = 2(\ell + 2)\lambda_k, \quad S_{k+1} = c_{j_k} S_k.$$

At the last cut the token may be the rightmost outgoing germ, but there is no subsequent event requiring stationary extrema. For all these encoded cuts the geometry and time can be decoded directly from the integer state:

$$g_k = \frac{2(\ell + 2)}{S_k} h_k, \quad \tau_k = \frac{2(\ell + 2)h_{k,j_k}}{S_k c_{j_k}}.$$

For the literal machine $S_k \leq 16Q 24^k$, and for the binary specialization $S_k \leq 8Q 12^k$. Every gap is bounded by S_k , and $u_k \leq q_{\text{max}} S_k$. Their bit length is $O(k + \log Q)$ for the fixed machine. Fixed coordinate dimension does not make integer-height growth free.

Exact points and accepting modes are different targets. In the literal mode closure of Appendix H, every active branch has stationary extreme labels, and every pivot speed is at least 3. Thus $S' \geq 3S$. Given two exact positive-span integer states $X_{\text{in}}, X_{\text{out}}$ in this domain, point-to-point reachability is decidable by deterministic simulation for at most $\lfloor \log_3(S_{\text{out}}/S_{\text{in}}) \rfloor$ steps when the ratio is at least one; a smaller target span rules out a positive-length run. For the fixed machine this costs polynomial time in the binary endpoint length. The universal question is reaching the fixed accepting mode or cone with unbounded output span, not reaching one prescribed integer point. No endpoint-size bound for acceptance is supplied.

Optional exact elapsed time. One additional natural coordinate suffices if the absolute collision timestamp is wanted. Starting at time zero, let $p_k = \lambda_k t_k$. The same branch updates it by

$$p_{k+1} = c_{j_k} p_k + h_{k,j_k}, \quad p_0 = 0.$$

Indeed, multiplying $t_{k+1} = t_k + h_{k,j_k}/(\lambda_k c_{j_k})$ by $\lambda_{k+1} = c_{j_k} \lambda_k$ gives exactly this integer-linear row. On literal encoded runs the timestamp is $t_k = 16p_k/S_k$. The timed frontend has 19 coordinates, or 11 in the binary specialization; the extra row still has coefficients bounded by 24, respectively 12. Its finite-horizon packet remains quadratic. This optional coordinate is separate from the 18-dimensional untimed reachability theorem.

The dimension retains every physical collision and is not an optimized arithmetic state dimension. For comparison, at completed Turing-machine reads a standard two-stack macro-description can use (A_L, A_R, Q, u) , with u coding the finite state and scanned-symbol pair times Q . A right move writing w and popping v from the right can use

$$Q' = 7Q, \quad A'_L = A_L + wQ, \quad A'_R = 49A_R - 7vQ,$$

with digit guard $vQ < 7A_R < (v+1)Q$; left moves are symmetric. This coarser quotient skips internal collisions and is distinct from the sparse projection frontend. Elementary stack arithmetic and the selector/copy technique are used as standard algebraic mechanisms, not claimed as new inventions.

[write] The selector/copy construction of Section 36, in which one selector per branch and one copy of the input per branch represent a disjoint union of polyhedral pieces, is the disjunctive (convex-hull) formulation of a union of polyhedra due to Balas [26, 27], here in homogeneous form with the selectors entering the strict guards; Jeroslow and Lowe [28] characterized the sets that such formulations represent with integer variables. Source 12 cites neither; the credit is added here, and source 12 claims no novelty for the technique.

36 A direct canonical quadratic Diophantine packet

36.1 Finite linear branch systems

Let F be a partial map on $\mathbb{N}^d$ with finitely many disjoint branches $r = 1, \dots, B$. Branch r has update $Y = \widehat{M}_r X$, where $\widehat{M}_r$ has integer entries, and finitely many homogeneous integer-linear guards of three kinds:

$$L(X) = 0, \quad L(X) > 0, \quad L(X) \geq 0.$$

The event map above has this form with $d = n$. Naturality of the input coordinates is part of the domain; its mode, germ, nonzero-span, first-collision, tie, and collision-legality conditions are included in the branch description. Collision legality is a finite compile-time filter.

For every branch introduce a selector $b_r \in \mathbb{N}$, a local input copy $X_r \in \mathbb{N}^d$, and one slack $s_{r,L} \in \mathbb{N}$ for each strict or weak inequality. Form the following affine residuals:

$$\sum_{r=1}^B b_r - 1, \quad (36.1)$$

$$X_i - \sum_r X_{r,i} \quad (1 \leq i \leq d), \quad (36.2)$$

$$Y_i - \sum_r (\widehat{M}_r X_r)_i \quad (1 \leq i \leq d), \quad (36.3)$$

$$L(X_r) \quad \text{for each equality guard}, \quad (36.4)$$

$$L(X_r) - b_r - s_{r,L} \quad \text{for each strict guard}, \quad (36.5)$$

$$L(X_r) - s_{r,L} \quad \text{for each weak guard}. \quad (36.6)$$

Let $\mathcal{R}$ be this list and define the explicit polynomial

$$P(X, Y; \mathbf{z}) = \sum_{R \in \mathcal{R}} R^2 + \sum_{r=1}^B \left(\sum_{h \neq r} b_h \right) \left(\sum_{i=1}^d X_{r,i} \right). \quad (36.7)$$

All sums here are finite and their coefficients are directly computable from the branch table. The second group consists of nonnegative complementarity products on natural arguments. No appeal to MRDP is involved.

Theorem 36.1 (Exactness and canonical witness). *For $X, Y \in \mathbb{N}^d$, there exists $\mathbf{z} \in \mathbb{N}^A$ with $P(X, Y; \mathbf{z}) = 0$ if and only if $Y = F(X)$. Whenever such a tuple exists, it is unique. The polynomial has total degree at most two and is nonnegative on every natural assignment.*

Proof. Every square and every displayed complementarity product is nonnegative on natural arguments. A zero therefore makes all terms vanish. Equation (36.1) forces exactly one selector, say b_{r_*} , to be 1 and all the others to be zero. For every $r \neq r_*$, the first factor of its complementarity term is 1, so all coordinates of X_r are zero. Equation (36.2) then gives $X_{r_*} = X$.

Homogeneity makes every inactive guard form zero; equations (36.5) and (36.6) consequently force its slack to zero. On the active branch an equality guard vanishes, a strict guard has value $1 + s_{r_*,L} \geq 1$, and a weak guard has value $s_{r_*,L} \geq 0$. Thus X lies on that branch and (36.3) gives $Y = \widehat{M}_{r_*} X$.

Conversely, for a legal step set its unique branch selector to 1, set its local copy to X , set its strict slack to $L(X) - 1$ and weak slack to $L(X)$, and set every inactive quantity to zero. These are natural values and every term vanishes. Disjoint branch guards fix the active selector, and all copies and slacks are then forced, proving uniqueness. The residuals are affine and the additional products are quadratic, proving the degree bound. $\square$

The product must use $\sum_{h \neq r} b_h$, as written. Replacing it by $1 - b_r$ would lose global nonnegativity on arbitrary natural assignments before the selector equation has been enforced. The polynomial is not asserted to be convex or a sum of squares over all real arguments.

[write] Two remarks place Theorem 36.1 in the collection. (i) Source 12 states nonnegativity on natural arguments only, but P is nonnegative on the whole real nonnegative orthant: the residuals enter squared, and each complementarity product $(\sum_{h \neq r} b_h)(\sum_i X_{r,i})$ is a product of two affine forms with nonnegative coefficients. So (36.7) has exactly the form $\sum_i A_i^2 + \sum_j B_j C_j$ of the spine

of *quadratic-orthant-certificates* (its equation `qoc:eq:spine`), the format of that report’s three Parts. This is a statement about the sign of P only; no claim is made here that the nonnegative real zeros of P are natural. (ii) The one-step graph $\{(X, F(X))\}$ of a homogeneous-guarded piecewise-linear map on $\mathbb{N}^d$ with finitely many branches is a semilinear set, so the *existence* of a single-fold representation of degree two, nonnegative on the real orthant, also follows from Theorem `cdc:of:thm:singlefolds1` of Part XI of *canonical-diophantine-certificates*, via Presburger quantifier elimination. The construction here is explicit, needs no elimination, and carries the exact ledger (36.8)–(36.9); that is what this Part contributes, not the existence statement. The fixed-horizon sum (36.10) is of the same “one natural zero, fixed horizon” kind as the trace polynomial of Theorem `pqc:pm:thm:trace` in *probabilistic-quantum-and-continuous-computation*, which handles a different dynamics with a clamp gadget.

[write] **Added 2 October 2026 (batch 79): the strong selector gates of *quadratic-orthant-certificates*.** Part V of that report (its source 16, batch-79 manuscript 09, written in commit 11abe5008) uses the same selector device for the branches of literal counter programs, with one change: its products $(\sum_{k \neq r} e_{j,k})(e_{j,r} + \sum_i x_{j,r,i})$ (its equation `qoc:um:eq:polynomial`) add the selector itself to the right factor. Two positive selectors would then make some product positive, so the gate forces one-hot selection over the nonnegative reals, and for natural inputs the nonnegative real zero fibre of that polynomial is its natural fibre (its Theorem `qoc:um:thm:poly`); its source 17 isolates the gate as Lemma `qoc:rn:lem:gates` and uses it again for reset-net traces in its Part VI. The products of (36.7) lack that term, and source 12 claims exactness over natural witnesses only. An elementary observation, not made by either source, shows how far the present gates reach. At a nonnegative real zero of (36.7), if two selectors were positive, every first factor $\sum_{h \neq r} b_h$ would be positive, so every local copy X_r would vanish and $X = 0$ by (36.2). Hence, for natural $X \neq 0$ and natural Y , exactly one selector equals one, every inactive copy and slack vanishes, the active slacks are the integers $L(X) - 1$ and $L(X)$, and the nonnegative real fibre is the natural one; only at $X = 0$ do the gates alone leave fractional selectors possible, among branches without a strict guard, since (36.5) forces $b_r = 0$ at $X_r = 0$. Adding b_r to each right factor, as in that report, would remove this exception without changing the counts (36.8)–(36.9). Neither report re-proves a theorem of the other.

36.2 The exact witness and residual ledger

Let E, T, W be, respectively, the total counts of equality, strict, and weak guards across all B branches. There are

$$A = B(d + 1) + T + W \quad \text{auxiliary natural variables,} \quad (36.8)$$

$$R = 1 + 2d + E + T + W \quad \text{squared affine residuals,} \quad (36.9)$$

$$B \quad \text{unsquared nonnegative quadratic products.}$$

The variables are exactly B selectors, Bd local coordinates, and $T + W$ slacks. For the literal frontend, replace d by 18, so $A = 19B + T + W$ and $R = 37 + E + T + W$. For the binary existence frontend, $A = 11B + T + W$ and $R = 21 + E + T + W$.

This is a variable-and-residual ledger, not a minimal arithmetic-gate count. Expanding a squared linear form may create many monomials. The release gives the literal machine’s numeric ledger and a complete factored coefficient description in Appendix H. No minimization or flat monomial expansion is claimed.

36.3 Fixed horizons and terminal reachability

For an externally fixed $K \geq 1$, introduce intermediate states $X_1, \dots, X_{K-1}$ and a separate packet witness $\mathbf{z}_k$ for each step. Define

$$P_K(X_0, X_K; \mathbf{Z}) = \sum_{k=0}^{K-1} P(X_k, X_{k+1}; \mathbf{z}_k). \quad (36.10)$$

There are exactly $KA + (K - 1)d$ witness variables with the two endpoints supplied as parameters, KR squared affine residuals, and KB complementarity products. The polynomial is quadratic and nonnegative on natural arguments. By Theorem 36.1, its natural fiber is empty or a singleton: determinism fixes every intermediate state, and the step witnesses are canonical.

A designated accepting mode is a finite disjunction of homogeneous linear mode equalities, optionally with the linear collision-free condition. Its finite disjunction can be handled by the same selector/copy construction. If the intended question is *first* acceptance at step K , disallow accepting input modes in the earlier branch packets. Nonaccepting null modes are never promoted to accepting ones. Reaching H and finishing the escape suffix are separate predicates, as discussed above.

The statement “there exists some finite K ” is an unbounded union of these fixed-horizon relations. A uniform fixed-arity Diophantine representation of that union would need an additional sequence or iteration compression theorem. None is supplied here. Universality of the dynamics and existence of a direct quadratic *step* polynomial do not remove that requirement.

37 Exact replay and the release contract

37.1 Literal machine counts

For $\ell = 6$, the two stack copies contribute 80 meta-signals and 334 rules. The controller has 15 write-state labels and 15 move-state labels. The 62 source transitions give 62 writing rules and the 15 target states give 15 acknowledgement rules. Two designated terminal pairs contribute four fresh labels, two terminal rules, and 32 transparent escape rules, one per terminal pair and each of the 16 stationary module labels. Hence

$$80 + 30 + 4 = 114, \quad 334 + 62 + 15 + 2 + 32 = 445.$$

The exported table lists every label with its integer speed and every input/output pair. Undefined encounters are explicitly forbidden; there is no implicit identity completion.

37.2 Replays and their meaning

The Python replay uses standard-library integer arithmetic and exact fractions. The core stack suite checks 1,120 operations over alphabets 2, 3 and 4, totaling 24,028 collisions and 34 simultaneous batches. A small compiled binary source machine is tested on all 63 binary words of length at most five, totaling 5,045 batches. These tests cover gadget behavior, conservation, local injectivity, complete earliest-event guards, simultaneous processing, and the rational/integer branch identity.

The source-machine audit checks all 184 transitions of Morita’s published example in both directions. The Figure 28 snapshots at times 0, 6, 59, 178 and 184 match state, head, and full displayed tape window. A separate bounded differential suite covers 2,430 source cyclic-tag cases,

7,687 macro checks and 169,019 Turing microsteps; 40 cases remain active at its 12-macrostep bound. A q_0 state alone is not a reliable macro boundary, so that test also compares the restored program, phase marker, and active word.

For the literal signal machine, each controller read is checked against an independent ordinary tape: controller state, returned symbol, and both complete rational half-stack values must all agree. The tested cases are

Program and word	TM steps	Signal batches	Outcome
(halt, YN, YYN), NYY	184	11214	halt
Same program, Y	2	154	halt
Same program, empty word	1	106	null
(halt), N	11	722	null
(halt, ε), NY	25	1602	null
Total	223	13798	

All these signal batches are single-collision batches, and each has an eight-crossing escape suffix. The primary example's final exact clock is

$$\frac{142256492930823919}{199397583417606}.$$

An additional event-timing implementation computes all pairwise intersections and groups coincident endpoints independently of the adjacent-gap selector. It matches all 11,214 primary-example batches and verifies the cumulative QD^k integer lift at every batch; the final largest numerator has 179,449 bits. Its generic suite checks 8,092 valid germs, including 1,184 simultaneous-event batches, 352 triple-or-larger events, and 3,430 initially zero-gap germs. A missing triple rule is explicitly rejected. There are also 1,296 two-half loader cases and 32 two-entry-side compiled runs totaling 2,574 batches. The pivot-scaled suite independently checks all 13,798 literal batches and 7,282 generic events, including 4,708 events whose total span changes. The primary example's final largest gap numerator has 39,320 bits under pivot scaling, compared with 179,449 under uniform scaling. It also checks exact fixed-span geometry and delay decoding, the production adaptive-lift helper, and the optional timestamp numerator at every event; the primary example's final time numerator has 39,329 bits. These are checks of the same rule data, not an independent proof of universality.

These finite comparisons substantiate the literal implementation. The proof for all inputs is the stack geometry, control invariant, and imported source universality theorem, not extrapolation from the tested cases.

37.3 Reproduction and boundaries

The release has four parts: the article source and PDF; original exact replay code; complete mathematical machine data; and public-safe replay receipts with provenance links. The run script recreates the source table and its checks, the generic stack checks, and the literal signal table and replays. A separate generic event-branch compiler and arithmetic packet demonstration exercise exact canonical witnesses and residual counts on a fully enumerated finite branch system. A separate closure compiler enumerates a forward-closed mode family for the literal initial mode and records all 80,501 numeric event branches, as described in Appendix H. This avoids enumeration of all 114^{18} possible label words.

No third-party source PDFs, source-page images, private working notes, or third-party software are bundled. Source PDFs can be obtained from the linked primary locations; their SHA-256 digests are recorded for comparison. The Python programs use only the standard library. Rebuilding the article additionally requires a standard LaTeX installation.

The strongest literal deliverable is the 18-live machine. The smaller ten-live result is derived. The general event theorem includes exact ties. The direct Diophantine result concerns a step and every fixed finite horizon. No fixed-arity polynomial for arbitrary halting histories, no fastest loader, and no optimal signal-count claim is made.

[write] **The release in this report.** “The release” is the delivered archive. Its programs are shipped as `code/12-conservative-signal-*` and its records as `data/12-conservative-signal-*`, with the delivered sub-directories `code/`, `numeric/`, `correction/`, `receipts/` and `data/` flattened into these two directories, and its `CORRECTION.md`, `SOURCE-PROVENANCE.md` and `numeric/PROOF_AND_LEDGER.md` at the report root under the same prefix; the README of this report maps every name. The delivered article source and PDF and the delivered README are not shipped: this Part prints the article, Appendix F prints its Morita table, and this report’s README replaces the delivered one. The two compact closure records `numeric/MODES.jsonl.gz` (1,054,196 bytes) and `numeric/EVENTS.jsonl.gz` (580,458 bytes) are *not* shipped either: they are regenerated by the shipped compiler in about a minute, with identical decompressed content, and the README gives the command (“Reconstructing the excluded data”); the machine file, the ledger and the circuit file named in Appendix H are shipped, and the verbose branch and residual-row files were already omitted from the delivered archive, as that appendix says. The programs need the delivered layout and must be run on a copy (README).

[write] **The corrected archive and the repository’s review.** This Part prints the corrected edition of the archive (batch 79 manuscript 16). An earlier edition (manuscript 07 of the same batch) has the same article, data and receipts, and differs only in the generic packet evaluator `code/quadratic_packet.py`. The repository’s Hilbert-tenth-problem programme reviewed that earlier edition [29]: it passed the stated construction, independently enumerated the complete outgoing successor set of all 49,700 modes and all 80,501 branches (the least mode-only closure), checked that the trillion-term monomial count of Appendix H cannot be reduced by cancellation, and found genuine input-validation defects in the generic evaluator (a padded output (1,999), a floating endpoint and a surplus slack were accepted). It wrote the repair `conservative_signal_packet_domains.patch` [29]. The corrected edition applies exactly that patch and says so in its `CORRECTION.md` (shipped as `12-conservative-signal-CORRECTION.md`); the shipped evaluator `code/12-conservative-signal-quadratic_packet.py` is the patched one, and the correction receipts are shipped. The repair does not change Theorem 36.1, the literal construction or any ledger, and it does not make arbitrary caller-supplied overlapping branch families disjoint. The programme then reviewed the corrected edition itself [31]: it reread the article and the proof notes, compared the two archives member by member, passed all ten original author entryptoints and the three correction runs, rechecked one explicit integral event for every branch, and found no new defect on the valid domain. That review also proves a reduction that source 12 does not claim, a natural-domain guard projection: deleting the germ rows, the span row and the first-event rows of unselected edges with negative closing speed (18 rows and slacks per branch) is a bijection of complete *natural* zero sets, though not of nonnegative real ones, and reduces the literal one-step packet from 4,196,998 to 2,747,980 witnesses and from 2,762,961 to 1,313,943 squared residuals, with an explicit evaluation schedule 7,489,424 operations shorter. The projection is the review’s, is not re-proved here, and changes no theorem of this Part. Neither review is a proof-assistant verification, and neither finds a new universal operation bound.

Part IV

Sparse Diophantine certificates for finite mass lattice dynamics (source 13)

[write] Part IV prints batch-79 manuscript 19, *Sparse Diophantine certificates for finite mass lattice dynamics*, “Research report prepared with AI assistance for Vladimir”, dated 2 October 2026, from the archive `Sparse_Lattice_Diophantine_Certificates.zip`. It is called source 13 after the prefix of its shipped files. The delivered article is a main file that reads five further files; they are printed here in the delivered order, without its separate table of contents. Its labels carry the prefix `smc:sl:`; its sections are Sections 38–48 here, and it has no appendices. Its statements keep their numbering by section; equations are numbered by section (the source numbered them consecutively). Source 13 names no repository commit and cites no repository file. Its comparison and sorting gadgets and its post-elimination compiler re-derive devices of *canonical-diophantine-certificates*; [write] notes at those places say so, and Section 1.7 describes the Part. Its abstract follows.

Finite conserved mass bounds the number of occupied unit records without bounding the distances between them. We exploit this distinction to compile a fixed finite horizon of one-dimensional partitioned lattice dynamics into one quartic sum of squares with a unique natural witness. The construction explicitly pays for collision detection, simultaneous redistribution, sorting, finite table lookup, and empty space. We give exact witness and residual ledgers, a coefficient-level reproducible implementation, and an optional strengthening whose nonnegative-real fiber is also exact. A source-critical interface to the reversible conservative construction of Morita and Imai includes a necessary sign correction and a reachable-legality hypothesis; a fully specified doubling example is separated from the existence of a fixed safe universal source. Complementary proofs show decidability under bounded orbit diameter and, without a diameter promise, at total mass at most two. At mass at most two, a stronger effective semilinearity argument yields horizon-free canonical quartics of fixed rule-dependent arity. No fixed-arity representation of unbounded universal computation, new classical universality result, or minimum universal mass is claimed.

38 Results and model

The geometric resource in this report is an unbounded integer gap, rather than an unbounded number of occupied cells. A conserved positive integer mass M permits a canonical description by M repeated unit records, even when the occupied positions are very far apart. The goal is to verify every lattice microstep directly on those records, including collisions and the assertion that no output has been omitted in empty space.

Three levels of claim must be kept separate. First, the main certificate theorem applies to *any supplied total conservative finite table*. Second, the corrected Morita–Imai compiler provides explicit reversible examples and an interface from suitably legal two-counter computations. Third, a fixed safe universal source exists by classical simulation constructions, but the release does not print a particular universal source machine and its numerical state count. Its fully printed worked machine computes doubling and is not universal.

The sparse construction and its exact ledgers are the arithmetic contribution considered here. The circuit-to-sum-of-squares device and classical reversible computational universality are not new claims. Dense fixed-box or ring tableaux already record bounded cellular computations; a direct sparse infinite-line argument additionally has to justify grouping, ordering, and empty space.

Priority of the particular constructions and the mass-two theorem below has not been established by an exhaustive literature review.

[write] The dense fixed-box tableau is, for example, the finite cellular-automaton compiler of Part VIII of *canonical-diophantine-certificates* (its Section `cdc:mem:sec:substrates`), whose one-hot tableau occupies a fixed box; this Part is the sparse infinite-line counterpart for one-dimensional partitioned conservative dynamics. Source 13 was written without this report and without that one.

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Fix $K \geq 1$, put $q = K + 1$, and supply every row of a function

$$g : \{0, \dots, K\}^3 \longrightarrow \{0, \dots, K\}^3, \quad r + c + l = g_L(r, c, l) + g_C(r, c, l) + g_R(r, c, l).$$

A configuration c gives a triple $(L_c(x), C_c(x), R_c(x))$ at each $x \in \mathbb{Z}$. One update is

$$F(c)(x) = g(R_c(x - 1), C_c(x), L_c(x + 1)). \quad (38.1)$$

Thus the *input* order of the table is (R, C, L) and its *output* order is (L, C, R) . In physical terms the right channel streams right, the left channel streams left, and the center stays in place; g then redistributes the incoming mass. The input-output order matters even when the numerical table happens to be the identity.

Conservation forces $g(0, 0, 0) = (0, 0, 0)$. Finite support stays finite, and the sum of all channel values remains M . If g is a permutation, then F is globally bijective: apply g^{-1} at each site and reverse the two lane shifts. Both operations preserve the zero background and finite support. Reversibility is useful for the intended substrate, but is unnecessary for the forward certificate theorem.

38.1 Canonical records and fixed parameters

Let the valid initial configuration be nonempty, with minimum occupied position o and diameter D . Replace each channel mass by that many copies of a record (x, z) , where $z = 0, 1, 2$ means left, center, right. Sort lexicographically, and shift the physical position by $T - o$, where $T \geq 0$ is the externally fixed horizon. Thus a record at physical site p has

$$x = p - o + T.$$

The initial row has exactly M records. At time $t \leq T$, every position lies between $T - t$ and $D + T + t$. Repeated records are allowed and have no individual labels. The sorted repeated-unit row therefore encodes the physical configuration uniquely. There is neither zero padding nor a hidden list of empty sites.

The initial data, its translation, the local table, and the horizon are compiler parameters, not quantified witnesses. Later we explicitly pay for a loader when two initial counter positions are allowed to remain free natural parameters. The vacuum $M = 0$ is a separate trivial constant orbit.

Theorem 38.1 (Sparse finite-horizon certificate). *For a supplied total conservative g , valid nonempty initial configuration of mass M , and fixed horizon T , there is an explicitly generated integer polynomial*

$$P(\mathbf{z}) = \sum_{j=1}^R \rho_j(\mathbf{z})^2, \quad \deg \rho_j \leq 2,$$

with exactly one zero in $\mathbb{N}^V$. Its row decoder gives precisely $c, F(c), \dots, F^T(c)$. The preferred full-table construction uses exactly

$$V = T[M(q^3 + 14) + 5M(M - 1)], \quad (38.2)$$

$$R = T[17M + 5M(M - 1)]. \quad (38.3)$$

Every witness in this unique solution is at most

$$H = 3(D + 2T) + M + 3K + 3. \quad (38.4)$$

Consequently the tuple has at most $V[\log_2(H + 1)]$ bits under fixed-width coordinate encoding. A prescribed endpoint of mass M can be imposed with $2M$ additional affine residuals and no witnesses, leaving zero or one natural solution.

For $T = 0$ the witness tuple is empty, and is the unique element of $\mathbb{N}^0$. The empty sum is the zero constant. A mismatched-mass endpoint can instead be rejected by adding the constant residual 1. The degree bound is at most four, not a claim that every small instance has degree exactly four. Dimension grows with T and M ; the theorem is not a fixed-arity equation with an unknown unbounded horizon.

39 The canonical arithmetic layer

39.1 Comparison and equality without ambiguous ties

For already established nonnegative integral expressions x, y , define $\text{CMP}(x, y)$ by fresh natural witnesses b, d and residuals

$$b(b - 1), \quad y - x - (2b - 1)d - b + 1. \quad (39.1)$$

If $b = 1$, the second equation says $d = y - x$, possible exactly when $y \geq x$. If $b = 0$, it says $d = x - y - 1$, possible exactly when $x > y$. This proves unique solvability, including ties, with

$$b = \mathbf{1}_{\{y \geq x\}}, \quad d = \begin{cases} y - x, & y \geq x, \\ x - y - 1, & x > y. \end{cases}$$

The inputs can be affine expressions; there is no charge for an unnecessary alias wire. To test equality, use two comparisons, $\text{CMP}(x, y)$ and $\text{CMP}(x + 1, y)$, and take the affine difference of their flags:

$$e(x, y) = b_{\text{weak}} - b_{\text{strict}} = \mathbf{1}_{\{x=y\}}. \quad (39.2)$$

This costs four witnesses and four residuals, with no additional equality-variable witness.

[write] The comparison gadget (39.1), with the same two residuals and the same case analysis, is Lemma `cdc:mem:lem:compare` of Part V of *canonical-diophantine-certificates*. Source 13 does not cite it; the gadget is printed here as delivered and no novelty is claimed for it.

39.2 Streaming and all simultaneous collision groups

Suppose the current row is (x_i, z_i) , $0 \leq i < M$, in sorted order. Each x_i, z_i is affine in earlier variables. Introduce natural L_i, C_i, R_i with residuals

$$L_i + C_i + R_i - 1, \quad z_i - C_i - 2R_i. \quad (39.3)$$

For valid integral $z_i \in \{0, 1, 2\}$, these force the unique one-hot channel vector. A streamed position y_i is forced by

$$y_i + L_i - x_i - R_i = 0. \quad (39.4)$$

It is nonnegative at every layer $t < T$, because $x_i \geq T - t \geq 1$. This is genuine line transport; there is no clipping at zero and no wraparound.

For each unordered pair $i < j$, compute $e_{ij} = \text{EQ}(y_i, y_j)$; set $e_{ji} = e_{ij}$ and $e_{ii} = 1$ as aliases and constants. The whole collision partition is determined at once. Introduce three incoming mass variables with residuals

$$u_i - \sum_j e_{ij} R_j, \quad v_i - \sum_j e_{ij} C_j, \quad w_i - \sum_j e_{ij} L_j. \quad (39.5)$$

Their order is (R, C, L) , as required by (38.1). Equal destinations have identical incoming triples. Each component is at most K : a particular incoming lane is a translated copy of one old lane at one site. That statement uses validity of the previous row, not a guessed bound on the number of simultaneous records.

39.3 A single table-row selector simplex

Index every input table row by $\tau = (r, c, l) \in \{0, \dots, K\}^3$. For particle i , introduce q^3 natural selectors $s_{i,\tau}$, three output masses a_i, b_i, c_i , and the seven affine residuals

$$\begin{aligned} & \sum_{\tau} s_{i,\tau} - 1, \\ & u_i - \sum_{\tau} r s_{i,\tau}, \quad v_i - \sum_{\tau} c s_{i,\tau}, \quad w_i - \sum_{\tau} l s_{i,\tau}, \\ & a_i - \sum_{\tau} g_L(\tau) s_{i,\tau}, \quad b_i - \sum_{\tau} g_C(\tau) s_{i,\tau}, \quad c_i - \sum_{\tau} g_R(\tau) s_{i,\tau}. \end{aligned} \quad (39.6)$$

A natural vector summing to one has exactly one nonzero entry, equal to one. The three input moment equations select precisely the row (u_i, v_i, w_i) , because all input triples are distinct and the table is total. The output equations then force $g(u_i, v_i, w_i)$. This single simplex avoids factoring the lookup into three separate selector vectors and all their products. Its small residual count does not make the q^3 selectors or the long coefficient sums free.

39.4 Rank redistribution

Within each destination group use the old record order to define its one-based rank

$$r_i = \sum_{j \leq i} e_{ij}.$$

This is an affine expression, not a fresh witness. Compute two flags by comparisons

$$h_i = \text{CMP}(r_i, a_i), \quad k_i = \text{CMP}(r_i, a_i + b_i),$$

where the notation denotes the returned flag. The next channel is the affine expression

$$z'_i = 2 - h_i - k_i. \quad (39.7)$$

Ranks within a group are exactly $1, \dots, u_i + v_i + w_i$. Conservation makes this number $a_i + b_i + c_i$. The first a_i ranks have $(h_i, k_i) = (1, 1)$ and channel zero; the next b_i have $(0, 1)$ and channel one; the last c_i have $(0, 0)$ and channel two. Thus the output multiset at the collision site is exact. The intermediate rank convention only chooses identical physical units, and no persistent identity survives.

39.5 A fixed sorting network

Sort the M records (y_i, z'_i) by a fixed bubble sorting network of $\binom{M}{2}$ adjacent compare-exchanges. For inputs $(x, z), (y, w)$ compare the keys $3x + z$ and $3y + w$ using (39.1), giving b, d . Introduce four output witnesses and residuals

$$\begin{aligned} x_{\min} &= y - b(x - y), & z_{\min} &= w - b(z - w), \\ x_{\max} &= x - y + x_{\min}, & z_{\max} &= z - w + z_{\min}. \end{aligned} \quad (39.8)$$

For $b = 1$ the left input is the smaller or tied key; for $b = 0$ the right is smaller. Ties retain the left input, but equal keys are already the same physical record. Since channels lie in $\{0, 1, 2\}$, key order is lexicographic order. Every comparator costs six witnesses and six residuals, including its comparison. The network outputs are the next row by alias, without copy variables. All equations in this layer are of degree at most two.

[write] This compare-exchange is the two-field case of the record comparator of the same Part V (its equations `cdc:mem:eq:record-low` and `cdc:mem:eq:record-high`, with the cost count of Proposition `cdc:mem:prop:comparator-cost`); it is printed as delivered, as a re-derivation of that device.

40 Correctness and exact paid costs

Proof of Theorem 38.1. Assume inductively that the current row is the canonical true row and every earlier auxiliary witness is uniquely fixed. The channel equations and comparison lemma uniquely fix streaming and the entire equality partition. Equations (39.5) fix the incoming mass. The single natural simplex forces the unique table row and its output. Conservation and rank redistribution then give exactly the outgoing multiset. The sorting network uniquely produces the canonical next row.

The empty-space assertion is indispensable. Every nonzero incoming site appears among the y_i , since it has at least one streamed unit. A site absent from that list has incoming triple $(0, 0, 0)$ and hence zero output. A listed site has positive incoming total, which conservation keeps positive. Therefore there is no hidden output outside the represented sites, no omitted occupied site, and no ghost site introduced by an unconstrained witness.

This proves soundness and uniqueness by induction through T layers. Conversely, start with the actual physical history, expand each configuration into repeated unit records, and evaluate the stated one-hot vectors, comparisons, masses, ranks, and sorting gates. They provide nonnegative integral values satisfying every residual. Thus the unique natural solution exists. Counting records of channel z at position x and undoing the shift by $T - o$ is the decoder. The deterministic gate evaluation is its inverse on the fixed-input, fixed-horizon solution set.

If g is reversible, the final physical configuration also determines all preceding physical configurations by F^{-1} . This does not assert reversibility of the auxiliary arithmetic circuit, nor any property of arbitrary real solutions.

For the height bound, positions stay in $[0, D + 2T]$. Sorting keys are at most $3(D + 2T) + 2$; comparison slacks are bounded by the appropriate key difference, group rank, or local mass. Incoming and outgoing lane masses are at most K , ranks at most M , and selectors at most one. Bound (38.4) covers all these quantities with slack. Endpoint equations only select or reject the already unique row. $\square$

40.1 Scalar ledger

The per-layer cost is as follows. The lookup row excludes the separately listed incoming masses.

Block	Witnesses	Residuals
Channel decoding	$3M$	$2M$
Streaming	M	M
Pairwise destination equality	$4\binom{M}{2}$	$4\binom{M}{2}$
Incoming lane masses	$3M$	$3M$
Full row-selector lookup	$M(q^3 + 3)$	$7M$
Rank comparisons	$4M$	$4M$
Sorting network	$6\binom{M}{2}$	$6\binom{M}{2}$
Total	$M(q^3 + 14) + 5M(M - 1)$	$17M + 5M(M - 1)$

The exact counts include slacks, Boolean comparison flags, sorting outputs, and repeated simultaneous-collision work. Constants and affine aliases are not witnesses. The local table itself has q^3 input rows and three output entries per row; its construction, range/conservation checks, storage, and any permutation test are separate explicit costs. Expanding weighted input into M unit records requires $\Omega(M)$ output work and storage. The size bounds are polynomial in the numerical mass M , not in its binary bit length. For a fixed source substrate, $M = m + 19$ is constant while gaps and their bit lengths may grow.

Let $\mathcal{L}$ be the total number of nonzero monomial terms in the residual list, and $\mathcal{O}$ the sum of the degrees of those monomials, including repeated variable occurrences. The release declares the following literal evaluator: initialize a monomial with its signed integer coefficient, multiply by each listed variable, add its value into a residual accumulator initially zero, square each residual, and add those squares starting at zero. Under exactly this convention,

$$\#\text{multiplications} = \mathcal{O} + R, \quad \#\text{additions} = \mathcal{L} + R. \quad (40.1)$$

Multiplication by a coefficient is already included by the initialization convention; negation signs reside in signed coefficients. Even a final zero residual is squared by this evaluator. The exporter supplies these exact counts and the largest residual-coefficient bit length for each instance.

There are $O(T[M^2 + Mq^3])$ such arithmetic operations. This is scalar arithmetic, not bit complexity. Witness height, coefficient size, polynomial-description length, arithmetic cost, and witness dimension are different resources. The statement concerns a coefficient-explicit residual-list sum of squares. Fully expanding and collecting the quartic polynomial can create many more terms; no equality with the residual-list cost is asserted.

40.2 Restricted row lists

A useful optional backend replaces all q^3 rows in (39.6) by a supplied list of S distinct allowed input rows, each conserving mass. It has

$$V_S = T[M(S + 14) + 5M(M - 1)], \quad R_S = R.$$

The same proof certifies exactly histories whose every occupied incoming triple belongs to that list. If some required row is absent, its input moments cannot be matched by a natural one-hot selector, and the certificate is unsatisfiable. This backend does *not* certify all histories of an arbitrary total

extension merely because the listed rows extend to one. Coverage of the particular history or a valid-trajectory invariant is a separate proof obligation. The worked source example uses such a coverage theorem; the generic theorem always uses the supplied full table.

40.3 The domain of uniqueness and an orthant-exact option

The default simplex equations rely on natural integrality. They need not be unique even over the nonnegative real orthant. For a concrete full-table example let $K = 2$, $M = 1$, $T = 1$, and $g(r, c, l) = (r, c, l)$ numerically, with the stated input/output orders. Start with one right-channel unit. Its incoming triple is $(1, 0, 0)$ and its output is one left unit. Replace the table selector at $(1, 0, 0)$ by half at $(0, 0, 0)$ and half at $(2, 0, 0)$. All three input moments and all output moments remain unchanged. Every other witness can keep its canonical value, so the original residuals still vanish, but the selector is fractional. The release checks this with exact rational arithmetic.

For any nonnegative simplex $s_1 + \dots + s_d = 1$, add the one residual

$$\sum_{j=1}^d s_j^2 - 1. \quad (40.2)$$

If it vanishes, then

$$0 = \left(\sum_j s_j\right)^2 - \sum_j s_j^2 = 2 \sum_{i < j} s_i s_j.$$

Every summand is nonnegative; at most one selector is nonzero, and that selector equals one. Adding (40.2) to each channel simplex and each table simplex therefore restores the inductive proof over $\mathbb{R}_{\geq 0}$ for fixed natural input data. Comparisons have Boolean flags already, and their nonnegative slacks are uniquely forced once the inputs are fixed integers. All subsequent witnesses remain the canonical natural values.

Corollary 40.1 (Exact nonnegative-real fiber). *The strengthened full-table or restricted-row compiler has the same witness count and exactly $2MT$ extra residuals. Its nonnegative-real solution set is the same singleton or empty set as its natural solution set. Residual degree stays at most two and polynomial degree at most four.*

This says nothing about the unrestricted real domain, where negative selectors invalidate the nonnegativity argument. An earlier factorized lookup remains an independently tested baseline: three q -selectors, q^2 pair products, and q^3 triple products give

$$V_{\text{fact}} = T[M(q^3 + q^2 + 3q + 14) + 5M(M - 1)],$$

$$R_{\text{fact}} = T[M(q^3 + q^2 + 19) + 5M(M - 1)].$$

Its orthant strengthening costs $4MT$ residuals. These are baseline counts, not the preferred single-row-simplex ledger in (38.2)–(38.3).

41 The reversible conservative source interface

Morita and Imai [19] construct a reversible number-conserving partitioned automaton from a reversible two-counter machine. The following source audit fixes the exact contract used here. It distinguishes the published theorem, a correction forced by its encoding, and an additional reachable-legality argument needed when unspecified local rows are completed arbitrarily.

41.1 Source syntax and encoding

A transition is (i, j, op, k) , with source state q_i , target q_k , counter $j \in \{0, 1\}$, and operation increment, decrement, no-op, zero test, or positive test. There are $m \geq 2$ states, q_0 has no incoming transition, and q_{m-1} is designated final with no outgoing transition. Distinct transitions with the same source may only be complementary zero/positive tests on the same counter; the same restriction applies to distinct transitions with the same target. This is the paper's syntactic deterministic and reversible model, a stronger specification than a vague assertion of injectivity for some other instruction language.

Put

$$a_i = i + 10, \quad B = m + 16, \quad b_i = B - a_i - \gamma_i,$$

where $\gamma_i = 2, 4, 6, 7$ for increment counter zero, decrement counter zero, increment counter one, decrement counter one respectively, and zero otherwise. The alphabet cap is $K = m + 18$ and the number of ordinary cell states is $(m + 19)^3$.

The encoding $\phi(q_i, n_0, n_1)$ places (a_i, b_i, γ_i) at the origin. Counter j contributes marker mass 2^j at position n_j , normally in the center lane. If $n_j = 0$ and q_i increments that counter, its marker is instead added to the right lane at the origin. Contributions at the same site add. Hence

$$M = B + 1 + 2 = m + 19 = q, \quad (41.1)$$

independently of counter magnitude. The support occupies at most three sites, all in $\{0, n_0, n_1\}$. Coincident markers have total three, retaining their separate zero/positive information through distinct weights. Input size is paid through distances and their bit lengths, not a variable number of particles.

41.2 The printed sign defect and the repaired local injection

On printed page 249, rule (8.2) has an output-center $+\gamma_k$. When a no-op enters a state incrementing counter h , the input and repaired output are

$$\begin{aligned} & (a_i, B - a_i + c2^h + c'2^{1-h}, 0) \\ & \longmapsto (a_k, B - a_k - \gamma_k + c'2^{1-h}, \gamma_k + c2^h), \quad c, c' \in \{0, 1\}. \end{aligned} \quad (41.2)$$

The printed plus would increase the local total by $2\gamma_k$, namely four or twelve. The minus conserves mass and gives exactly the declared target encoding. This is a necessary correction to that printed formula; no author-issued erratum is asserted. The target's increment counter h must also be read from q_k , rather than borrowed from the no-op quadruple's semantically irrelevant counter tag. Opposite-tag cases are included in the executable audit.

[write] The repository's Hilbert-tenth-problem programme checked this sign against the primary paper (printed page 249 of [19]) in its review of source 13 [30]: subtracting the target opcode is forced by mass conservation and the target encoding, and the target's increment counter, not the no-op tag, controls the marker routing. Like source 13, the review records a mathematical correction to a published formula, not an author erratum. No report or README of the repository made a claim that this correction refutes.

For clarity, the structural proof behind finite row checks is as follows. The ordinary signal rules route signal mass or transfer a marker between center and signal; waiting rows are identities. After (41.2), state-update rows preserve B plus the zero-marker mask. Startup and final rows

exchange a unit signal plus a reservoir $m + 15$ with control mass B . For nonfinal i , the smallest baseline obeys

$$b_i \geq m + 6 - (m - 2) - 7 = 1;$$

the final baseline is seven. Added marker mass is at most three, all center values are at most $m + 18$, and state signals are at most $m + 9$. Thus all coordinates stay in range.

Routine signal domains differ from control domains by their nonzero lane pattern and by signal values below or above ten. At a fixed control state its unique operation type chooses the rule family. Zero masks are distinct; complementary test branches have disjoint masks. The no-outgoing convention at the final state removes domain conflicts there. This proves the function property.

For injection, different target state signals have different left components. At a fixed target, its center/right pair recovers the zero-mask bits: an increment target carries its own zero marker in the right signal and leaves the other bit in the center; otherwise both bits remain centered. A target with a movement or no-op predecessor has only that incoming transition. If it has two incoming tests, they test the same counter with complementary outcomes and hence disjoint masks. The *whole output triple* distinguishes them, even when the center alone does not. Waiting outputs have right component zero, whereas a transition into an operation state has a positive right signal. Startup targets q_0 , which has no incoming source transition, and the final unit signal with its large reservoir is distinct from an ordinary travelling unit signal. The remaining routine signal output patterns are disjoint by lane and signal type. Together these observations establish a mass-preserving partial injection.

Lemma 41.1 (Completion within mass fibers). *Every finite mass-preserving partial injection on the local triple alphabet extends effectively to a mass-preserving permutation. This applies to the repaired table.*

Proof. For each s , let Ω_s be the triples with coordinate sum s . Used inputs and used outputs have equal cardinality within Ω_s . Bijection of their unused complements, for example by pairing lexicographically sorted lists, completes this fiber. Their union is the required permutation. The unique weight-zero triple is fixed. Lane transport followed by the local permutation is a global reversible conservative CA. $\square$

Simply assigning identity to every unspecified input is generally wrong. The specified row $(2, 1, 0) \mapsto (0, 0, 3)$ can leave $(0, 0, 3)$ unspecified as an input. Its identity image would duplicate an already used output. Also, Lemma 41.1 is about a partitioned/block local permutation; it is not a completion theorem for arbitrary partial ordinary-neighborhood CA rules.

41.3 Exact source microtimes and the pulse observer

For a *legal* source step, the repaired specified rows, without using any completion row, carry $\phi(I)$ to $\phi(J)$ in the following number of microsteps:

Source operation on a counter of value n	Microsteps
Increment, including $n = 0$	$2n + 2$
Decrement, with $n > 0$	$2n$
No-op or legal zero/positive test	2

Here is the geometric reason for the clocks. The state signal travels from the origin to site -1 and back, while the control waits for its operation signal. An increment signal traverses the distance to its marker, carries that marker one step farther right, deposits it, and returns to the origin, a total of $2n + 2$ steps. When $n = 0$, the marker is already in the initial right signal, giving the same two-step formula. A decrement signal reaches its positive-position marker, reflects while carrying it one step left, deposits it, and returns in $2n$ steps. At $n = 1$ the returning marker joins the origin's zero mask directly. The other counter's marker is transparent to these routes, including coincident marker positions, because the rule families carry its distinct mass unchanged. Tests and no-ops only require the state signal's two-step return. The state-update rows then redistribute the zero mask into exactly the target encoding. These routes also explain why no constant slowdown is promised.

Let t_h be the sum of these times up to first entry into $q_f = q_{m-1}$. From $\phi(q_f, n_0, n_1)$ the state signal is in the right lane at -1 at $t_h + 1$; the final rule creates a left unit at the origin at $t_h + 2$; and

$$L_{t_h+3}(-1) = 1. \quad (41.3)$$

That unit subsequently moves left forever. Before final entry, the specified simulation trajectory never has this left-unit signal. After its departure, the fixed centers are

$$C(0) = m + 15 + \mathbf{1}_{\{n_0=0\}} + 2\mathbf{1}_{\{n_1=0\}}, \quad C(x) = \mathbf{1}_{\{x=n_0\}} + 2\mathbf{1}_{\{x=n_1\}} \quad (x > 0).$$

All right channels are zero. The results remain readable while information escapes spatially. This is an anchored pulse, not an absorbing globally halted configuration. The complete successful orbit has unbounded diameter.

The equivalence between this observer and source halting requires that *every reachable nonfinal source configuration on the input under discussion has a legal next instruction*. This is called reachable legality below. It is not asserted for every arbitrary counter pair at every internal state.

41.4 An explicit false observer on an unsafe source

Take $m = 2$ and the sole source instruction “decrement counter zero from q_0 to q_1 .” On counters $(0, 0)$ that instruction is illegal, so the designated final state is not reached. Its encoded initial CA state is $(10, 7, 4)$ at site zero, of mass 21. At time one the nonzero sites are

$$-1 : (0, 0, 10), \quad 0 : (0, 7, 0), \quad 1 : (0, 0, 4).$$

The next origin input $(10, 7, 0)$ is unspecified. Both this input and the output $(1, 16, 0)$ are unused, and both have mass 17, so adding

$$(10, 7, 0) \longmapsto (1, 16, 0)$$

is compatible with Lemma 41.1. At time two the configuration is $0 : (1, 16, 0)$ and $2 : (0, 0, 4)$; at time three it has a left unit at -1 , falsely satisfying the proposed observer. The release supplies the entire resulting 9,261-row permutation and a replay through time eight.

Thus an arbitrary partial reversible source and an arbitrary conservative permutation completion do not provide a halting equivalence on every encoded pair. Missing test branches and unnamed dead states have the same logical risk. This example does not affect the generic theorem, which always certifies the actual supplied CA dynamics, and does not refute a safe universal-source construction.

42 Safe universal sources and the paid loader

42.1 The existence argument and its exact scope

Morita's 1996 theorems [20] give designated-final-state simulation equivalences. The following preservation of reachable legality is an explicit invariant argument through those constructions, rather than an attribution of a separate totality theorem to that paper.

Start with a fixed universal deterministic Turing machine whose transition is defined at every nonhalting state/symbol pair. On its computably valid program/input encodings, choose a complete guarded counter simulation: decrement is entered only after positivity, every appropriate test branch exists, and each arithmetic routine terminates on its finite inputs. A fresh initial state and a terminal designated final state give the required endpoint conventions. This yields a safe irreversible counter simulator. Splitting high-indegree target states through finite no-op merge chains, as in Morita's preparatory reduction, preserves legal progress: one legal step only acquires a finite chain of unconditional steps.

The reversible history compiler adds a history counter H and scratch counter W . At an original-state boundary the invariant is

$$(\text{simulated counters correct}), \quad H \in \mathbb{N}, \quad W = 0.$$

An already reversible incoming relation is copied. For an overlapping incoming pair, the compiler takes the original legal operation and then records which predecessor was used. It transfers H to W through positivity-guarded decrements and reconstructs $2H$ or $2H + 1$ in the history counter while emptying W . Each decrement is legal by its positive guard; required positive back-edge tests follow increments; each transfer loop terminates by decreasing its source counter. The boundary invariant is restored at the target. Thus a legal step expands into a finite nonempty legal path, without a new nonfinal dead end. Erasing history is unnecessary for this existence argument.

The reversible multi-counter machine is then represented by two counters

$$(N, S) = \left(\prod_{i=1}^k p_i^{v_i}, 0 \right), \quad N \geq 1, \quad (42.1)$$

where the p_i are distinct fixed primes. This is a constrained encoding of all data, history, and scratch counters, not an arbitrary pair in $\mathbb{N}^2$. The prime compiler has the following legal-progress invariants.

- An increment of counter i transfers N to scratch through guarded decrements and reconstructs $p_i N$ while consuming scratch. Both loops terminate and end with scratch zero
- A legal decrement has $v_i > 0$, so $p_i \mid N$. After transfer, each positive scratch batch has at least p_i units. Its p_i successive decrements are legal even where only the first is explicitly guarded. Consuming all batches gives $(N/p_i, 0)$
- A test computes quotient and residue by a guarded finite-state division loop. Divisibility by p_i is exactly $v_i > 0$; nonzero residue is exactly $v_i = 0$. The appropriate source branch is selected, and quotient and residue restore the original N through terminating positively guarded loops
- A no-op is copied. If a source has only one test exit, the omitted exit is never needed on its legal encoded trajectory

The original state, prime encoding, and scratch-zero invariant hold again at each boundary. Fresh internal states preserve the source's required reversible incoming syntax and initial/final conventions. Induction proves that every reachable nonfinal primitive two-counter state on a valid input has a successor. If the original machine never halts, each of its steps expands into a nonempty finite legal path, so no stuck intermediate configuration can masquerade as a halt.

Fixing the universal program fixes the resulting finite two-counter source and hence, by the corrected local construction and a chosen completion, one finite reversible conservative CA. Its valid program/data encodings vary only through the initial counter positions. Eventual anchored pulse occurrence is therefore computably enumerable hard on this computable valid-input family; direct simulation makes it computably enumerable. This supplies the usual undecidable local-observation problem for a fixed finite-mass substrate.

The release does not materialize a chosen universal source table or give its numerical m . It does not claim faithful source interpretation for all $(n_0, n_1) \in \mathbb{N}^2$, intrinsic CA universality, or an efficient simulation. In particular, the often cited 96-state construction using an ultimately periodic infinite background is not a finite-mass substitute. Exact-target reachability is also a different question: the final counter positions need not be computable in advance, and the escaping halt signal is not a specified complete target. Transferring exact-target undecidability would require an additional normal form or compute/uncompute argument.

42.2 A uniform initial loader with no hidden input oracle

For fixed source data m , initial opcode γ_0 , and fixed T , leave $n_0, n_1 \in \mathbb{N}$ as free polynomial parameters. At position T form ten left records, $m + 6 - \gamma_0$ center records, and γ_0 right records. Add one counter-zero record at $n_0 + T$ and two counter-one records at $n_1 + T$. A marker's channel is right precisely when its position parameter is zero and q_0 increments that counter; otherwise it is center.

Two comparisons $\text{CMP}(n_j, 0)$ determine the zero flags, costing four witnesses and four residuals. The initial list is sorted by the same network as before. Its exact loader cost is

$$V_{\text{load}} = R_{\text{load}} = 4 + 3M(M - 1). \quad (42.2)$$

Even unused zero tests are included in this declared ledger. The exported residual coefficients are independent of the supplied values of n_0, n_1 ; only those parameter values and the canonical witnesses vary. Bound (38.4) remains valid with $D = \max(n_0, n_1)$, including loader comparison slacks.

42.3 Paying for the exact first pulse

For $T \geq 1$, site -1 is shifted to $T - 1$. For each $t = 1, \dots, T$ and each record, compute $\text{EQ}(x_{t,i}, T - 1)$. Reuse the channel one-hot vectors already present at $t < T$, and add $3M$ channel witnesses with $2M$ residuals for the final row. Add one residual at each time,

$$\sum_i e(x_{t,i}, T - 1) L_{t,i} - \mathbf{1}_{\{t=T\}}. \quad (42.3)$$

This forces detector left mass zero at every earlier observed time and one at T . The initial detector is blank because ϕ is supported on nonnegative sites. The additional cost is exactly

$$V_{\text{obs}} = 4MT + 3M, \quad (42.4)$$

$$R_{\text{obs}} = 4MT + 2M + T. \quad (42.5)$$

For the orthant-exact variant add M final decoder norm residuals, on top of the core $2MT$ extra residuals. No observation witnesses are needed for a fixed endpoint beyond the core row witnesses, but that is a different predicate.

These guards mean *zero before and one now*. For an arbitrary CA they are not the generic first occurrence of the value one, because earlier detector masses greater than one are also excluded. On valid source simulations they match the exact unique pulse in (41.3). Combining (38.2), (38.3), (42.2), (42.4), and (42.5) gives the full paid uniform-input first-pulse certificate. Safe-source legality concerns the semantic interpretation of that predicate, not the arithmetic soundness of the actual-CA certificate.

42.4 The fully specified doubling example

The worked source has seven states and the following complete transition list; all other source transitions are absent:

Source	Counter	Operation	Target
q_0	1	zero test	q_1
q_1	0	zero test	q_6
q_1	0	positive test	q_2
q_2	0	decrement	q_3
q_3	1	increment	q_4
q_4	1	increment	q_5
q_5	1	positive test	q_1

On $(q_0, n, 0)$ every reachable nonfinal instruction is legal: the first test sees zero, each decrement follows a positive counter-zero test, and the last positive test follows two increments. The machine reaches $(q_6, 0, 2n)$ after $5n + 2$ source steps. Summing the exact microclocks gives

$$t_h = 5n^2 + 7n + 4, \quad t_{\text{pulse}} = 5n^2 + 7n + 7. \quad (42.6)$$

Indeed iteration $k = 0, \dots, n - 1$ takes a positive test, a decrement of $n - k$, increments at $2k$ and $2k + 1$, and a final test, totalling $2n + 6k + 10$ microsteps. The initial and final zero tests add four; summing yields (42.6).

Here $K = 25$, $M = q = 26$. The corrected specified local table has 97 distinct rows, and its lexicographic mass-fiber completion has all $26^3 = 17,576$ rows. For $n = 2$, twelve source steps reach $\phi(q_6, 0, 4)$ at time 38, create a left unit at the origin at time 40, and produce the detector pulse at time 41. The full table and trajectory checks are shipped. This explicit example verifies the interface and its costs without representing a universal-machine claim.

43 Geometric barriers and translating periodicity

The next results use a more general model than the triple-channel compiler. Let Q be a finite alphabet, 0 a quiescent state, and $F : Q^{\mathbb{Z}} \rightarrow Q^{\mathbb{Z}}$ a deterministic translation-invariant CA of radius r . Let

$$w : Q \rightarrow \mathbb{N}, \quad w(0) = 0, \quad w(a) \geq 1 \quad (a \neq 0),$$

conserve total weight on every finite-support configuration. Thus the vacuum is the unique zero-weight symbol. A mass- M configuration has at most M occupied sites. Distinct states may have the same positive weight; this matters when comparing particle bounds in numerical-state models.

Zero-weight active symbols, signed weights allowing cancellation, and a nonzero infinite periodic background are outside these hypotheses. Quiescence and locality preserve finite support at each finite time, but do not bound its position or its diameter uniformly in time. A finite-support configuration on $\mathbb{Z}$ is not a finite-ring configuration.

Write $(\mathcal{T}_a x)(i) = x(i - a)$ for translation to the right. For nonempty x , let $\ell(x) = \min \text{supp}(x)$ and $\mathcal{N}(x) = \mathcal{T}_{-\ell(x)} x$ be its normalized shape. The vacuum is handled separately.

Proposition 43.1 (Deciding a diameter bound and its tail). *Given F , a finite configuration x , and an integer $D \geq 0$, one can decide whether $\text{diam}(F^t x) \leq D$ for every $t \geq 0$. Under this full-orbit bound, exact finite-configuration reachability and finite-word occurrence, anchored or at any translate, are decidable. Without the bound promise, the same procedure decides occurrence before the first escape and otherwise reports the escape, without concluding anything about later unrestricted occurrence.*

Proof. There are finitely many normalized nonempty shapes of diameter at most D , at most $(|Q| - 1)|Q|^D$. With mass M a sharper bound is

$$\sum_{k=1}^{\min(M, D+1)} \binom{D}{k-1} (|Q| - 1)^k. \quad (43.1)$$

Position zero is occupied, the remaining $k - 1$ positions are chosen in $\{1, \dots, D\}$, and all occupied symbols are nonzero. An exact-weight count is the coefficient of z^M in

$$\left(\sum_{a \neq 0} z^{w(a)} \right) \left(\sum_{a \in Q} z^{w(a)} \right)^D.$$

Simulate until the diameter exceeds D or a normalized shape repeats. One event must happen in finitely many steps. If

$$\mathcal{N}(F^i x) = \mathcal{N}(F^j x), \quad i < j, \quad p = j - i, \quad \delta = \ell(F^j x) - \ell(F^i x),$$

then determinism and translation equivariance give, for every $0 \leq s < p$ and $k \geq 0$,

$$F^{i+s+kp} x = \mathcal{T}_{k\delta} F^{i+s} x. \quad (43.2)$$

Consequently no future shape escapes. An escape disproves the diameter bound, while a repeat proves it.

For exact target y , inspect the finite prefix and then each phase $z = F^{i+s} x$. It can reach y exactly when its normalized shape agrees and

$$\ell(y) - \ell(z) = k\delta \quad \text{for an integer } k \geq 0.$$

This is a divisibility and sign test; when $\delta = 0$ it is simply equality of offsets.

For a full word on a fixed interval, each occupied phase site moves as $u + k\delta$. Required nonzero symbols give position equalities and symbol tests; required zeros exclude the moving occupied sites from their positions. The resulting finite Boolean combination of affine equalities and inequalities in

one integer $k \geq 0$ is decidable, including an all-zero word. For occurrence anywhere, translation of the entire phase does not change the answer, so a finite prefix and one list of phases suffice. A local predicate allowing several alphabet symbols is a finite union of these tests.

If an escape is encountered before an observed hit, it only disproves the bound. It does not reject a target that might appear after the escape. This is why the unrestricted-reachability conclusion requires the stated full-orbit promise. $\square$

If F is injective on finite configurations, a normalized repeat has no true transient. From $F^j x = \mathcal{T}_\delta F^i x$ and equivariance, cancel F^i to obtain $F^{j-i} x = \mathcal{T}_\delta x$. Thus a wholly bounded-diameter reversible orbit is translating periodic from time zero. If the existence of some diameter bound is promised but no value is supplied, simply simulate normalized shapes until a repeat; the promise ensures termination. Without that promise this is only a semidecision of boundedness.

These arguments are elementary single-orbit consequences of finite normalized shape space. Classical number-conserving CA dynamics already studies bounded-diameter classes and periodic or generalized-shift behavior; see Durand et al. [21] and Formenti–Grange [22]. No new classification theorem is claimed here.

Proposition 43.2 (No computable complete diameter cutoff). *Suppose a fixed F and computable finite input encodings $e \mapsto x_e$ reduce an undecidable predicate to eventual occurrence of an effective finite local observation, possibly anchored. There is no total computable $B(e)$ such that, whenever the observation first occurs at h , every configuration through time h has diameter at most $B(e)$.*

Proof. If such B existed, simulate until a hit, an escape from the bound, or a normalized repeat. A hit answers yes. Escape before a hit answers no by the assumed completeness of B . A normalized repeat permits the exact tail decision in Proposition 43.1, including anchored observations. One of these events occurs in finitely many bounded-width shapes, contradicting undecidability. $\square$

This does not forbid sparse witnesses with large gaps, unbounded integer values, or an externally varying horizon. It forbids a computable complete geometric cutoff through successful runs. A computable complete time or witness-length cutoff would similarly turn the relevant finite search into a decision procedure.

[write] In *canonical-diophantine-certificates* the finite-search step of this proof is the standard argument of Proposition `cdc:bd:prop:nobound` (also used at the end of Part I’s Section 13); the statement, a cutoff on orbit *diameter*, is source 13’s.

43.1 Finite rings and global halting

On a specified n -cell ring, every mass sector is finite, so orbit exploration decides reachability. Its cardinality is

$$[z^M] \left(\sum_{a \in Q} z^{w(a)} \right)^n \leq \sum_{k \leq M} \binom{n}{k} (|Q| - 1)^k.$$

For triple-channel cap K it is $[z^M](1 + \dots + z^K)^{3n}$, at most $\binom{3n+M-1}{M}$. Fixed M makes the bound polynomial in the *numerical* ring length, not necessarily in its binary encoding length. A reversible ring orbit is a cycle from its start.

A line computation from support diameter D_0 is reproduced through a specified horizon T by a ring whose periodic copies have disjoint radius- rT influence intervals; $n > D_0 + 2rT$ suffices. The zero padding and ring length depend on T . This cannot replace the infinite line for all times at once.

An injective dynamics cannot enter an existing fixed point or finite cycle from outside: the cycle already supplies the unique predecessor of each of its points. A reversible simulator therefore reports a halt by an observation while retaining information elsewhere. Stabilization of a finite window, or a departing pulse, must not be called global CA halting.

44 Decidability at total mass at most two

Theorem 44.1 (Two-unit-mass decision theorem). *Under the positive-weight, unique-vacuum hypotheses of Section 43, exact finite-configuration reachability and finite-word occurrence, anchored or anywhere, are decidable from every finite-support initial configuration of total mass at most two. Reversibility is not required. For a fixed CA F , these problems have an algorithm polynomial in the input size when finite configurations are sparse lists of nonzero coordinate/state pairs with binary coordinates, and a pattern is an explicit full finite word, whose length counts toward input size. An anchored word has a binary anchor coordinate.*

The theorem rules out universality by these observation problems at masses zero, one, and two within this model. It does not show that three suffices, or identify a least universal mass. The proof is given in detail because the absolute-position and long-excursion issues are easy to miss.

44.1 Single particles are finite-state walkers

Let $Q_j = \{a \in Q : w(a) = j\}$. From a singleton of type $a \in Q_1$ at position u , positivity and conservation force the next state to be a single weight-one site. Locality puts it within r of u . Thus computable maps

$$\alpha : Q_1 \rightarrow Q_1, \quad d : Q_1 \rightarrow [-r, r] \cap \mathbb{Z}$$

give its next type $\alpha(a)$ and displacement $d(a)$. A type trajectory has a finite transient and cycle. For two hypothetical independent walkers use the product type map on Q_1^2 . Its transient μ and period p satisfy $\mu + p \leq |Q_1|^2$. At each cycle phase $0 \leq s < p$ their positions are

$$L(\mu + s + kp) = L_s + k\Delta_L, \quad R(\mu + s + kp) = R_s + k\Delta_R, \quad k \geq 0. \quad (44.1)$$

Types are fixed within a phase. The displacements over a full product cycle are constants. Reversibility would make the singleton type map a permutation, but no part of the proof needs that simplification.

44.2 Exact acceleration to the next encounter

A mass-two configuration is *near* if its diameter is at most $2r$. Every other such configuration consists of two weight-one sites $L < R$ with gap greater than $2r$. No radius- r neighborhood sees both. Quiescence then implies that the next update is exactly the superposition of the isolated updates, as long as the configuration starts far.

Each step changes the gap by at most $2r$. Starting with gap greater than $2r$, the walkers cannot cross without entering the near region. For each affine phase of (44.1), solve

$$R_s - L_s + k(\Delta_R - \Delta_L) \leq 2r, \quad k \geq 0. \quad (44.2)$$

If the drift is negative, the first admissible k is the appropriate integer ceiling, truncated below by zero. For zero or positive drift, the phase either already satisfies the inequality at $k = 0$ or never

does. Inspect the finite transient too and choose the least resulting physical time. This gives the exact first encounter or proves no encounter occurs.

At that first near time the independently calculated positions are still correct, because the preceding update began far. Stop independent motion there and use the actual CA rule for subsequent behavior. Hypothetical crossings or negative signed gaps after that first time are never used as real dynamics. This boundary is essential for collision correctness.

44.3 Only finitely many encounter shapes

There are exactly

$$N_{\text{near}} = |Q_2| + 2r|Q_1|^2 \quad (44.3)$$

normalized mass-two near shapes: one weight-two site, or an ordered pair of weight-one types at separation $1, \dots, 2r$. Record each near shape together with its time and absolute offset. From a previously unseen near configuration, perform one actual CA step. If the result is still near, continue directly. If it is far, accelerate to its first subsequent near encounter by (44.2), or conclude that this is a terminal forever-far segment.

A weight-two singleton may split, but its next diameter is at most $2r$ and is included. A near pair may merge into one weight-two site. Simultaneous collision, exchange, crossing, and coincident outputs are all handled by the actual CA update rather than a proposed particle collision law. A step starting near has diameter at most $4r$, so every newly departing far excursion starts with gap at most $4r$.

After at most $N_{\text{near}} + 1$ near visits, either a normalized near shape repeats or a terminal forever-far segment has been found. Every finite excursion is computed exactly. A repeated near shape gives the translating periodic tail (43.2). This proves termination of the compressed orbit construction. If $r = 0$, there are no separated-pair interactions, the near-pair term in (44.3) is zero, and each occupied site evolves locally; the same singleton and phase arithmetic handles the case directly.

Mass zero is the stationary vacuum. Mass one is the finite-state walker already described. These complete the orbit descriptions needed for Theorem 44.1.

44.4 Testing every intermediate time

Skipping an excursion without checking its intermediate configurations would not decide reachability. On a phase of a far segment restrict the integer k in (44.1) to its exact interval, which may have no upper bound. The two types a_s, b_s are fixed.

For an exact target with two weight-one sites (u, a) and (v, b) , $u < v$, require the type matches and solve

$$L_s + k\Delta_L = u, \quad R_s + k\Delta_R = v$$

for a common admissible integer k . Incompatible mass or occupied-site counts reject the phase. A weight-two singleton is checked at actual near states.

For a full word on an anchored interval $I = [A, B]$, there are only four possibilities relevant to a far segment:

1. More than two nonzero symbols, or a weight-two nonzero symbol, cannot match
2. Two nonzero symbols require the two matching type and position equations

3. One nonzero symbol at u requires a matching walker there and the other walker strictly outside I
4. An all-zero word requires both walkers strictly outside I

Each case is a finite union of intersections of linear equalities and inequalities in one integer k . Solve them by interval intersection, ceiling/floor operations, and divisibility. This also treats the mass-one walker by omitting the second position.

For occurrence at an arbitrary translate, anchor any nonzero word at a matching occupied site. Two nonzero word symbols require their specified relative gap and types. With one nonzero symbol the other particle must lie outside the corresponding translated interval. An all-zero finite word always occurs sufficiently far from finite support. Again the tests reduce to one-variable affine arithmetic. An empty word, if allowed, occurs everywhere by convention.

A later translated copy of a repeated near loop may hit an anchored target that the first loop missed. Therefore do not stop at a failed first copy. Expand the finite loop into its phases and apply (43.2) to every phase, including all intermediate excursion times; the offset arithmetic then searches every later copy. Unanchored queries need only one copy. This establishes correctness of every stated observation decision.

44.5 Why the fixed-rule algorithm is polynomial

Fix F . Alphabet size, radius, product-type transients and periods, and the number of near shapes are constants. Only the initial far gap may be arbitrarily large. Formula (44.2) computes its first encounter, if any, using binary arithmetic; that time is at most $O(g + 1)$ with a rule-dependent constant, but the algorithm does not simulate g steps. The resulting coordinates have input-linear bit length.

Every subsequent returning excursion begins with gap at most $4r$. During the constant-length product-type transient and one cycle, gap changes by a rule-dependent constant. If cycle drift is nonnegative and this inspected prefix has no encounter, none will occur. If drift is negative, it is an integer at most -1 , so the remaining number of cycles to encounter is also bounded by a rule-dependent constant. Thus every returning post-encounter excursion has constant duration for fixed F .

There are constantly many distinct near visits before a repeat or forever-far tail. Accordingly the compressed orbit has constantly many affine-phase pieces, and every repeating near loop has constant length. The target tests use numbers of input-linear bit length and a fixed number of arithmetic variables. Scanning the explicitly supplied word and performing those tests is polynomial time. No optimal exponent or uniform complexity bound in the description size of F is claimed. Binary encoding of a gigantic gap is crucially different from listing every empty site in that gap.

This proof resembles classical one-counter reachability: after quotienting translation, a separated pair has one unbounded gap, finite internal control, and exceptional dynamics only near a fixed threshold. The direct argument additionally retains absolute displacement for anchored observations. Haase et al. [23] provides related one-counter context; it is not cited as stating this precise CA theorem. A targeted search did not establish priority of the exact statement.

44.6 What particle bounds from the literature do and do not say

Alhazov and Imai’s 2016 paper on particle complexity [24] is directly relevant. Its verified abstract describes finite number-conserving CA with nonnegative integer states and finite total numerical sum, and announces a universal construction with five particles and a sufficiently large finite radius. It also announces a radius-one construction based on two-register machines. The abstract does not assert reversibility. These are finite-mass infinite-line claims, not finite-ring universality.

Only the abstract-level model and upper-bound statements are relied on here; the full construction and any optimality claim have not been verified for this report. In a numerical-state NCCA with weight equal to the state value, there is only one weight-one state. The broader weighted-alphabet model of Theorem 44.1 allows multiple internal types of weight one. A small-mass construction in that broader model cannot automatically be quoted as a numerical-state particle bound. Neither the five-particle literature statement nor the mass-two decision theorem establishes a three-particle reversible universal model or a minimum universal mass.

45 Horizon-free semilinearity at mass at most two

The low-mass decision argument supports a stronger Diophantine conclusion. Unlike the generic sparse certificate, the following result has no supplied horizon and has fixed witness arity after the rule and its arithmetic description are fixed. It applies to a decidable subclass, so it creates no fixed-arity statement for arbitrary universal computation.

Theorem 45.1 (Effective timed and untimed semilinearity). *Fix a CA satisfying the hypotheses of Theorem 44.1, and fix the ordered nonzero labels of an initial configuration and a target, each of mass at most two. The numerical-coordinate relation*

$$\mathcal{R}_F(\mathbf{x}, \mathbf{y}, T) \iff T \in \mathbb{N} \text{ and } F^T(x) = y$$

is effectively Presburger-definable. Its projection $\exists T \mathcal{R}_F$ is also effectively Presburger-definable. Equivalently these are effective semilinear coordinate relations, with signed coordinates interpreted by orthants or the canonical natural encoding below. Taking a finite union allows the labels to vary over their finite allowed cases.

Here Presburger arithmetic has addition, order, integer constants, and quantification, but no multiplication of two unbounded variables. Semilinearity concerns the numerical coordinates, not a language of their binary digit strings. The proof must show why the initial unbounded gap never becomes a variable period or a variable coefficient in a later phase.

45.1 A linear guard for the entire independent prefix

Fix the initial ordered weight-one types at positions $u < v$, with $g = v - u > H_0 = 2r$. Let λ_i, ρ_i be the cumulative independent displacements at time i , starting at zero. For the product-type transient μ and period p , put $i_s = \mu + s$ and write

$$\begin{aligned} t &= i_s + pk, \\ L(t) &= u + \lambda_{i_s} + D_L k, & R(t) &= v + \rho_{i_s} + D_R k, & k &\geq 0. \end{aligned} \tag{45.1}$$

All displayed coefficients except u, v, k are fixed by the rule and initial types. Set $c_i = \rho_i - \lambda_i$ and $D_* = D_R - D_L$. The gap at phase s is $g + c_{i_s} + D_* k$.

Define $\text{Safe}(g, T)$ to mean that every gap at $0 \leq h < T$ exceeds H_0 . This ensures that actual motion agrees with the independent motion at time T , including a possible first near time. It is already a Presburger statement by quantifying h and expanding its finite phase alternatives. A quantifier-free finite guard in the chosen phase is also explicit.

If $T = i < \mu$, require $g + c_h \geq H_0 + 1$ for $0 \leq h < i$. If $T = \mu + s + pk$, first impose those inequalities for all $h < \mu$. For a cycle phase $j < s$, earlier occurrences have indices $0, \dots, k$; for $j \geq s$ they have indices $0, \dots, k-1$, an empty set if $k = 0$. Minima of these arithmetic progressions give the remaining guards:

Drift	$j < s$	$j \geq s$
$D_* \geq 0$	$g + c_{i_j} \geq H_0 + 1$	$k = 0$ or $g + c_{i_j} \geq H_0 + 1$
$D_* < 0$	$g + c_{i_j} + D_*k \geq H_0 + 1$	$k = 0$ or $g + c_{i_j} + D_*(k-1) \geq H_0 + 1$

These are Boolean combinations of constant-coefficient affine inequalities. At time zero all earlier-index sets are empty, as required. Conjoin the applicable guard, the time equation, the current type matches, and target-coordinate equations from (45.1). A finite disjunction over phases describes every target time before or at first near entry, as well as all times on a forever-free initial flight.

45.2 First entry with the absolute position preserved

For $r \geq 1$ the first near entry from an initially far pair has two distinct weight-one sites and gap $e \in \{1, \dots, H_0\}$. A step from gap at least $2r + 1$ leaves gap at least one, so a merger or crossing cannot skip this boundary. There are finitely many entry shapes E .

For a transient entry time $\tau = i$, impose its Safe guard, phase labels, and

$$g + c_i = e, \quad \xi = u + \lambda_i, \quad \tau = i.$$

For a periodic entry at $\tau = \mu + s + pk_0$, impose its Safe guard, $k_0 \geq 0$, phase labels, and

$$g + c_{i_s} + D_*k_0 = e, \quad \xi = u + \lambda_{i_s} + D_Lk_0, \quad \tau = \mu + s + pk_0. \quad (45.2)$$

Here ξ is the absolute leftmost entry coordinate. The guard enforces *earliest* entry, rather than an arbitrary later meeting of hypothetical walkers. Every actual first entry satisfies one such case; conversely the guard and bounded displacement prove that each satisfying case is the real first entry. Thus the full relation between initial coordinates, entry shape, time, and absolute offset is Presburger.

45.3 Finite rule-fixed orbit templates after entry

For each normalized near shape E at offset zero, preprocess its orbit by the near/far algorithm of Theorem 44.1. There are only N_{near} such shapes, each fixed by the rule. Every returning excursion has rule-dependent constant duration; preprocess until a normalized near repeat or a terminal nonreturning free flight. Expand the finite returning flights and one complete recurrent loop when present.

This gives a finite collection of templates, each with fixed occupied-site count, fixed ordered labels, and integer constants $\theta, \pi, \eta_j, \delta_j$, representing

$$\text{elapsed time} = \theta + \pi n, \quad \text{site } j = \eta_j + \delta_j n. \quad (45.3)$$

A listed isolated time fixes $n = 0$; an infinite template has $n \geq 0$ and $\pi > 0$. A translating loop has the same displacement for every particle in a phase. A terminal free flight has the finite-state phase templates already used above, possibly with different left and right drifts. The occupied-site order holds on every represented time, by translation for a loop and by nonreturn for a free tail.

Starting E at offset ξ adds ξ to all positions. Compose (45.3) with (45.2):

$$T = \tau + \theta + \pi n, \quad y_j = \xi + \eta_j + \delta_j n.$$

Substitution gives affine expressions in the unbounded variables k_0, n . There is no product $k_0 n$: the entry shape has been chosen from a *finite* set, so every subsequent period and drift is a rule-fixed constant, independent of the initial gap. A finite union over entry cases and templates therefore remains Presburger. Targets before or at entry were already covered; these compositions cover every later target. Overlap of descriptions is harmless for definability.

An initially near input selects one of the same finite templates directly, with its initial offset and $\tau = 0$. A mass-two singleton can split and a pair can merge, so one discards mismatched *template* site counts rather than assuming the occupied-site count is conserved. A total-mass mismatch is impossible. Vacuum and mass-one cases have stationary or finite-state affine descriptions. At radius zero, each occupied coordinate is fixed and its label follows a finite map, giving a finite transient and periodic label condition. These arguments prove the timed part of Theorem 45.1; existentially quantifying T proves the untimed part.

45.4 Effective elimination and canonical signed input

Standard effective Presburger quantifier elimination converts the constructed formula into a quantifier-free Boolean combination of affine inequalities and fixed-modulus congruences; see Cooper [25]. The effective equivalence with semilinear sets is classical, as in Ginsburg–Spanier [7]. These standard results are used for the final arithmetic normal form, rather than assumed as a CA-specific reachability theorem.

Encode each signed coordinate a by natural inputs (a^+, a^-) with

$$a = a^+ - a^-, \quad a^+ = 0 \text{ or } a^- = 0.$$

This gives exactly one representation of each integer. Include this domain condition and strict ordering of occupied coordinates in the predicate before elimination. With at most two initial and two target sites, there are at most eight natural coordinate inputs, and at most nine external natural inputs when T is retained. For untimed reachability, eliminate T itself before the next compiler. Leaving it as a witness would generally destroy uniqueness when a target is visited at multiple times.

46 Canonical fixed-arity quartics for the low-mass relation

Here is a complete deterministic compiler from a fixed quantifier-free Presburger predicate $\Phi(\mathbf{z})$ on natural inputs. Normalize its atoms to $L(\mathbf{z}) \geq 0$ or $L(\mathbf{z}) \equiv 0 \pmod{d}$, where L is affine with integer coefficients and $d \geq 1$ is fixed. Equality becomes two inequalities and strict inequality a shifted weak inequality. Express the Boolean structure by AND, OR, and NOT gates. Let I, C, G be the actual numbers of inequality atoms, congruence atoms, and Boolean gates in the chosen circuit, respectively. Identical nodes may be shared or repeated; the counts refer to the circuit actually used.

46.1 Inequalities and congruences have unique arithmetic witnesses

For $L \geq 0$ introduce natural b, s with residuals

$$b(b-1), \quad L - (2b-1)s - b + 1.$$

They force $(b, s) = (1, L)$ when $L \geq 0$, and $(b, s) = (0, -L-1)$ otherwise. Thus both witnesses exist uniquely for every input, and b is the truth bit even when the atom is false.

For $L \equiv 0 \pmod{d}$ introduce six natural witnesses q^+, q^-, a, h, b, s and five residuals

$$\begin{aligned} q^+ q^-, \quad L - d(q^+ - q^-) - a, \\ a + h - (d-1), \quad b(b-1), \quad a - (2b-1)s - b. \end{aligned} \quad (46.1)$$

The first three impose the unique signed Euclidean quotient $q = q^+ - q^-$ and remainder $0 \leq a < d$. The product zero condition gives the unique sign-separated quotient representation; $h = d - 1 - a$. The last two force $b = s = 0$ if $a = 0$, and $b = 1, s = a - 1$ if $a > 0$. Therefore the congruence truth bit is the affine expression $1 - b$. All witnesses are uniquely fixed, including negative quotients and modulus one.

46.2 Boolean gates and the exact ledger

For each Boolean gate introduce one natural output z_g and its corresponding residual

$$\begin{aligned} \text{AND} : \quad z_g - uv, \\ \text{OR} : \quad z_g - u - v + uv, \\ \text{NOT} : \quad z_g - 1 + u. \end{aligned}$$

Inputs are preceding outputs, atom truth bits, or constants zero and one. They are already Boolean, so each gate output is uniquely its correct Boolean value. Add one residual $o - 1$ for the final output o . Summing the squares of all residuals gives a polynomial of degree at most four with exactly

$$K_\Phi = 2I + 6C + G \quad \text{natural witnesses}, \quad E_\Phi = 2I + 5C + G + 1 \quad \text{residuals}. \quad (46.2)$$

Every input has exactly one complete assignment to the atom and gate witnesses. The final residual accepts it exactly when Φ is true. Thus the natural fiber is a singleton for true inputs and empty for false inputs. This does not rely on a product of branch equations with unconstrained inactive witnesses.

Corollary 46.1 (Horizon-free canonical low-mass certificates). *For each fixed CA and ordered label case in Theorem 45.1, timed and untimed exact reachability admit effectively constructible integer polynomials of degree at most four with zero or one natural auxiliary witness tuple. Their witness arity is the rule- and compiler-dependent constant K_Φ in (46.2), after effective quantifier elimination and a chosen Boolean circuit. No supplied time bound is required.*

These constants may be extremely large. The report neither instantiates a quantified-formula eliminator for every possible rule nor claims a small universal numerical bound, efficient uniform preprocessing, or an optimal arity. Total arity is the number of external natural inputs plus K_Φ , at most $9 + K_\Phi$ timed or $8 + K_\Phi$ untimed for fixed labels. Mass two does not mean two witnesses. Coefficients and their bits, formula length, and Boolean-circuit size remain paid parts of a concrete instantiation.

Uniqueness here comes from the *post-elimination arithmetic compiler*, not from CA determinism: an untimed target may be reached at many or infinitely many times. No corresponding real or rational witness characterization is claimed. No analogous semilinearity or finite-fold statement for arbitrary recursively enumerable sets follows from this decidable low-mass result.

[write] **A second route, at a higher degree.** Corollary 46.1 rests on the general fact that a semilinear set has a single-fold Diophantine representation, and for that fact the collection already has a sharper statement: Theorem `cdc:of:thm:singlefolds1` of Part XI of *canonical-diophantine-certificates* gives, for every semilinear set, an integer polynomial of degree at most *two*, nonnegative on the real nonnegative orthant, with exactly one natural witness for each member and none otherwise, built in the same way (Presburger quantifier elimination, then deterministic evaluation of the order and congruence atoms, with the comparison of `cdc:of:lem:unboundedcompare` and a Euclidean congruence gadget, and Boolean gates). The compiler of this section is a second route to that fact, of degree four (squared residuals) rather than an orthant-nonnegative quadratic, which carries the exact ledger (46.2). What is source 13's here is the semilinearity of the low-mass relation (Theorem 45.1), to which either compiler applies. Theorem 45.1 is related only loosely to Part I's rigidity theorem (Theorem 13.1): there a fixed globally nonnegative quadratic forces a semilinear zero set, here low-mass dynamics forces a semilinear reachability relation.

47 Reproduction and evidence

The accompanying ZIP contains editable LaTeX, the rendered PDF, standard-library Python implementations, coefficient-explicit fixtures, completed finite local tables, structured test receipts, and a SHA-256 manifest. It contains no copies of third-party papers. Source links and exact attribution limits are given in the bibliography and provenance document.

47.1 What each layer of verification establishes

The producer compiler constructs each sparse polynomial as a canonical list of integer-coefficient monomials. An export includes the public input/table descriptor, variable and residual ordering, canonical witness values, row aliases, degree and scalar ledgers, and every residual coefficient. Its bound verifier regenerates the declared circuit and compares it before evaluating the witness. This catches deletion of equations, changed input binding, changed row aliases, altered accounting, and malformed scalar types. Merely checking that a supplied witness satisfies some unbound list of equations would be weaker. A caller seeking one particular input must still compare the public descriptor with that intended input; no file can infer an unstated intention.

An independent implementation separately constructs the preferred row-selector polynomials. Its cross-check compares residual coefficient multisets under an explicit wire-name correspondence, not only decoded trajectories or final zero scores. Separate exact dynamics tests compare sparse and dense physical evolution. A third set of source checks verifies the corrected rule schemes, legal source macrosteps, mass-fiber completion, and the false-observer example. Finally, the low-mass audits test encounter acceleration and query arithmetic; they are not exhaustive tests of all CA rules.

Finite tests support the implementations and expose edge cases; the general correctness results rest on the proofs in this report. None of the code constitutes a proof-assistant formalization, an exhaustive literature search, or a test of every natural witness.

47.2 Selected literal fixtures

Fixture	V	R	Multiplications	Additions
Binary three-way collision	288	243	1,614	1,424
Same, orthant-exact	288	261	1,830	1,559
Gap 10^{100}	108	88	540	495
Mixed ternary mass	610	370	3,694	3,287

All counts use the literal convention (40.1). The huge-gap example has maximum witness bit length 334 and residual-coefficient bit length 333, illustrating that sparse dimension can remain small while geometric values and coefficients grow. The mixed-mass fixture has largest witness bit length four. The orthant collision fixture adds eighteen residuals, exactly $2MT$.

The restricted-row doubling fixture is bound to the full completed local table but uses the declared 97-row simulation subset. It verifies an actual first pulse, rather than merely simulating a source trajectory outside the polynomial. The larger independently replayed $n = 2$, $T = 41$ case has 257,872 witnesses, 157,683 residuals, 1,263,585 nonzero residual terms, 1,694,649 literal multiplications, and 1,421,268 additions. Its largest witness is 45. Both coefficient-explicit instances are shipped. The smaller $n = 0$, $T = 7$ instance has 45,712 witnesses, 28,585 residuals, 222,845 residual terms, 299,323 multiplications, and 251,430 additions. Its largest witness is 25. The counts for each are generated and checked by the included script.

47.3 Audit coverage

The independent dense/sparse dynamics tests cover 13,200 randomized steps and 73,728 steps across all 36 binary conservative local permutations, for 86,928 comparisons. The independent preferred literal compiler checks 36 instances containing a total of 17,058 witnesses and 9,663 core residuals, and verifies the $2MT$ orthant increment. The earlier factorized independent compiler checks are retained separately with their own ledgers; those results are not relabeled as preferred-backend tests.

The preferred producer/independent coefficient cross-check matches 54 compilers, 21,096 witnesses under explicit renaming, and 11,844 residuals coefficientwise. It additionally checks 20 uniform-loader instances, 24 paid loader-plus-observer instances (12 valid first pulses and 12 correctly rejected earlier pulses), six restricted-row coefficient comparisons, and ten accounting or declaration mutations. Restricted-domain exits and a restricted table omitting the vacuum are rejected. These tests use the final preferred backend, not the older factorized compiler.

The source audit exhaustively enumerates 373 valid three-state source machines under its stated syntax and endpoint conventions, checking 8,672 legal macrosteps with both counters between zero and three. It also checks 3,000 seeded random source machines with 2–12 states for local properties, and 22,004 legal macrosteps on the first 300. Every rule family is exercised. There are 672 startup-signal predecessor tests. The doubling source is replayed for $n = 0, \dots, 10$, including exact microtimes, final counters, and persistent result centers. The nonuniversal doubling and false-signal completions contain 17,576 and 9,261 rows respectively.

The independent mass-two arithmetic audit checks 93,663 cases: 13,656 exhaustive and 20,000 random encounter instances; 20,000 anchored-word, 20,000 unanchored-word, and 20,000 exact-configuration queries; and seven large-integer or unbounded cases. A separate encounter regression checks 6,750 far segments and a 100-digit encounter. These are tests of the affine arithmetic used in the proof, not claims about that many distinct CA rules.

The semilinearity component regression checks 1,697,593 cases: 484,788 exhaustive and 1,210,000 random free-prefix guards, four hundred-digit time cases, 81 inequality and 1,452 signed congruence bounded-fiber cases, twelve Boolean truth-table cases, 256 composed-polynomial inputs, and 1,000 single-variable mutations. An independent audit checks another 837,150 arithmetic and actual-CA cases, including 394,740 timed configurations with splitting, merging, recurrent near templates, and terminal free flights. Both audits include exact affine and negative-quotient edge cases. Neither implements arbitrary-rule quantifier elimination or constitutes formal verification.

The release runner regenerates outputs in a separate working directory and compares stable results with the shipped receipts. Its README lists commands, dependencies, and the optional larger reproduction. The manifest detects accidental modification; it is an integrity record, not a replacement for the mathematical or semantic checks.

[write] **The release in this report.** The programs are shipped as `code/13-sparse-lattice-*`, the receipts, generic fixtures and the two completed tables as `data/13-sparse-lattice-*`, and the provenance record as `13-sparse-lattice-SOURCE-PROVENANCE.md`; the delivered sub-directories `replay/`, `receipts/` and `tables/` are flattened, and the three programs named `audit.py` keep their directory in their shipped names (README of this report). The two completed tables (`data/13-sparse-lattice-doubling-complete.csv` and `data/13-sparse-lattice-false-signal-complete.csv`) are kept byte for byte with their delivered CRLF line ends. The two coefficient-explicit Morita fixtures named above (the $n = 0$, $T = 7$ and $n = 2$, $T = 41$ instances, 0.84 and 4.37 MB compressed) are *not* shipped: the shipped generator rebuilds them in about 14 s and 94 s with identical decompressed payloads, and the README gives the commands; their receipts are shipped. The delivered manuscript (six L^AT_EX files), PDF, README, checksum ledger, packager and one byte-identical copy of the Morita audit are not shipped. The programs need the delivered layout and POSIX (LF) newlines, and must be run on a copy (README). Since batch 80 the corrected code edition (next paragraph but one) adds the exactness suite `code/13-sparse-lattice-test_poly_exactness.py`, its receipt `data/13-sparse-lattice-poly-exactness.json` and the correction note `13-sparse-lattice-CORRECTION.md`.

[write] **The repository’s review.** The programme reviewed this archive in full [30]: it found the generic history theorem, the loader and observer, the corrected source interface, the diameter barrier, the mass-two decision proof and the semilinearity argument sound within their stated domains, regenerated both large Morita fixtures, and ran both complete eighteen-stage replays. Its one finding is a low-level exactness defect: the public Poly constructor of `sparse_mass.py` accepts floating coefficients and mutable terms, so a hand-built floating residual can round -1 to 0. The high-level compiler and the JSON decoder already require exact coefficients, so the finding refutes no theorem and no exported certificate. The review’s repair is the patch `sparse_mass_exact_polynomials.patch` in the programme’s directory [30]; the shipped `code/13-sparse-lattice-sparse_mass.py` was the delivered, unpatched file until batch 80 (see the paragraph after this one). Two later reductions of the programme bear on this Part’s ledgers without changing its theorems: a wire projection of the preferred compiler with a fixed initial configuration [32] removes $T[7M + M(M - 1)]$ natural variables and as many rows, leaving $V = T[M(S + 7) + 4M(M - 1)]$ and $R = T[10M + 4M(M - 1)]$ with quadratic residuals and a bijection of natural zero sets; and a five-witness congruence atom [33] lowers the witness count (46.2) of the post-elimination compiler from $2I + 6C + G$ to $2I + 5C + G$ at the same residual count. Both are the programme’s, are not re-proved here, and are not gate counts.

[write] **Corrected code edition (added 2 October 2026, batch 80).** `Sparse_Lattice_Diophantine_Certificates_corrected.zip` (arrival 4e270aa46, batch 80, manuscript 07; placed by 8a4e64732, batch 80, cluster K1; the archive dates its correction 3 October 2026) repairs the review’s

finding; its authors read the review’s commit but did not execute its code. `Poly` now validates directly supplied terms: immutable tuples, exact integer indices and coefficients (not Booleans, floats or subclasses), nonzero coefficients, sorted indices, distinct sorted monomials; `Poly.make` remains the canonicalizing constructor, and negative (parameter) and repeated (power) indices and the zero and constant polynomials stay valid. The new suite `code/13-sparse-lattice-test_poly_exactness.py` rejects 24 malformed constructor categories and the floating false zero (the exact residual at $2^{60} + 1$ is -1 , with score 1), checks canonicalization and mutation isolation, and compares 2,700 exact expressions on 300 seeded pairs; with `-baseline-module` it reproduces the defect on the original (residual 0.0, accepted witness) and compares the same expressions with the original producer. The release runner runs it first (nineteen stages with `-regenerate-source`, eighteen before). The repair is written independently and is equivalent to the programme’s patch: the patched original passes the suite and the unpatched original fails it. The patch is not to be applied to the shipped file, which already contains the repair (applied anyway, it would insert a second constructor check). The manuscript, the fixtures, both tables, every receipt value other than the producer hash and the new stage, and every theorem and ledger of this Part are unchanged; at the batch-79 write the shipped file was the original, and since batch 80 it is the corrected edition’s. The correction note is `13-sparse-lattice-CORRECTION.md`, and the shipped provenance record `13-sparse-lattice-SOURCE-PROVENANCE.md` now carries the corrected edition’s dated preface. The programme’s correction audit [36] confirms the repair, that removing the new guard leaves the module’s syntax tree identical to the original’s, and the unchanged mathematics.

48 Further questions and boundaries

The strongest next steps are narrowly defined rather than general claims of universality improvement.

1. **Print one safe universal source.** Choose and export a concrete universal counter simulator, retain the history and prime-encoding invariants, and provide its actual m , complete local table, valid input encoding, and proof of reachable legality. This would replace the present existence interface with a fully numerical instance
2. **Compress sparse microtime when structure permits it.** The certificate pays one layer per physical lattice step. Large counter gaps create long round trips. An exact arithmetic macrostep compiler for the source could remove these journeys without changing the finite-mass semantics, but would have to pay for branch tests, timing, and valid intermediate routes
3. **Separate mass, state, radius, and degree costs.** The row-selector simplification reduces residual count dramatically while retaining q^3 witnesses and long sums. Other explicit lookup or collision organizations may trade these resources. None of the displayed counts is a lower bound or minimality record
4. **Resolve small-particle model comparisons.** Check the full particle-complexity constructions and their precise observation, state-weight, radius, and reversibility conventions before claiming a sharper universal mass threshold. The mass-two theorem is a rigorous exclusion in its stated positive-weight model; it is not a proof of a matching upper bound
5. **Keep exact targets separate from halt signals.** A reduction to a specified complete target needs its own output-normalization argument. The safe-source pulse interface alone cannot supply it

The construction gives a family of finite-horizon Diophantine certificates with explicit variable

growth. Treating T as an additional unknown in a single fixed-arity universal equation, or hiding all horizons behind an unspecified MRDP representation, would be a different result and would not inherit these scalar counts. No such transfer is made for the universal higher-mass sparse family. The horizon-free fixed-arity construction in Sections 45–46 is instead for the decidable mass-at-most-two subclass and pays its actual post-elimination circuit size.

[write] Two of these questions touch questions elsewhere in the collection. The first is the lattice counterpart of Part III’s literal source: Part III prints a complete universal conservative signal machine, while source 13 deliberately leaves its safe universal source as an existence construction. The certificate of Theorem 38.1 also bears on the question “Local certificates on adaptive support geometry” of Part XVI of *canonical-diophantine-certificates*, which asks for a sparse lattice support checked canonically at a cost proportional to support and boundary, paying for sorting, distinctness and nearest-neighbour tests: source 13 pays exactly those costs (pairwise equality and a bubble-sorting network) for one-dimensional conservative partitioned automata, at size $O(T(M^2 + Mq^3))$, which is not proportional to support plus boundary, and not for sandpiles. That question stays open.

A Part I: a minimal worked use of the compiler

The following code can be run from the archive root. ([write] “The archive root” is the delivered layout, with `code/signal_certificates.py`; in this report the module is shipped as `code/07-collision-geometry-signal_certificates.py`, so the snippet runs in a copy that restores the delivered name (README of this report).) It builds the first-collision tournament and constructs a complete certificate without using an external algebra system.

```
import sys
sys.path.insert(0, "code")
from signal_certificates import Machine, compile_skeleton

machine = Machine(
    {"A": 2, "B": 0, "C": -1, "H": 0, "R": 0},
    {
        frozenset(("A", "B")): ("H",),
        frozenset(("B", "C")): ("R",),
        frozenset(("A", "B", "C")): ("R",),
    },
)
skeleton = (((0, 1),),)
cert = compile_skeleton(machine, ("A", "B", "C"), skeleton)
gaps = (1, 1)
slacks = cert.witnesses(gaps)
assert cert.evaluate(gaps, slacks) == 0
assert not cert.accepts((3, 1)) # An earlier BC collision.
assert not cert.accepts((2, 1)) # A simultaneous ABC collision.
print(cert.to_dict())
```

The order of rows in the exported matrices is lexicographic and need not match the order used in the displayed mathematical formula. Row order changes only the order of slack coordinates, not the represented relation or the uniqueness statement.

B Part I: dependency map and claim status

Claim	Principal ingredients	Status in this package
Exact chronology chamber	Intercept induction; affine endpoint lemma; static identity and order validation	Full manuscript proof; exact tests
Event-sparse row bound	Local adjacency replacement count	Full manuscript proof; every exported ledger checked
Unique-slack quadratic	Integer strict inequalities; sum-of-squares zeros	Full manuscript proof; zero and mutation tests
Canonical first-hit quartic	Disjoint first-hit chambers; natural one-hot selector equation	Full manuscript proof; finite union tests
Causal denominator and coefficient bounds	Integer speed differences; induction on causal depth	Full manuscript proof; finite denominator checks
Small integral realization	Homogeneity; rational vertex; Cramer’s rule; Hadamard bound	Full manuscript proof
Robustness and codimension	Linear margin estimate; rank of equalities	Full manuscript proof
Finite-horizon semilinearity	Exact chamber theorem; classical Presburger and semilinear-set equivalence	Derived theorem using cited literature
Exact quasi-polynomial counts	Chamber homogeneity; face inclusion–exclusion; classical Ehrhart theorem	Derived theorem; worked counts checked
Universal signal-machine existence	Durand-Lose’s published construction	External result; rule table not reimplemented
Fixed-arity unbounded Diophantine existence	Computable finite-prefix simulation; classical MRDP	External existence theorem, not new compiler
Global convex-quadratic barrier	Positive semidefinite quadratic completion; Presburger decidability	Full manuscript proof

[write] The row “Global convex-quadratic barrier” is a special case of Part XI of [11] (Remark 13.3); source 07’s proof is kept as a second route. The rows “Exact chronology chamber”, “Event-sparse row bound”, “Unique-slack quadratic”, “Causal denominator and coefficient bounds” and “Finite-horizon semilinearity” have second proofs in Part II (Section 1.2).

C Part II: replay and build guide

The accompanying archive contains this article in editable LaTeX, the compiled PDF, exact Python scripts, expected receipts, example coefficient dictionaries, and a manifest with file hashes. It contains no third-party source PDFs. The scripts require Python 3.10 or later and only the standard library. Assertions must remain enabled; do not run Python with `-O`.

From the extracted package root run:


```
python3 scripts/run_replay.py
```

The runner copies the executable scripts into a temporary directory, executes the suites there, compares fresh JSON receipts to the stored expectations, and writes a summary under `replay-output`. The stored expected receipts and packaged examples are not overwritten. Use `-output DIR` to select a different result directory. The worked-example receipt also contains its concrete witness and explicit rejection reason for the false final-cut schema.

[write] These are the delivered names. In this report the five scripts are shipped as `code/11-signal-certificates-*.py`, `build.sh` as `code/11-signal-certificates-build.sh`, the two example files and the manifest as `data/11-signal-certificates-*.json`, and the four expected receipts as `data/11-signal-certificates-expected-*_receipt.json`. The runner looks for `scripts/`, `examples/` and `expected_receipts/` in the parent of its own directory and runs the scripts by their delivered names, so it runs only in a copy that restores the delivered layout (see the README); it writes nothing into the shipped files. Of the archive contents listed above, the editable source is printed as this Part rather than shipped separately, and the compiled PDF and the checksum file `CHECKSUMS.sha256` are not shipped (the checksum file was verified, 17 of 17 files, at placement); the manifest is shipped, and its payload list still names the delivered paths and the delivered PDF and article source.

The main executable interfaces are:

- `signal_geometry.py`: exact simulator, full-layer compiler, polynomial evaluation and rational Gaussian rank calculation;
- `signal_sparse.py`: event-coordinate and adjacency-lifetime compiler;
- `verify_additional.py`: separately seeded edge cases and sharp height checks;
- `verify_examples.py`: the article’s explicit examples and convention checks.

The low-level compilers use assertions to reject malformed schema partitions or structurally impossible collision directions. They are reference mathematical implementations, not hardened parsers for untrusted inputs. JSON residual dictionaries use `@` for the constant term, `x0`, `x1`, and so on for input positions, and the listed variable names for natural witnesses. A certificate evaluates to the sum of the squares of those rows. No expanded polynomial is required to replay it.

To rebuild the article run:

```
sh build.sh
```

A working TeX Live installation with `pdflatex`, Latin Modern fonts, AMS packages, `geometry`, `microtype`, `booktabs`, `enumitem` and `hyperref` suffices. The build script uses fixed document metadata and a fixed source-date epoch; exact PDF byte equality across different TeX distributions is not promised. The mathematical replay receipts are exact and version-independent within supported Python semantics.

D Part II: claim and source guide

The source-supported context is limited to the following claims. All certificate theorems and proofs in this article are supplied here rather than attributed to the literature.

- Durand-Lose 2009: finite configurations in Definition 1; integer inputs in Section 2; speeds in Figure 4; cyclic-tag simulation and halting in Section 5 and Theorem 5. Common-denominator rescaling is our inference, proved in (19.1).
- Durand-Lose 2007: linear-register event simulation in Section 4 of the author version, especially its ninth PDF page; rational models in Section 5. No Diophantine theorem is attributed to it.
- Durand-Lose 2006: energy and live-population bounds in Definition 3 and the following paragraph; accumulation qualifications in Sections 2.2 and 6.
- Ginsburg and Spanier 1966: equivalence of Presburger-definable and semilinear subsets of $\mathbb{N}^n$, Theorem 1.3, pp. 288–289.

Primary sources were checked on 2 October 2026. The bibliography links to the papers or their official bibliographic records; copyright-bearing PDFs are not redistributed in this package.

[write] In this report the Durand-Lose 2009 and Ginsburg–Spanier entries are merged with Part I’s entries for the same papers [3, 7]; the merged entries add source 11’s editors and links to Part I’s.

E Provenance of this report

[write] The report was built from two manuscripts that arrived together in the archives of commit 808b53ed8 (2 October 2026) and were placed by commit 798b0c5d4, which staged their programs, data and provenance files byte-identical to the delivery. (Parts III and IV were added later from two manuscripts of batch 79; their provenance is at the end of this appendix.)

Source	Archive (batch 78)	Pin	Contributed	Printed as
07 (base)	Collision_Geometry_Diophantine_Signal_Machines.zip, manuscript 07; main file <code>collision_geometry/article.tex</code> , 29-page PDF	f1edb38f9	the chamber theorem, the event-sparse bound, the unique-slack quadratic, denominators and heights, small realizations, the NP bound, robustness, semilinearity, the first-hit quartic, Ehrhart counts, the tournament, the clock, the barrier, nine research directions	Part I with Appendices A and B
11	Signal_Machine_Diophantine_Certificates.zip, manuscript 11; main file <code>signal-diophantine-release/article.tex</code> , 15-page PDF	none named	the same theorem with event-coordinate witnesses, halting semantics, full column rank, the size ledgers, the full-layer construction, the four-speed height bound, the bounded shuttle, positive witnesses, five questions	Part II with Appendices C and D

Source 07’s pin is the full commit f1edb38f93aa0a362a34b531a600909f6e74608a, named in its Section 14 and in its delivered PROVENANCE.md (shipped as 07-collision-geometry-PROVENANCE.md); it is an ancestor of the placement commit, and the audit blob it cites is unchanged at the write.

Source 11 names no repository snapshot and cites no repository file; its only repository-related statement, that the 75- and 87-operation constructions are not improved, is true at the write.

Where the merge had to choose.

- *Base and order.* Source 07 is the base (Part I): it allows signed rational speeds without a normalization interface, proves more about the chamber, and is the longer manuscript. Source 11 follows as Part II. Every result, proof, example, remark, research question, appendix and limitation of both sources is printed, in the sources' order and with their numbering inside each Part.
- *The common theorem.* Its two statements have different inputs and witnesses, so both are printed, each with its proof. The lemmas of Part II that coincide with lemmas of Part I (Lemmas 19.1, 20.1 and 21.1, Proposition 26.1) open with a [write] note naming their Part I counterpart, and source 11's proofs are kept as second routes. Section 1.2 compares the two routes, including the two shuttles and the two missed-simultaneity examples, which are kept in both Parts because they are different machines.
- *A result already in the collection.* Part I's Theorem 13.1 and Corollary 13.2 are a special case of Part XI of *canonical-diophantine-certificates*. They are printed with source 07's statements and proofs, as a second route, and credited in Remark 13.3; credit notes were added to source 07's abstract and conclusion. Remark 10.3 relates the first-hit quartic to the same Part.
- *Research questions.* Both agendas are printed in their Parts; Table 3 pairs the coinciding ones, and one [write] note re-scopes the part of Part II's Question 3 that both sources already prove.
- *Front matter and bibliography.* The two title pages are replaced by one; each source's title, author line, abstract and keywords open its Part. Section 1 is new. The two bibliographies are merged: source 11's keys DL09 and GS66 are cited as source 07's `durandlose` and `ginsburgspanier`, whose entries now carry source 11's editors and links; entries [11, 12, 13, 14, 15] were added for the cross-references.
- *Typesetting.* Equations are numbered by section in both Parts (source 07 numbered them consecutively). Source 11's macro `\N` is written `\MN` because source 07's `\N` prints $\mathbb{N}_0$; both print the natural numbers $\{0, 1, 2, \dots\}$ as their sources did. No symbol was renamed in the printed text and no normalization changed.
- *Added text.* Every sentence written in the merge is marked [write]: Section 1; the openings of the two Parts; notes in Part I in Sections 2, 5, 6, 7, 10 (and Remark 10.3), 11, 12, 13 (Remark 13.3), 14 (shipped file names, the pin), 15 and 16, and in its abstract; notes in Part II in Sections 17, 18, 19, 20, 21, 24, 25, 26 and 27; notes in the four source appendices; this appendix; and the bibliography entries marked [write]. Labels were added to the first lemma of Part I's Section 5, to one example subsection, the research directions, the conclusion and the second appendix of Part I, and to several sections, one corollary, the questions, the conclusion and the second appendix of Part II.

Files. The report directory holds this text (`article.tex`, `article.pdf`), a README, source 07's `PROVENANCE.md` and source 11's `REPRODUCIBILITY.md` (as `07-collision-geometry-PROVENANCE.md` and `11-signal-certificates-REPRODUCIBILITY.md`), and the two sources' programs in `code/` and data in `data/`, under the prefixes `07-collision-geometry-` and `11-signal-certificates-`.

The README maps each delivered name to its shipped name and gives rerun instructions that do not overwrite the shipped files. Source 11’s manuscript, both PDFs, source 11’s delivered README and both checksum ledgers (source 07’s `SHA256SUMS`, 14 of 14 files, and source 11’s `CHECKSUMS.sha256`, 17 of 17, both verified at placement) are not shipped; they survive in the archives of commit `808b53ed8`. Source 07’s delivered README was the placed `README.md` and has been replaced by the report README.

Parts III and IV (batch 79). [write] Two manuscripts of batch 79 of ProveIt’s incoming reports were added on 2 October 2026. Both arrived in the archives of commit `aebfa386e` and were placed by commit `a7ae02511`, which staged their programs, data and provenance files byte-identical to the delivery (36 files of source 12, 35 of source 13) and retired the archives from the drop zone. A corrected code edition of source 13, `Sparse_Lattice_Diophantine_Certificates_corrected.zip`, arrived in batch 80 (commit `4e270aa46`) and was placed by `8a4e64732` (batch 80, cluster K1), which replaced five of source 13’s files by their corrected bytes and added three (38 files of source 13 now); nothing printed changes (Section 47).

Source	Archive (batch 79)	Pin	Contributed	Printed as
12	<code>Conservative_Signal_Frontend_Corrected.zip</code> , manuscript 16; main file <code>paper/conservative-signal-diophantine.tex</code> with <code>paper/morita-table.tex</code> , 21-page PDF	none named	the literal universal conservative signal machine, the stack and loader arithmetic, the complete event normal form, the pivot-scaled integer lift, the one-coordinate mode control, the canonical quadratic step and horizon packet with its ledger, the finite mode closure	Part III with Appendices F–H
13	<code>Sparse_Lattice_Diophantine_Certificates.zip</code> , manuscript 19; main file <code>paper/sparse-lattice.tex</code> reading five further files, 27-page PDF; corrected code edition <code>Sparse_Lattice_Diophantine_Certificates_corrected.zip</code> (batch 80, manuscript 07)	none named	the sparse finite-horizon quartic certificate and its ledgers, the orthant-exact option, the corrected Morita–Imai interface, the false observer, the paid loader and first pulse, the doubling example, the diameter barriers, the mass-two decision and semilinearity theorems, the post-elimination quartic, five questions	Part IV

Neither manuscript names a ProveIt commit. Source 12’s delivered `CORRECTION.md` cites the programme’s review commit `9f033fa6e`, an ancestor of the placement, as the origin of its patch. Source 12’s archive is the corrected edition of batch-79 manuscript 07 (`Conservative_Signal_Diophantine_Frontend.zip`, arrival `2a8a39599`): 36 of its 38 files are byte-identical to manuscript 07’s, its article is the same, and its evaluator `code/quadratic_packet.py` is manuscript 07’s with the programme’s patch applied. Manuscript 07 is therefore superseded; nothing of it is shipped, and it survives in its arrival commit. The delivered checksum ledgers verified at placement (manuscript 07, 37 of 37 files; source 12, 44 of 44; source 13, 48 of 48) are not shipped.

Where the batch-79 addition had to choose.

- *Placement and order.* Both manuscripts answer questions of this report (Section 1.11), so they are additions to it rather than a new report; source 12, the signal machine, comes first as Part III, and source 13, its lattice counterpart, follows as Part IV. Every result, proof, example, remark, question, appendix and limitation of both is printed, in the sources' order.
- *The two editions of source 12.* The corrected edition is printed; its article is identical to the earlier edition's. The correction concerns only the generic evaluator program and is described in Section 37.
- *Author lines.* Source 12's title page describes it as a "Research note prepared with AI assistance for private review"; as the report is public, Part III's opening describes it neutrally as an AI-assisted research note, and the delivered wording is recorded here. Source 13's author line, "Research report prepared with AI assistance for Vladimir", is quoted in Part IV's opening.
- *Re-derived results.* Part III's event lemma and uniform lift, Part IV's comparison and sorting gadgets, its post-elimination quartic and the finite-search step of its cutoff proposition re-derive results of Parts I–II or of *canonical-diophantine-certificates*. They are printed as delivered, with [write] notes naming the earlier result, as second routes; no novelty is claimed for them. The selector/copy construction of Part III is credited to Balas and to Jeroslow and Lowe.
- *Front matter.* The title, subtitle and abstract were widened; Section 1 gained Sections 1.7–1.11, and dated [write] notes were added where batch 79 made earlier sentences stale (Sections 1.1, 1.3, 1.4, 1.5 and 1.6, Table 3, Part I's direction 15.2 and Part II's Questions 1 and 2). Nothing was numbered anew: the notation of Parts III and IV is an uncaptioned table, so that the earlier tables keep their numbers, and Part III's three appendices follow this appendix, so that Appendices A–E keep their letters.
- *Labels.* Source 12's 49 labels are `smc:cs:` plus their delivered names, and source 13's 53 are `smc:sl:` plus theirs; none was dropped or renamed apart from the prefix. The labels added in batch 79 (the two Parts and the five subsections of Section 1) also use `smc:cs:`, although the subsections cover both Parts.
- *Bibliography.* Source 12's keys `dl2007` and `dl2006` are cited as the existing entries [4, 5] for the same papers, and source 13's `ginsburgspanier` is the existing entry [7]. The sources' other entries are appended as delivered, marked with their source; the entries for Balas, Jeroslow–Lowe and the programme's reviews are added and marked [write].
- *Typesetting.* Both sources' macro `\N` is written `\NN`; it prints the natural numbers $\{0, 1, 2, \dots\}$ as in the sources. Equations are numbered by section. Source 12's table file is printed inline, two forced page breaks of the sources and source 13's own table of contents (with its formatting of contents lines) are omitted, and the titles of Part III's appendices begin with "Part III:". No symbol was renamed and no normalization changed.
- *Added text.* Every sentence written in this addition is marked [write]: the title-page and abstract text on Parts III and IV; Sections 1.7–1.11 and the dated notes listed above; the openings of the two Parts; in Part III, notes after Lemma 34.1, in Section 35 (the uniform lift, the selector/copy credit), after Theorem 36.1, at the end of Section 37 (the shipped release, the correction and the review), and in Appendices F and H; in Part IV, notes in Sections 38 and 39 (Part V of *canonical-diophantine-certificates*), 41 (the review's check of the sign correction), 43, 46 (the second route), 47 (the shipped release and the review) and 48; this paragraph and the bibliography entries marked [write].

- *Excluded data.* Four regenerable files of the deliveries are not shipped: source 12's compact closure records `numeric/MODES.jsonl.gz` and `numeric/EVENTS.jsonl.gz` and source 13's two coefficient-explicit Morita fixtures `replay/core/fixtures/morita_doubling_n0_pulse_T7.json.gz` and `..._n2_pulse_T41.json.gz` (6.84 MB together). The shipped programs regenerate them with identical decompressed content; the README gives the commands and the retrieval from the arrival commit.

Files added in batch 79. The report directory gained source 12's `CORRECTION.md`, `SOURCE-PROVENANCE.md` and `numeric/PROOF_AND_LEDGER.md` and source 13's `SOURCE-PROVENANCE.md` at its root, with the prefixes `12-conservative-signal-` and `13-sparse-lattice-`, and the two sources' programs in `code/` and records in `data/` under the same prefixes. The README maps each delivered name to its shipped name. Not shipped: both manuscripts and PDFs, both delivered READMEs, the checksum ledgers, source 12's copy of the patch (it is the programme's file [29]), an in-archive duplicate of source 12's machine file, source 13's manifest checker, packager and duplicate Morita audit, the four regenerable files above, and all of manuscript 07.

F Part III: the complete Morita transition data

In the table, bRq_s means write b , move right, and enter q_s ; bLq_s is analogous. A dash is unspecified. The two words halt and null are designated terminal cells, not transitions. Symbol order is the release's digit order $b, Y, N, *, \$, 1$, assigned digits 1 through 6.

[write] The delivered article reads this table from a separate file, `paper/morita-table.tex`; it is printed here inline and unchanged. The same table is shipped as data in `data/12-conservative-signal-MORITA_15_6_TABLE.json`.

State	b	Y	N	$*$	$\$$	1
q_0	$\$Lq_2$	$\$Lq_1$	bLq_{11}	—	—	—
q_1	halt	YLq_1	NLq_1	$*Rq_0$	bLq_1	—
q_2	$*Lq_3$	YLq_2	NLq_2	$*Lq_2$	null	—
q_3	bRq_{10}	$1Rq_4$	bRq_5	bRq_8	—	—
q_4	YRq_6	YRq_4	NRq_4	$*Rq_4$	$\$Rq_4$	—
q_5	NRq_6	YRq_5	NRq_5	$*Rq_5$	$\$Rq_5$	—
q_6	bLq_7	—	—	—	—	—
q_7	NLq_3	YLq_7	NLq_7	$*Lq_7$	$\$Lq_7$	YLq_3
q_8	—	YRq_8	NRq_8	$*Rq_8$	$\$Rq_9$	—
q_9	—	YRq_9	NRq_9	$*Rq_9$	YRq_0	—
q_{10}	—	YRq_{10}	NRq_{10}	$*Rq_{10}$	$\$Lq_3$	—
q_{11}	$*Lq_{12}$	YLq_{11}	NLq_{11}	$*Lq_{11}$	$\$Lq_{11}$	—
q_{12}	bRq_{14}	YLq_{12}	NLq_{12}	bRq_{13}	—	—
q_{13}	NRq_0	YRq_{13}	NRq_{13}	$*Rq_{13}$	$\$Rq_{13}$	—
q_{14}	—	YRq_{14}	NRq_{14}	$*Rq_{14}$	$\$Lq_{12}$	—

G Part III: a small complete quadratic packet

Consider the one-dimensional partial-branch presentation of the total map $F(x) = 2x$ on $\mathbb{N}$: branch 0 has guard $x = 0$ and update 0; branch 1 has guard $x > 0$ and update $2x$. The packet has selectors

b_0, b_1 , copies z_0, z_1 , and one slack s :

$$P(x, y; b_0, b_1, z_0, z_1, s) = (b_0 + b_1 - 1)^2 + (x - z_0 - z_1)^2 + (y - 2z_1)^2 \\ + z_0^2 + (z_1 - b_1 - s)^2 + b_1 z_0 + b_0 z_1.$$

Here $B = 2$, $d = 1$, $E = 1$, $T = 1$, $W = 0$, so the ledger gives five variables, five squared residuals, and two products. For $x = 0, y = 0$ the unique witness is $(1, 0, 0, 0, 0)$; for $x > 0, y = 2x$ it is $(0, 1, 0, x, x - 1)$. Every other input/output pair has no natural witness. The core replay exhaustively checks 100,000 bounded assignments, including every auxiliary tuple in $\{0, \dots, 4\}^5$ for $0 \leq x < 4$, $0 \leq y < 8$.

The generic event-branch compiler also fully enumerates a three-signal transparent test machine with speeds $2, 0, -2$. Its 19 branches have $E = 20$, $T = 84$, $W = 0$, yielding 160 auxiliary variables, 111 squared affine residuals, and 19 complementarity products. Both pivot and uniform scalings pass 144 valid-germ tests, including 88 legal steps and two simultaneous steps, with every canonical residual exactly zero.

This example illustrates the exact polynomial mechanics without suggesting that a universal branch family has only two branches. For the signal frontend, the same constructor takes the actual finite label-mode and first-collision branch descriptions.

H Part III: the complete finite mode closure and numeric packet

H.1 A finite family covering every encoded run

The naive family of all 114^{18} ordered label words is unnecessary. Starting from the literal initial mode, consider each nonempty subset J of eligible adjacent closing pairs. A pair is eligible only when its exact incoming labels have a defined rule and their speeds are in incoming order. Exclude subsets with consecutive edges, since those would constitute an undefined collision of at least three signals. Replace every selected pair atomically by its speed-sorted output pair and add the resulting mode. Repeating to exhaustion gives the least mode-only closure for this syntactic successor operation.

Every legal event of an encoded run uses one of these nonadjacent eligible subsets, by the complete-collision lemma. Induction therefore places every event mode and branch in the closure. Conversely, every retained branch has exactly the original mode, germ, earliest-collision and rule guards, so it introduces no spurious geometric step. Restricting the integer map to this closed mode family preserves every encoded computation.

The closure is an overapproximation of modes numerically reachable from the specified input family: successive graph edges may be realizable at different gap vectors which cannot be spliced into one orbit. No assertion of reachability of every graph node is made. Nevertheless every branch is individually feasible. If c is its speed-difference vector, choose

$$h_i = \begin{cases} c_i & i \in J, \\ \max(c_i, 0) + 1 & i \notin J, \end{cases} \quad u = q \sum_i h_i.$$

All gaps are strictly positive, its next delay is one, exactly J collides, and the output is an admissible germ. The exporter checks this witness for every branch.

The completed enumeration has 49,700 modes and 80,501 branches: 65,557 single-collision branches and 14,944 branches with two simultaneous disjoint collisions. The maximum graph

discovery depth is 321. This depth is a graph statistic, not a computation-time bound or a numeric reachability certificate.

Among the modes, 1,728 have no eligible legal pair. Only 36 of those have no closing edge at all and are collision-free; the other 1,692 are blocked by undefined collisions. Of the 36 collision-free modes, six carry an accepting halt label and 30 carry the null label. The fixed final accepting mode in Section 35 is the target for encoded runs. Neither all graph sinks nor the union of those six label-bearing modes replaces that target.

H.2 Exact guard and witness counts

For each branch the exported guard convention retains one mode equality, $|J| - 1$ nonpivot tie equalities, one strict germ guard for each $c_i \geq 0$, the strict span guard, the strict pivot-gap guard, and all $17 - |J|$ non- J endpoint guards. Some strict guards are redundant; retaining them makes the convention simple and fully reproducible. Nonnegative coordinates are the ambient natural domain, so no weak-gap slacks are introduced.

The resulting exact ledger is

Quantity	Count
Branches B	80,501
Equality guards E	95,445
Strict guards T	2,667,479
Weak guards W	0
Auxiliary variables $19B + T$	4,196,998
All variables including X, Y	4,197,034
Squared affine residuals $37 + E + T$	2,762,961
Complementarity products	80,501

Thus the literal one-step polynomial is fully specified, quadratic, nonnegative on natural arguments, and canonical by Theorem 36.1. For externally fixed K and supplied endpoints, its unrolling has exactly $4,197,016K - 18$ witness variables. This explicit constant is large; the small physical population does not imply a comparably small Diophantine arity.

The untimed affine rows contain 15,323,489 nonzero variable-coefficient entries and one nonzero constant. The maximum absolute affine coefficient has 124 bits. Gap coefficients remain bounded by 24; the larger values arise from finite mode coding. These are sparse affine-circuit statistics, not claims about minimal gate complexity. Adding the optional timestamp coordinate retains the same branch guards and adds one local copy coordinate per branch and two global input/output residuals. Its derived ledger is 4,277,499 auxiliaries and 2,762,963 squared affine residuals per step.

H.3 What the numeric export contains

The mode file gives each mode's 18 label indices, speed differences, exact code and graph index. The compact event file records every branch index, source/target index, canonical pivot and bitmask of the complete set J . Together with the explicit formulas above, these fixed numbers determine every sparse matrix and guard coefficient. The supplied exporter generates the full numeric branch records, including selector, copy and slack indices. A separate circuit file fixes the global variable order and the polynomial (36.7). All large codes and generated coefficients are serialized as decimal strings.

The shared selector sum is an expression, not another witness variable. Writing its difference with b_r is exactly $\sum_{h \neq r} b_h$, which is nonnegative on all natural assignments. Keeping this expression factored avoids a quadratic-in- B expansion of the complementarity part. The exporter can regenerate every squared affine residual row or answer individual expanded coefficient queries. The verbose branch and full residual-row files were generated and checked during preparation. They are omitted from the compact archive because they are deterministically regenerable from its complete mode/event data and code.

The supplementary incidence calculation gives 1,059,563,096,023 nonzero terms in the full monomial expansion and a maximum absolute coefficient of 248 bits. Those expansion statistics are derived exactly; no trillion-term monomial file was emitted. The actually generated full affine-residual export was checked independently, including all 15,323,489 nonzero variable-coefficient entries.

This is an explicit coefficient description of one fixed step polynomial and each fixed-horizon family. It does not convert the unknown horizon into a fixed number of natural variables, and it does not assert that the mode graph's finite size bounds the length of a universal run.

[write] In this report the machine file, the ledger and the circuit file are shipped as `data/12-conservative-signal-MORITA_18_SIGNAL_MACHINE.json`, `data/12-conservative-signal-LEDGER.json` and `data/12-conservative-signal-POLYNOMIAL_CIRCUIT.json`, and the exporter as `code/12-conservative-signal-compile_packet.py`. The mode file and the compact event file (`numeric/MODES.jsonl.gz` and `numeric/EVENTS.jsonl.gz` in the delivered layout) are not shipped; the exporter's `build` command regenerates them with identical decompressed content (README of this report). The programme's review [29] enumerated the same 49,700 modes and 80,501 branches independently and derived every entry of the ledger above from the closure.

References

- [1] Vladimir Reshetnikov. *ProveIt*, especially `Computability/HilbertTenthProblem/README.md`. Repository inspected 2 October 2026. <https://github.com/VladimirReshetnikov/ProveIt>.
- [2] *Reversible finite-history models: a bounded arithmetic audit*. In the `ProveIt` repository, `Computability/HilbertTenthProblem/Papers/1980/REVERSIBLE_FINITE_HISTORY_MODELS.md`. Inspected 2 October 2026; blob `cb31d0a108b9a19fc33da6fa89a73b604e15e260`.
- [3] Jérôme Durand-Lose. Small Turing universal signal machines. In T. Neary, D. Woods, A. K. Seda and N. Murphy, editors, *The Complexity of Simple Programs 2008, Electronic Proceedings in Theoretical Computer Science* **1** (2009), 70–80. doi:10.4204/EPTCS.1.7. <https://arxiv.org/abs/0906.3225>. ([write] Merged entry: source 07’s `durandlose` and source 11’s `DL09`.)
- [4] Jérôme Durand-Lose. Abstract geometrical computation and the linear Blum, Shub and Smale model. In S. B. Cooper, B. Löwe and A. Sorbi, editors, *Computation and Logic in the Real World*, CiE 2007, LNCS 4497, pp. 238–247. Springer, 2007. doi:10.1007/978-3-540-73001-9_25. [Author version](#).
- [5] Jérôme Durand-Lose. Reversible conservative rational abstract geometrical computation is Turing-universal. In A. Beckmann, U. Berger, B. Löwe and J. V. Tucker, editors, *Logical Approaches to Computational Barriers*, CiE 2006, LNCS 3988, pp. 163–172. Springer, 2006. doi:10.1007/11780342_18. [Author version](#).
- [6] Florent Becker, Mathieu Chapelle, Jérôme Durand-Lose, Vincent Levorato, and Maxime Senot. *Abstract Geometrical Computation 8: Small Machines, Accumulations and Rationality*. Preprint, 2013. <https://arxiv.org/abs/1307.6468>.
- [7] Seymour Ginsburg and Edwin H. Spanier. Semigroups, Presburger formulas, and languages. *Pacific Journal of Mathematics* **16**(2) (1966), 285–296. doi:10.2140/pjm.1966.16.285. <https://msp.org/pjm/1966/16-2/pjm-v16-n2-p09-s.pdf>; [publisher PDF](#). ([write] Merged entry: source 07’s `ginsburgspanier` and source 11’s `GS66`, with both sources’ links.)
- [8] Matthias Beck and Sinai Robins. *Computing the Continuous Discretely: Integer-Point Enumeration in Polyhedra*, second edition. Springer, 2015. See the rational-polytope Ehrhart theorem and its generating-function form. Author’s information and updated draft: <https://matthbeck.github.io/ccd.html>.
- [9] Yuri V. Matiyasevich. Enumerable sets are Diophantine. *Doklady Akademii Nauk SSSR* **191** (1970), 279–282. English translation: *Soviet Mathematics Doklady* **11** (1970), 354–357.
- [10] Jonas Bayer, Marco David, Benedikt Stock, Abhik Pal, Yuri Matiyasevich, and Dierk Schleicher. *Diophantine Equations and the DPRM Theorem*. Archive of Formal Proofs, 6 June 2022. https://isa-afp.org/entries/DPRM_Theorem.html.
- [11] [write] ProveIt research report *canonical-diophantine-certificates*, collection `SetTheory/Cardinals/docs/reports/hilbert-tenth-problem/canonical-diophantine-certificates/`; Part XI (order-free self-assembly and a positivity-restricted degree threshold) and Part IX (the boundary of fixed-arity compression). Cited by label names, not by page.

- [12] [write] ProveIt research report *liveness-beyond-halting*, collection `SetTheory/Cardinals/docs/reports/hilbert-tenth-problem/liveness-beyond-halting/`; Part V, the question of its source 11 “Optimal degree under stronger geometric restrictions”.
- [13] [write] ProveIt research programme, *Seven incoming computational-substrate reports*, review `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/incoming_substrate_review_808b53ed8.md`, commit 126028588, 2 October 2026, with its replay script and receipt in the same directory.
- [14] [write] ProveIt research programme, *Independent signal-machine review*, arrival `808b53ed8`, `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/incoming_signal_review_808b.md`, commit 126028588, 2 October 2026.
- [15] [write] ProveIt research programme, *Two-state block automata: exact tables and the finite-input gap*, bounded primary-source audit `Computability/HilbertTenthProblem/Papers/1980/EXPLORATION_BINARY_BLOCK_CA_AUDIT.md`, as at the write of this report.
- [16] J. Durand-Lose, *Abstract geometrical computation 6: a reversible, conservative and rational-based model for Black hole computation*, International Journal of Unconventional Computing, 2012. Sections 3–4 and Figure 2. [Author manuscript](#). (Source 12.)
- [17] K. Morita, *Reversible computing and cellular automata—a survey*, Theoretical Computer Science 395 (2008), 101–131. Author draft, especially Definitions 3.1–3.3, Theorem 3.3, Table 5, and Figure 28: [Author manuscript](#). (Source 12.)
- [18] K. Morita, A. Shirasaki and Y. Gono, *A 1-tape 2-symbol reversible Turing machine*, IEICE Transactions E72(3) (1989), 223–228. [Publisher record](#). (Source 12.)
- [19] K. Morita and K. Imai, *Number-conserving reversible cellular automata and their computation-universality*, RAIRO Theoretical Informatics and Applications **35** (2001), 239–258. https://www.numdam.org/item/ITA_2001__35_3_239_0/. (Source 13.)
- [20] K. Morita, *Universality of a reversible two-counter machine*, Theoretical Computer Science **168** (1996), 303–320. [https://doi.org/10.1016/S0304-3975\(96\)00081-3](https://doi.org/10.1016/S0304-3975(96)00081-3). (Source 13.)
- [21] B. Durand, E. Formenti, A. Grange, and Z. Róka, *Number conserving cellular automata: new results on decidability and dynamics*, DMTCS Proceedings AB (2003), 129–140. <https://dmtcs.episciences.org/2301/pdf>. (Source 13.)
- [22] E. Formenti and A. Grange, *Number conserving cellular automata II: dynamics*, Theoretical Computer Science **304** (2003), 269–290. [https://doi.org/10.1016/S0304-3975\(03\)00134-8](https://doi.org/10.1016/S0304-3975(03)00134-8). (Source 13.)
- [23] C. Haase, S. Kreutzer, J. Ouaknine, and J. Worrell, *Reachability in Succinct and Parametric One-Counter Automata*, CONCUR 2009, LNCS **5710**, 369–383. Author-hosted paper: <https://people.mpi-sws.org/~joel/publications/onecounter09.pdf>. (Source 13.)
- [24] A. Alhazov and K. Imai, *Particle Complexity of Universal Finite Number-Conserving Cellular Automata*, CANDAR 2016, 209–214. <https://doi.org/10.1109/CANDAR.2016.0045>. Publisher abstract: <https://ieeexplore.ieee.org/document/7818615>. (Source 13.)

- [25] D. C. Cooper, *Theorem Proving in Arithmetic without Multiplication*, Machine Intelligence **7** (1972), 91–99. Primary paper scan: https://www.cs.cmu.edu/~emc/spring06/home1_files/Cooper.pdf. (Source 13.)
- [26] [write] Egon Balas. Disjunctive programming. *Annals of Discrete Mathematics* **5** (1979), 3–51.
- [27] [write] Egon Balas. Disjunctive programming and a hierarchy of relaxations for discrete optimization problems. *SIAM Journal on Algebraic and Discrete Methods* **6** (1985), 466–486.
- [28] [write] Robert G. Jeroslow and James K. Lowe. Modelling with integer variables. *Mathematical Programming Study* **22** (1984), 167–184.
- [29] [write] ProveIt research programme, *Conservative signal frontend: complete report and source review*, Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/review_conservative_signal_2a8a39599.md, with the independent patch review review_conservative_signal_independent.md, their checkers and receipts, and the repair conservative_signal_packet_domains.patch in the same directory; commit 9f033fa6e, 2 October 2026. It reviews the earlier edition of source 12 (batch-79 manuscript 07, arrival 2a8a39599).
- [30] [write] ProveIt research programme, *Review: sparse-lattice Diophantine certificates*, Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/review_sparse_lattice_aebfa386e.md, with its checker, receipt and the repair sparse_mass_exact_polynomials.patch in the same directory; commit 9975af7e1, 2 October 2026.
- [31] [write] ProveIt research programme, *Corrected conservative signal frontend: full review*, Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/review_conservative_signal_corrected_aebfa386e.md, with its checker and receipt in the same directory; commit 9df1f72ca, 2 October 2026.
- [32] [write] ProveIt research programme, *Projecting sparse lattice history wires*, Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/sparse_lattice_projection.md, with its program, receipt and independent review review_sparse_projection_aebfa.md; commit 352ec5c1c, 2 October 2026.
- [33] [write] ProveIt research programme, *Five-witness congruence atom*, Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/presburger_congruence_five.md, with its program, receipt and independent review; commit d175245d7, 2 October 2026.
- [34] [write] ProveIt research programme, *Review of the ten-report intake at aebfa386e*, Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/incoming_substrate_review_aebfa386e.md, as at commit 653349f6a (2 October 2026).
- [35] [write] ProveIt research programme, *Exact placement review: a7ae02511*, Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/review_placement_a7ae02511.md, with the layout stager replay_placed_substrates_a7ae02511.py, its receipt and inventory; commit 653349f6a, 2 October 2026.
- [36] [write] ProveIt research programme, *Batch 80: corrected Tree Calculus, reset-net and sparse-lattice packages*, Computability/HilbertTenthProblem/Papers/research-wip/nat

`ive-stream-queue/review_batch80_corrected.md`, with its checker and receipt in the same directory; commit `abfc0cb25`, 2 October 2026. It reviews the corrected code edition of source 13 (arrival `4e270aa46`).